UNIVERSITY
OF OSLO

Master's thesis

Rethinking Multilevel k-way Graph Partitioning using an Objective Function that Considers the Distribution of Communication Volume

Riccardo Ieva

Informatics: Programming and Systems Architecture

Department of Informatics
Faculty of Mathematics and Natural Sciences

Spring 2026

Riccardo Ieva

Rethinking Multilevel k-way Graph Partitioning using an Objective Function that Considers the Distribution of Communication Volume

Supervisors:
Xing Cai
James D. Trotter

Disclaimer

In this scientific work, generative artificial intelligence (AI) has been used. All data and personal information have been processed in accordance with the University of Oslo's regulations, and I, as the author of the document, take full responsibility for its content, claims, and references. An overview of the use of generative AI is provided below.

The service Claude, developed by Anthropic, has been used to improve the content of the thesis. A first version of the work was pasted in its entirety, and the model was given the prompt "Fix my grammar in this master thesis". The text was then iterated a few times through the model where new prompts were used to get the correct structure of the text. The final result was partially cut out, fact-checked, and mostly rewritten by the author.

Abstract

Parallel applications are an increasingly important area of development. The goal of graph partitioners is to equally distribute the load across a parallel computer to improve runtime. Generally speaking, a parallel application is as slow as its slowest process, suggesting that improving the most heavily loaded partition would yield runtime gains.

Most partitioners base their objective functions on global terms like total communication volume, which in turn may leave individual partitions with disproportionately large volume loads. We present NVOL, a new implementation of a partition-aware objective function within the METIS framework. The idea to optimize for the highest partition volume is not new, however, the implementation together with the required calculations to perform it are. With NVOL as the objective function, the refinement algorithm examines each partition's outgoing communication volume and performs only those moves that do not increase the maximum outgoing communication volume.

We evaluate NVOL against METIS's own total communication volume objective function across 20 graphs from two datasets - a set of cardiac finite-element matrices and a selection of matrices from the SuiteSparse Matrix Collection - with each graph partitioned into 2 to 2048 parts. With NVOL, we got a lower maximum outgoing communication volume than METIS's volume objective function in most runs. For the SuiteSparse graphs, it wins 89% of the time (mean improvement 9.2%, peak 56.7%) and for the cardiac finite-element runs 93% of the time(mean 4.7%, peak 17.6%). Additionally, it produces more evenly distributed partition volumes overall. Using an estimate for communication time, parametrized by real machine bandwidths, NVOL yields the lowest maximum communication time per-partition in 65% of our tests.

NVOL yields the best results on numerical simulation graphs and shows good potential for citation networks when partitioned into a sufficiently large number of parts. It also looks promising for improving the communication time of the slowest process in a parallel application. However, the increased complexity of the objective function comes at the cost of partitioning time, averaging 7-11× slower than METIS running the total volume objective function. Even so, NVOL's good results and strong potential make it an interesting area of development.

Preface

Here comes your preface, including acknowledgments and thanks.

Contents

1 Introduction . . . 1
1.1 Thesis Goal . . . 2
1.2 Thesis Structure . . . 3
2 Background . . . 5
2.1 Graph Partitioning Problem . . . 5
2.1.1 Objective Functions . . . 6
2.1.2 Shortcomings of the Objective Functions . . . 6
2.2 Graph Partitioning Algorithms . . . 7
2.2.1 Recursive Bisection . . . 7
2.2.2 Spectral Bisection . . . 8
2.2.3 Graph Growing Partitioning . . . 8
2.2.4 Streaming Graph Partitioning . . . 8
2.2.5 Multilevel Graph Partitioning . . . 8
2.3 Applications of Graph Partitioning . . . 9
2.3.1 Parallel Processing . . . 9
2.3.2 Complex Networks . . . 9
2.3.3 VLSI Physical Design . . . 9
2.3.4 Neural Networks . . . 10
3 Multilevel k-way Graph Partitioning . . . 11
3.1 Preliminaries . . . 11
3.2 Coarsening Phase . . . 12
3.2.1 Sorted Heavy Edge Matching . . . 13
3.3 Initial Partitioning Phase . . . 13
3.4 Uncoarsening Phase . . . 14
3.4.1 Projection and preparation before refinement . . . 15
3.4.2 Greedy Refinement . . . 16
4 Communication Volume Calculation . . . 19
4.1 Basic definitions . . . 20
4.2 VOL gain calculation . . . 21
4.3 NVOL gain calculation . . . 23
4.4 Examples . . . 26
4.4.1 Two partition example . . . 26
4.4.2 Three partition example . . . 28
4.4.3 Case 4 example . . . 30
5 Greedy Refinement Using Communication Volume . . . 33
5.1 GR with VOL . . . 33
5.2 GR with NVOL . . . 34
5.2.1 Encouraging more moves . . . 35

6 METIS with NVOL and Supporting Tooling . . . 39
6.1 METIS . . . 39
6.1.1 Inputs . . . 39
6.1.2 Output . . . 40
6.1.3 Implementing NVOL . . . 40
6.2 Graph Tools . . . 43
6.2.1 translate_to_metis . . . 43
6.2.2 calc_partitions . . . 44
6.2.3 form_statistics . . . 44
6.2.4 graph_results_vol . . . 44
6.2.5 stats_vol . . . 44
6.2.6 model_estimates . . . 45
7 Results . . . 47
7.1 Graphs used for testing . . . 47
7.2 Maximum outgoing volume per partition . . . 48
7.2.1 Heart dataset . . . 49
7.2.2 SuiteSparse Matrix Collection . . . 52
7.3 Tuning NVOL . . . 62
7.3.1 Heart dataset with 4% imbalance . . . 63
7.3.2 SuiteSparse Matrix Collection with 4% imbalance . . . 64
7.4 Communication times estimates . . . 65
7.4.1 coPapersDBLP graph estimate results . . . 67
7.4.2 italy_osm graph estimate results . . . 68
8 Conclusions and Future Work . . . 71
8.1 Conclusions . . . 71
8.1.1 Contributions . . . 71
8.1.2 RQ1 - Does NVOL improve the volume balance? . . . 71
8.1.3 RQ2 - Do the partitions produced by NVOL improve performance? . . . 72
8.2 Future Work . . . 73
8.2.1 Improved efficiency . . . 73
8.2.2 Introduce hardware knowledge . . . 74

List of Figures

2.1 Vertex A has an outgoing communication volume of 1 but an edge-cut of 2. Each color represents a partition. 7

3.1 The phases of the multilevel k-way partitioning algorithm. During the coarsening phase, the vertices are contracted together and the graph size is successively decreased; during the initial partitioning phase, a k-way partition is performed on the smallest graph; and during the uncoarsening phase, the partitions are successively refined and projected back to a larger graph. . . 12
3.2 The process of bisecting a graph into 7 partitions.. 14

4.1 Vertex A has an outgoing communication volume of 2. Each color represents a partition. 20
4.2 Vertex A has an outgoing communication volume of 1. Each color represents a partition. 20
4.3 The total communication volume is 8. 21
4.4 Partitioned graph with a volume of 4, each partition has an outgoing volume of 2. 27
4.5 Partitinoed graph with a volume of 4, each partition has an outgoing volume of 2 after the move. 27
4.6 Partitioned graph with a total communication volume of 8.. 28
4.7 Partitioned graph has a volume of 6 after *B* is moved to the blue partition . . 29
4.8 Partitioned graph has a volume of 8 after *B* is moved to the green partition . 29
4.9 Partitioned graph with a total communication volume of 8.. 30
4.10 Partitioned graph has a volume of 5 after the move. 31
4.11 Partitioned graph has a volume of 6 after the move. 31

6.1 Layout of NVOL's data structures. `graph_t` stores pointers to two parallel arrays of length k (one entry per partition): an array of `pinfo_t` structs holding each partition's outgoing volume, and an array of `vol_refinement_table` structs, each of which points to its own `gain_list` of k integers. Only the first `gain_list` is drawn; the others follow the same pattern. 41

7.1 Standard deviation normalized by the average volume for runs with $k = 2$ on the Heart dataset. VOL (blue) shows nonzero variation across graphs; NVOL (orange) is 0 on every graph, meaning its two partitions have identical outgoing volume.. 50
7.2 Comparison of the outgoing communication volumes of the partitions of heart04 $k = 4$ for VOL (blue) and NVOL (orange). 52

7.3 Percentage difference in $MV^{\max}$ between NVOL and VOL for italy_osm graph across different k. Each value is the result of $(MV^{\max}_{\text{VOL}} - MV^{\max}_{\text{NVOL}})/MV^{\max}_{\text{VOL}} \times 100$. 55
7.4 Comparison of the average normalized standard deviation on coPapersDBLP across different k for VOL (blue) and NVOL (orange). 56
7.5 Comparison of the outgoing communication volumes of the partitions of coPapersDBLP $k = 8$ for VOL (blue) and NVOL (orange).. 56
7.6 Comparison of the outgoing communication volumes of the partitions of coPapersDBLP $k = 128$ for VOL (blue) and NVOL (orange). 58
7.7 Comparison of the average normalized standard deviation and total communication volume on kmer_V2a across different k for VOL (blue) and NVOL (orange). 59
7.8 Comparison of the average normalized standard deviation and total communication volume on kmer_U1a across different k for VOL (blue) and NVOL (orange). 59
7.9 Comparison of the outgoing communication volumes of the partitions of kmer_U1a $k = 128$ for VOL (blue) and NVOL (orange).. 60
7.10 Percentage difference in $MV^{\max}$ between NVOL and VOL for our selection of geometric graphs across different k $((MV^{\max}_{\text{VOL}} - MV^{\max}_{\text{NVOL}})/MV^{\max}_{\text{VOL}} \times 100)$. 62
7.14 Estimated communication time per partition on italy_osm $k = 32$ for VOL 3%(black) and NVOL 3%(orange) when ignoring the latency. 70

List of Tables

4.1 Notation used in Chapter 4. 19

7.1 Heart dataset [35]. 48

7.2 Selection of matrices from SuiteSparse Matrix Collection [11] 48

7.3 Percentage difference in $MV^{\max}$ between NVOL and VOL for the Heart dataset across different k. Each value is the result of $(MV^{\max}_{\text{VOL}} - MV^{\max}_{\text{NVOL}})/MV^{\max}_{\text{VOL}} \times 100$. Cells marked with N/A were not run. 49

7.4 Average, best, and worst percentage difference in $MV^{\max}$ per graph for VOL and NVOL (Heart dataset). 49

7.5 Comparison of the standard deviation of the outgoing volumes and total volume on heart04 graph for VOL and NVOL 51

7.6 Percentage difference in $MV^{\max}$ between NVOL and VOL for the SuiteSparse Matrix Collection selection across different k. Each value is the result of $(MV^{\max}_{\text{VOL}} - MV^{\max}_{\text{NVOL}})/MV^{\max}_{\text{VOL}} \times 100$. Cells marked with N/R and N/A were not run.. 52

7.7 Average percentage difference in $MV^{\max}$, vertex number, average degree, and standard deviation of vertex degree per graph (SuiteSparse Matrix Collection). 53

7.8 Comparison of the standard deviation of the outgoing volumes and total volume of VOL and NVOL for $k = 2$ runs where NVOL is worse in Table 7.6.. . . . 53

7.9 Comparison of the total communication volume and the standard deviation of the outgoing volumes on a selection of graphs from Table 7.2 using VOL and NVOL for $k = 32$. 54

7.10 Comparison of the percentage difference of total volume and standard deviation on hugetric-00000 for VOL and NVOL. A positive value means that optimizing NVOL produces a lower value than optimizing VOL, and vice versa. 61

7.11 Percentage difference in $MV^{\max}$ between NVOL 4% and VOL for the Heart dataset across different k. Each value is the result of $((MV^{\max}_{\text{VOL}} - MV^{\max}_{\text{NVOL}})/MV^{\max}_{\text{VOL}} \times 100)$. Cells marked with N/A were not run. 63

7.12 Average, best, and worst percentage difference in $MV^{\max}$ per graph for VOL and NVOL 4% (Heart dataset). 63

7.13 Percentage difference in $MV^{\max}$ between NVOL 4% and VOL for the SuiteSparse Matrix Collection selection across different k. Each value is the result of $((MV^{\max}_{\text{VOL}} - MV^{\max}_{\text{NVOL}})/MV^{\max}_{\text{VOL}} \times 100)$. Cells marked with N/R and N/A were not run. 64

7.14 Average, best, and worst percentage difference in $MV^{\max}$ per graph for VOL and NVOL 4% (SuiteSparse Matrix Collection).. 65

7.15 Percentage difference for the slowest communication time estimate for NVOL 3% and NVOL 4% compared to VOL 3%(e.g. $\Delta R_{\mathrm{NVOL}} = (R_{\mathrm{VOL}} - R_{\mathrm{NVOL}})/R_{\mathrm{VOL}}$). 66

List of Algorithms

1 VOL gain calculation 23
2 NVOL gain calculation 26
3 GR with VOL 34
4 GR with NVOL 37

Acronyms

VOL	Total Communication Volume objective function
NVOL	Partition-aware Communication Volume objective function
GP	Graph Partitioning
GPP	Graph Partitioning Problem
BFS	Breadth-First Search
SGP	Streaming Graph Partitioning
GGP	Graph Growing Partitioning
KL	Kernighan-Lin
SHEM	Sorted Heavy Edge Matching
GR	Greedy Refinement
$MV^{\max}$	Maximum Outgoing Partition Volume

Chapter 1

Introduction

The demand for computational resources in modern scientific and engineering applications continues to grow, making parallel computing an essential area of development. While processor performance continues to improve, the growth in single-core performance has slowed considerably since the end of Dennard scaling [22], making it impractical to tackle the largest problems on a single processor alone. Instead, it has become necessary to distribute scientific and engineering applications across the many processors of a parallel computing system.

To facilitate this, many models have been proposed that represent scientific and engineering applications as graphs or hypergraphs [5, 6, 8, 19, 48]. The vertices correspond to units of computation and the edges encode data dependencies among them [19]. Efficiently distributing the computation then becomes a question of how to partition this graph among processors: balancing the workload evenly while minimizing inter-processor communication. This is also known as the *graph partitioning problem*. Although graph partitioning is known to be NP-hard, a variety of algorithms have been proposed that produce good quality solutions in practice by using heuristics [20, 25, 28, 29]. Beyond parallel computing, graph partitioning plays a critical role in numerous other application areas, including task scheduling, neural network simulations, operations research and more.

More than 20 years ago, the scientific community already argued that the future of software development lies in concurrency, e.g., [45]. As computational demands continue to grow, our programs and requirements consistently outpace hardware improvements. Sutter and Larus highlight the need for better parallel programming tools, given the inherent difficulty of developing parallel solutions [45]. A decade later, on July 29, 2015, President Obama issued Executive Order 13702, establishing the National Strategic Computing Initiative (NSCI) to ensure continued United States leadership in high-performance computing [13], underscoring the importance of this field and the need for sustained research. Graph partitioning algorithms are central to making this possible.

Most modern graph partitioning algorithms minimize *edge-cut* (see Section 2.1.1), seeking to minimize the number of inter-partition edges. The appeal of this metric lies in its simplicity: it is easy to compute, and for the structured meshes that dominate scientific computing, it correlates reasonably well with actual communication costs [19]. However, edge-cut has several well-known shortcomings as a representation for communication [18, 19]. First, it over-counts *communication volume* (see Section 2.1.1): a vertex with multiple edges crossing into the same partition needs to be communicated only once, yet each edge is counted separately when using edge-cut. Second, and most relevant to this work, minimizing the *total* edge-cut does not account for how

communication is distributed across partitions. In a parallel application, performance is generally limited by its slowest process [19], meaning that even if the total edge-cut is good, some process might bear a greater communication load than others, creating bottlenecks.

Karypis and Moulitsas highlighted the need for partitioning algorithms that account for heterogeneous architectures and proposed using *communication volume* as an optimization objective function instead of edge-cut [36]. This communication volume objective function was subsequently added into the METIS software package (see Section 6.1). However, like the edge-cut objective function, METIS focuses on the total communication volume without considering how volume is distributed among partitions. Regardless of the hardware architecture, focusing only on the total volume of a partitioned graph may lead to bottlenecks in some partitions.

Efficiently distributing computation across processors is central to high-performance computing, and graph partitioning is the standard technique for achieving this. In *Graph Partitioning and Parallel Solvers: Has the Emperor No Clothes?* [18], Hendrickson concludes that one of the key needs is to develop a more accurate metric for communication cost. Researchers have proposed alternative minimization objective functions that target per-processor communication metrics rather than global totals. An example is UMPa [8], which minimizes the maximum data sent by any single processor in a distributed memory setting. As Hendrickson noted, minimization objective functions that target communication costs are harder to optimize than the standard edge-cut, which partly explains their limited adoption.

1.1 Thesis Goal

In this thesis we present a partition-aware communication volume objective function called *NVOL* (see Chapters 4-5). NVOL minimizes the *per-partition* maximum outgoing communication volume, aiming to reduce uneven communication loads. To be able to correctly calculate the partition-specific change in communication volume we also develop a novel algorithm in Chapter 4 to perform this calculation. Using the METIS software as our baseline multilevel k-way partitioner, we present a new implementation of a partition-aware objective function and address two questions:

- **RQ1**: Can a partition-aware objective function produce more balanced communication loads than a global one?
- **RQ2**: Can the performance of a parallel application be improved in practice by using a partition-aware objective function?

NVOL is implemented in the refinement stage of the multilevel k-way algorithm. After an initial partition is performed, the partitions are gradually refined by moving vertices among partitions. NVOL gathers information about how every partition's communication volume is affected by a vertex move, allowing it to correctly evaluate how much partition a will change if a vertex v is moved to another partition b. Using this information, NVOL prioritizes moves that improve the per-partition maximum. No move that increases the maximum volume held by a single partition is accepted. This means that in some cases NVOL might even accept moves that increase the total communication volume, as long as they decrease the per-partition maximum. The goal is to provide more balanced partitions, avoiding overloading some for the sake of a better total communication volume.

1.2 Thesis Structure

Chapter 2: *Background*

Chapter 2 covers the fundamentals of graph partitioning. It surveys some of the algorithms in common use today, along with the main application areas in which they are applied.

Chapter 3: *Multilevel k-way Graph Partitioning*

Chapter 3 presents the multilevel k-way graph partitioning algorithm on which NVOL is built. This approach offers a good balance among partition quality and runtime, making it a popular choice for large graphs. The chapter focuses on the standard formulation, which uses edge-cut; the following chapters then extend the algorithm to support volume-based objective functions such as NVOL.

Chapter 4: *Communication Volume Calculation*

Chapter 4 explains how communication volume change is evaluated for volume-based objective functions. It covers the calculation of both total communication volume and per-partition volume, beginning with formal definitions of the calculations followed by worked examples. The final sections describe how these calculations are integrated into the multilevel k-way algorithm.

Chapter 5: *Greedy Refinement Using Communication Volume*

Chapter 5 covers the refinement stage and explains how it is implemented for volume-based objective functions. Building on Chapter 4, it shows how partitions can be refined using communication volume objective functions. The total communication volume objective function and NVOL are each treated in their own section, and both conclude with a simplified algorithm representing the implementation.

Chapter 6: *METIS with NVOL and Supporting Tooling*

Chapter 6 presents the tools used in this thesis to run and analyze the multilevel k-way partitioning algorithm with NVOL. It introduces METIS, the partitioning framework that NVOL extends, and describes its input and output formats. It also presents a small collection of Python scripts developed alongside this work for preprocessing input graphs and analyzing the per-partition communication volumes produced by the experiments.

Chapter 7: *Results*

Chapter 7 presents the results of running NVOL across a variety of graphs and partition counts. It compares the partitions across multiple metrics and estimates of the resulting communication time among partitions produced by NVOL and METIS.

Chapter 8: *Conclusions and Future Work*

Chapter 8 concludes the thesis and evaluates the contributions of this project. It revisits the initial goals set out in Chapter 1 and discusses how well they have been fulfilled. It discusses some of the current shortcomings and how they might be addressed, and introduces new ideas that would build naturally on the NVOL implementation.

Chapter 2

Background

This chapter gives a description of what the *graph partitioning problem* (GPP) is, as well as what objective functions are used to evaluate partitions. It also discusses some of the common practices in the field with some examples of concrete application areas for *graph partitioning* (GP).

2.1 Graph Partitioning Problem

The GPP refers to a group of problems that deal with distributing the vertices of a graph into groups, clusters or communities. The goal is to group together vertices that are similar or that share many edges between them to minimize the number of edges among groups. The edges in some graphs are directed, meaning that they contain information about the communication flow among vertices. However, directed edges, or more generally directed graphs, are outside the scope of this thesis, and we consider only undirected ones.

Consider an undirected graph $G = (V, E)$ with $|V| = n$ vertices and $|E| = m$ edges. The goal is to partition V into k disjoint subsets $V_0, V_1, \ldots, V_k$ such that $V_i \cap V_j = \emptyset$ for $i \neq j$ and $\bigcup_i V_i = V$ [28]. The weight of the graph is $W = \sum_{v \in V} w(v)$, where $w(v)$ is the weight of vertex v (vertices can also have multiple weights, but in this thesis we only consider graphs with a single vertex weight). Note that a graph with unit weights will have a weight equal to the number of vertices $W = n$. A balance constraint requires that all subsets have approximately equal weight. The partition weight equals the weight of each vertex it contains, $w(V_i) = \sum_{v \in V_i} w(v)$. Using an imbalance parameter ϵ, this constraint requires that each subset satisfies

$$w(V_i) \leq \left\lceil \frac{W}{k} \right\rceil \cdot (1 + \epsilon), \quad i = 0, 1, \ldots, k.$$

The parameter ϵ represents the fractional excess weight a subset may have beyond the perfectly balanced weight $\lceil W/k \rceil$. When $\epsilon = 0$, the partition must be perfectly balanced [4]. The goal of GPP is to minimize a given objective function (see Section 2.1.1) while satisfying the balance constraint. GPP is NP-complete, meaning that for large graphs a solution can only be achieved through heuristics or approximation algorithms [17]. Unlike heuristics, which seek good solutions empirically without formal guarantees, approximation algorithms are required to come with a mathematically proven bound on how far their output can deviate from the optimum [49]. Despite their theoretical importance, approximation algorithms either underperform compared

to state-of-the-art GP solvers or are too slow for large graphs [4], making heuristics the most common choice in practice.

Another common representation of computational tasks in GP is *hypergraphs*. A hypergraph is a generalization of a graph where an edge, also called a hyperedge, can connect any number of vertices [3]. Similarly, GP divides the vertices into partitions while trying to maintain roughly equal size for each. However, given this more complex structure, utilizing hypergraphs requires more complex algorithms both in terms of implementation and running time. For the scope of this thesis, we will focus on standard GP; however, most of the techniques presented in this section (see Section 2.2) have been or can be applied to a solution based on hypergraphs as well.

2.1.1 Objective Functions

An objective function is a mathematical formula used to define the goal of a problem, aiming to improve a specific value. The most common objective function in GPP is to minimize the total edge-cut [4]. An edge $\{u, v\} \in E$ is said to be *cut* if its endpoints belong to different partitions, i.e., $u \in V_i$ and $v \in V_j$ with $i \neq j$. The total edge-cut is then the sum of all such cut edges across partitions. This also applies for graphs with edge weights, where the total edge-cut is then the sum of the edge weights of cut edges.

Despite being a less precise measure for actual communication cost, edge-cut has been widely adopted as the standard objective function. Buluç et al. [4] attribute this to two factors: cut optimization appears to be easier in practice, and for graphs with high structural locality, the cut tends to perform similarly to other objective functions.

Another common objective function is communication volume, which provides a more accurate measure of the data exchange between partitions of a communicating task. For a partition V_i, the outgoing communication volume is defined as $\text{comm}(V_i) = \sum_{v \in V_i} D(v)w(v)$, where $D(v)$ denotes the number of distinct partitions, other than the one containing v, in which v has at least one edge to another vertex. The communication volume is then multiplied with the vertex weight, $w(v)$, as that represents the outgoing communication volume.

2.1.2 Shortcomings of the Objective Functions

Even though edge-cut is accepted as the standard, it comes with several shortcomings. Most notably, it does not distinguish between multiple edges connecting to the same neighboring partition. Consider Figure 2.1, where each color represents a partition. Vertex A has two edges, both leading to the blue partition. Calculating the edge-cut, A contributes 2 to the total edge-cut. The communication volume, however, is only 1, since both edges connect to the same partition. In general, edge-cut can overestimate the actual communication burden, particularly for vertices with many neighbors in a single remote partition.

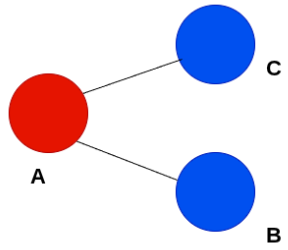


Figure 2.1: Vertex A has an outgoing communication volume of 1 but an edge-cut of 2. Each color represents a partition.

The performance of a parallel application is generally limited by its slowest processor. This means that even if the computational work is balanced, the communication effort may not be [4]. In such cases, it is more meaningful to minimize the maximum communication load incurred on a single processor rather than the total load across all processors. Neither the standard edge-cut metric nor the total communication volume captures this type of objective.

Furthermore, on many architectures the time to send data, or messages, depends not on geographic distance but on the number of network switches the data must traverse. Although modern machines employ cut-through or wormhole routing to move individual messages quickly between distant processors, the communication network handles many messages simultaneously, and a message between distant processors occupies links that are unavailable to other traffic. Minimizing edge-cut does not account for this locality of communication, and a partition that places frequently communicating processors far apart in the network topology can lead to significant message contention [4, 19].

Regardless of the objective function chosen, GP remains a fundamental tool across a wide range of domains. The following sections discuss some of the most used GP algorithms and some of their application areas.

2.2 Graph Partitioning Algorithms

In this section we will present some of the most popular GP algorithms used today. The goal is to show some of the differences in the approaches and to highlight their typical use cases. This will help the reader to understand why the multilevel partitioning approach is deemed the best option for large scale graphs. These are, however, only *some* of the existing GP algorithms, a more complete list can be found in [4, 7].

2.2.1 Recursive Bisection

A popular partitioning method is recursive bisection (RB). Instead of partitioning a graph directly into k subsets, RB first bisects the graph into two roughly equal parts using algorithms such as [1, 14, 31], and then recursively bisects each part until k partitions have been formed. When k is not a power of two, modified schemes are used [9, 20, 27, 42]. Although simple and widely used, RB has been shown to produce suboptimal k-way partitions [42], since each bisection is made independently without knowledge of the cuts that will follow at deeper levels of the recursion.

2.2.2 Spectral Bisection

Spectral Bisection was one of the first methods used to split a graph into two partitions and is still used today [4]. Its approach is to use the eigenvector, also known as the *Fiedler vector*, of the second smallest eigenvalue of the Laplacian matrix to deduce global information of the connectivity of the graph [15]. In order to form the two partitions, the median value $\overline{m}$ of the Fiedler vector is calculated, and every vertex with a value smaller or equal to $\overline{m}$ is assigned to one partition, and the rest to the other one.

The main computational cost of the spectral method lies in the calculation of the Fiedler vector. To optimize this, Barnard and Simon [1] have used a multilevel method to produce an approximation of the Fiedler vector. The spectral method has also been extended to produce a partition for more than only two partitions and to support edge and vertex weights [20].

2.2.3 Graph Growing Partitioning

Another bisection approach is called graph growing partitioning (GGP) [27]. GGP is mainly based on breadth-first search (BFS). The core idea is to start from a random vertex and to use BFS to construct a partition. BFS is run until half the vertices are assigned to one partition, the remaining vertices are then assigned to the other one. The choice of starting vertex significantly affects the result, meaning that this algorithm tends to require multiple restarts to obtain a good solution. The GGP approach has also been extended to produce more than two partitions, called *Bubble framework* [12]. In this version k starting vertices are chosen, evenly distributed in the graph. Starting from these vertices, k BFS traversals are performed. Throughout, *Bubble* uses local search algorithms to balance the partitions in the graph. At the end of an iteration, a new starting vertex is chosen for all partitions. The new starting vertex is the vertex with the minimum total distance to all other vertices in the same partition. The algorithm is repeated until no new changes occur in the choice of starting vertices, or no improvement for the partitions is found. The major drawback of this approach is its high computational cost.

2.2.4 Streaming Graph Partitioning

Streaming data models, where input is processed on the fly rather than all at once, have gained popularity due to their small memory requirements. In the field of GP, the input graph is also passed as a stream, requiring much less space than the graph size [4]. Streaming graph partitioning (SGP) is also very fast, even faster than multilevel algorithms (see Section 2.2.5). However, these algorithms produce lower quality partitions compared to multilevel approaches. They are nonetheless useful in applications where speed is important, and an initial partition can be obtained from a stronger GP algorithm. For more details about SGP, we refer the reader to [39, 44, 47].

2.2.5 Multilevel Graph Partitioning

Multilevel GP is considered to be the most successful heuristic for partitioning large graphs [4, 27, 29]. Because of this, multilevel GP was chosen as the base on which to build our new objective function NVOL. In short, the idea is that a graph is coarsened to a smaller graph. Once the graph is at its smallest, an initial partition is performed, and then it is iteratively uncoarsened to the original size. During the uncoarsening, heuristics are applied to improve the partitions. The intuition behind multilevel GP is that it is

easy to run intensive partitioning algorithms on the small graph without affecting the total runtime. Additionally, starting from an already good solution, the refinement stage is not costly to perform and can quickly improve the partitions. Chapter 3 will explain in detail the algorithm, providing the necessary background for understanding how NVOL is integrated into the framework.

2.3 Applications of Graph Partitioning

Graph partitioning arises in a broad variety of domains. This section highlights several of the most prominent application areas, though the list is by no means exhaustive.

2.3.1 Parallel Processing

Perhaps the most widespread application of GP is distributing computational work across the processors of a parallel machine [32]. GP is the standard approach in scientific computing applications such as sparse direct and iterative solvers. When the problem domain remains fixed throughout the computation, the partitioning can be performed once at the outset; this is known as static partitioning [4]. Other common applications in parallel processing include eigenvalue computations [2], breadth-first search [3], and algorithms for PageRank and connected components [41].

2.3.2 Complex Networks

GP is also relatively popular in the field of complex networks [4]. Complex networks, in this context, refer to weighted graphs with nontrivial structure arising from real world or modelling processes [38]. The objective functions applied in this field are usually domain specific, making it hard to develop optimization algorithms.

Several application domains illustrate this breadth. In power grids, disturbances and cascading failures may lead to blackouts. GP can be used to isolate self-sufficient islands to prevent propagation of an outage [33]. Cut based objective functions are sometimes extended to enhance the robustness of the system [34].

Geographically embedded networks have experienced a growth in requirements with the advances in location-aware devices (such as GPS). It is now vital that algorithms be able to analyze networks like these extremely fast. In these networks the vertices commonly represent entities in the real world tied to geographic places. The cut-based approach therefore needs to be reinforced with spatial contiguity constraints [4, 34].

Many biological systems can be modeled as graphs [4], protein-protein interaction or gene co-expression graphs, for example. Partitioning serves multiple purposes ranging from data reduction to the detection of biological processes. More about this field can be found in [23].

In social networks, community detection is one of the most popular topics. In contrast to traditional GPP, the number of subgroups or partitions is not known a priori [4]. Apart from this, GP has contributed greatly to the development of community detection algorithms [16, 37].

2.3.3 VLSI Physical Design

VLSI circuit design has historically been one of the primary application areas for graph partitioning [4]. The objective is to decompose a circuit into smaller subcircuits, ranging from gate arrays to complete integrated circuits, while minimizing the total

interconnection cost, typically measured as the weight of connections crossing partition boundaries. In this formulation, nodes represent cells (i.e., basic logic or functional units such as gates) and edges represent wires. Scalability is essential, as circuits can contain millions of modules and partitioning constitutes a bottleneck in the overall design flow. Beyond the basic cut objective function, practical VLSI partitioning must also respect domain specific constraints such as I/O placement, groups of cells that are required to remain in the same block, and upper bounds on the cut size between block pairs. Further details can be found in [10, 24].

2.3.4 Neural Networks

Graph neural networks (GNNs) extend deep learning to graph structured data, enabling the processing of irregular domains that traditional neural network architectures cannot readily handle. However, as graph sizes grow, training GNNs becomes increasingly expensive due to rising communication overhead, particularly in distributed or multi-device settings. Graph partitioning offers a natural approach to this scalability challenge by distributing subgraphs across devices while minimizing inter-device communication.

A recent example is CaPGNN [43], a parallel full batch GNN training framework designed for multi-GPU servers. CaPGNN combines resource aware graph partitioning with an adaptive caching mechanism that exploits both CPU and GPU memory to reduce redundant data transfers. By taking into account the heterogeneous computational and communication capacities of individual GPUs, the framework achieves substantial reductions in training time with minimal impact on model accuracy. While this is only one instance of a broader trend, it illustrates how classical graph partitioning techniques are finding renewed relevance in the context of modern machine learning workloads.

Another recent example is ParDNN [40], an automatic, generic, and non-intrusive partitioning strategy for Deep Neural Networks represented as graphs. Like CaPGNN, this strategy has the goal of minimizing the training time of the model without affecting accuracy. Through their experiments, Qararyah et al. show significant improvements compared to related work in efficiency, superlinear scaling of batch size and training throughput.

Chapter 3

Multilevel k-way Graph Partitioning

This chapter presents the Multilevel k-way Graph Partitioning (GP) algorithm, one of the most widely used GP algorithms in practice. The multilevel approach scales to large graphs by first coarsening the input down to a small graph, partitioning that, and then projecting the partition back to the original size while refining it at each intermediate level. We cover each of these three phases: coarsening, initial partitioning, and uncoarsening with refinement.

The chapter is written with a particular focus on the techniques used by METIS (see Section 6.1), the partitioner employed in this thesis. The refinement step is treated in the most detail, as it is the phase in which our objective function, NVOL, is integrated. However, NVOL itself is not described here; the next two chapters cover its design and how it plugs into the refinement framework presented in this chapter. The aim of this chapter is to give the reader an understanding of how the multilevel k-way graph partitioning algorithm works, before introducing our objective function, NVOL.

3.1 Preliminaries

The overall structure of a multilevel k-way graph partitioning algorithm is fairly straightforward. The process begins by coarsening the input graph until it is reduced to a sufficiently small size. At this coarsest level, an initial k-way partitioning is computed using a graph growing algorithm (see Section 2.2.3) [27]. The graph is then uncoarsened back to its original size in successive steps. At each level of uncoarsening, the partitioning is refined to improve quality by exploiting the increased degrees of freedom available at the finer level. During this phase, the partitions are improved using an objective function (see Section 2.1.1). NVOL, our objective function, is implemented in this phase, which is discussed in Chapter 5. Karypis and Kumar demonstrate in [28] that this multilevel approach yields high-quality partitions, often comparable to or better than those produced by recursive bisection [27].

In Figure 3.1, this process is illustrated, showing how an initial graph G_0 is coarsened until it reaches G_4, which is smaller in size. At this level, an initial partition is performed on the coarsest graph. After that, the graph is step by step projected back to its original size, refining the partition at each level in order to minimize the chosen objective function [30].

The multilevel partitioning problem uses the same definitions as in Section 2.1. A k-way partition is represented through a vector P. The vector has a length of n, where n is the number of vertices in the graph. Each element of P is an integer between 1 and k, indicating the partition the vertex belongs to. Specifically, a vertex $v \in V$ assigned

to partition a will have $P[v] = a$. The next sections will now explain how each phase works and how the partitions are formed.

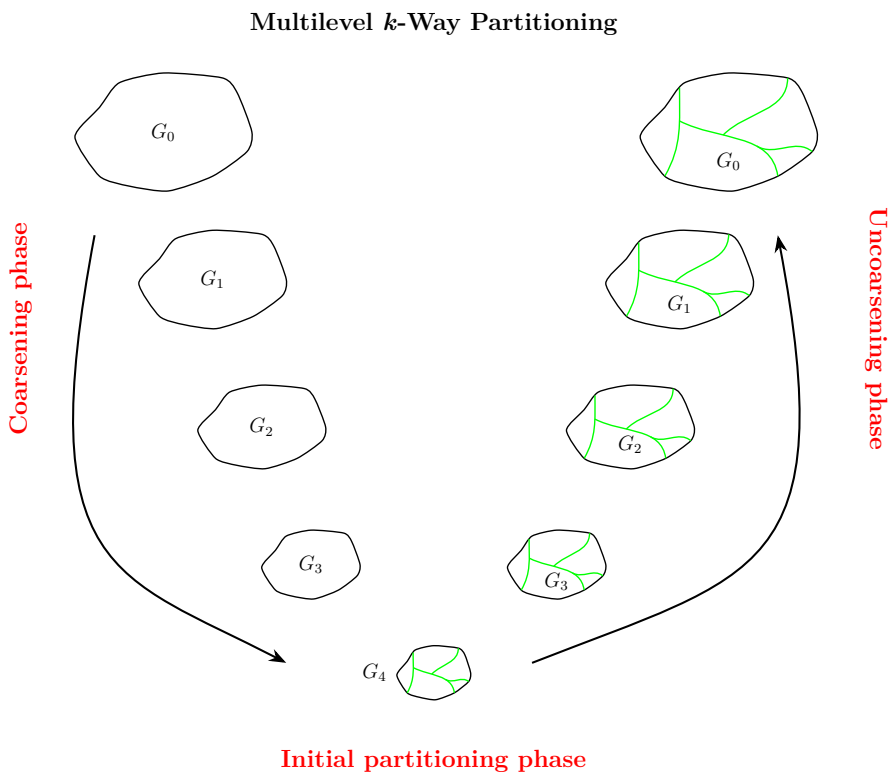


Figure 3.1: The phases of the multilevel k-way partitioning algorithm. During the coarsening phase, the vertices are contracted together and the graph size is successively decreased; during the initial partitioning phase, a k-way partition is performed on the smallest graph; and during the uncoarsening phase, the partitions are successively refined and projected back to a larger graph.

3.2 Coarsening Phase

During the coarsening phase, a sequence of increasingly smaller graphs is constructed, maintaining a similar structure to the original graph with fewer degrees of freedom. The most common approach is to aggregate groups of vertices into a single coarse vertex through matching [4, 27]. Formally, the idea is to contract pairs of matched vertices, where we define a matching as a set of edges no two of which share a vertex [4, 27]. Since the goal of coarsening is to reduce the graph size, a matching is considered good if it is composed of many matched edges. To achieve this, *maximal matchings* are used. A matching is maximal if any edge in the graph that is not contained in the matching has at least one of its endpoints matched [27]. The following subsection describes in detail the implementation of Sorted Heavy Edge Matching (SHEM), which is the matching algorithm employed by the partitioner we used in this thesis. For further reading on other coarsening algorithms, we refer the reader to [4, 27].

Once a matching is performed the coarse vertex has a weight equal to the sum of the weights of all the vertices it contains. Additionally, the coarse vertex receives all the edges of the vertices it contains. Vertices that are not matched are simply copied over

to the smaller graph. This process is repeated until we reach a graph G_m with a small enough number of vertices [25].

For a given graph $G_i = (V_i, E_i)$, a subset of vertices $V_i^v \subseteq V_i$ is merged into a coarse vertex $v \in V_{i+1}$ for the next coarser graph G_{i+1}. The coarse vertex v has weight $w(v) = \sum_{u \in V_i^v} w(u)$. This ensures that the total vertex weight is preserved across coarsening levels. To preserve connectivity, the new vertex v receives the union of the edges of the vertices in V_i^v. For any neighboring vertex u, the edge weight is $w(\{v, u\}) = \sum_{z \in V_i^v} w(\{z, u\})$. This aggregation strategy preserves structural and weight-related properties of the original graph, thereby enabling efficient and balanced processing at coarser levels [28].

3.2.1 Sorted Heavy Edge Matching

SHEM begins by generating a random permutation of the graph's vertices, which is then sorted based on vertex degrees. Each vertex gets assigned a bucket based on the square of its degree, sorted from lowest to highest. This ordering ensures that low-degree vertices, which have fewer opportunities for matching, are prioritized and thus more likely to be successfully matched. Delaying them would increase the chance of remaining unmatched. Once the vertices are sorted, they are traversed in order. For each unmatched vertex u, the algorithm selects the unmatched neighboring vertex v connected to u via the heaviest incident edge. The edge $\{u, v\}$ is then added to the maximal matching M_i, and both u and v are marked as matched. If no suitable neighbor exists, the vertex u remains unmatched. This process continues until all vertices have been processed.

The greedy nature of SHEM tends to preserve strong edge connections in the coarser graph, contributing to a significant reduction in total edge weight and, consequently, in the number of edge-cuts [27, 29]. Karypis and Kumar show in [25] that the coarser graph has lower total edge weight, and consequently a lower edge-cut.

3.3 Initial Partitioning Phase

Once the graph has been coarsened to the smallest level, the initial partitioning is performed. The goal of this stage is to produce k roughly equal partitions before further refinement. Several partitioning algorithms can be used here, including spectral partitioning [20], graph growing partitioning [27], and multilevel recursive bisection [42], among others. Since the graph is relatively small at this stage, the partitioning is fast. The partitioner we used, METIS, uses a multilevel recursive bisection approach.

The multilevel recursive bisection approach takes the coarse graph and bisects it using graph growing partitioning (GGP, see Section 2.2.3). The resulting two subgraphs are then each bisected again. This process is repeated recursively until the desired number of partitions is reached. When the number of partitions is not a power of 2, the algorithm keeps bisecting until one subgraph has 3 partitions remaining. At this level only one of the produced subgraphs is bisected further. Consider Figure 3.2, where graph G is to be partitioned into 7 groups. At the first level, G_0 has 4 partitions remaining while G_1 has 3. Since G_1 has only 3 partitions remaining, only G_4 is bisected further. The left side (G_0), on the other hand, continues bisecting until all partitions have been produced.

During each bisecting step, GGP is used. As described in Section 2.2.3, this bisection approach is based on breadth-first search (BFS). Starting from a random seed it tries to group together vertices connected to each other. At every step it checks that both partitions maintain a weight threshold. Once a partition is computed the graph is further

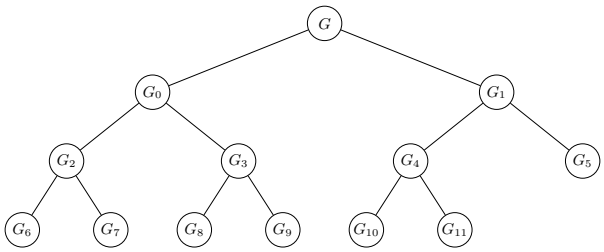


Figure 3.2: The process of bisecting a graph into 7 partitions.

refined using the refinement algorithm proposed by Fiduccia and Mattheyses (FM) in [14]. The FM refinement algorithm iterates through the boundary vertices of the graph, that is, vertices that share at least one edge with a vertex in the other partition. At each step the vertex move that yields the greatest reduction in edge-cut (see Section 2.1.1) is selected. To avoid a subpar solution, FM allows a limited number of moves that worsen the edge-cut. This process is repeated over the boundary vertices several times, keeping the partition with the lowest edge-cut.

Despite the non-deterministic nature of the GGP algorithm, METIS produces the same result for identical re-runs. METIS uses a fixed default seed, so the "random" choice is the same across re-runs. In other words, if the seed is not manually passed, the initial partition will always be the same when partitioning a graph with the same parameters multiple times.

It is worth noting that for our volume based objective function we also use this approach based on edge-cut for the initial partition. There are a few reasons for this. Firstly, the initial partition is going to be heavily refined later during the uncoarsening phase, making the quality of the initial partition not as important. Additionally, algorithms like FM are fast and work well with edge-cut; using another objective function would make this phase significantly slower. For these reasons, it is reasonable to use edge-cut as the objective function for this first partitioning.

3.4 Uncoarsening Phase

The final step is the uncoarsening and refinement phase, during which the graph is gradually projected back to its original size. The coarsest graph G_m is successively expanded through finer representations $G_{m-1}, G_{m-2}, \ldots, G_1$. At each projection from G_{i+1} to G_i, every vertex contained within a coarse vertex v inherits the partition assignment that v held in G_{i+1}. Once all coarse vertices at a given level have been expanded, the resulting partition is improved using local refinement heuristics.

The partitioner we used, METIS, employs a variant of the Kernighan-Lin (KL) algorithm [31], called Greedy Refinement (GR), to refine the graph. The KL algorithm iteratively swaps vertices among partitions of a bisection to minimize an objective function, continuing until no further improvement is possible and a local minimum is reached [28]. Refining a k-way partition is significantly more complex since each vertex has $k-1$ possible moves as opposed to just one move in a bisection [28]. This substantially enlarges the optimization space and increases the computational cost, since a gain must be calculated for each possible move. The gain represents the improvement in the objective function's value resulting from relocating a vertex to a new partition.

The refinement procedure described above is general; what changes between objective functions is how the gain of a move is computed. In the rest of this section, we present the

standard edge-cut-based formulation used by METIS, as it is the simplest case and makes the underlying mechanics easiest to follow. The total communication volume objective function and our partition-aware variant, NVOL, both build on this same refinement framework and use the GR algorithm. They differ in how the gain is computed and which moves are accepted; these are covered in Chapter 4 and Chapter 5, respectively.

3.4.1 Projection and preparation before refinement

Once the graph is projected up a level, it undergoes a preparation phase before actual refinement. The new finer graph contains new vertices and edges, requiring a full recomputation of the underlying structures describing the graph. The whole graph is therefore traversed in order to obtain accurate information about the new connections in this finer level.

During this phase, the boundary set, the set that defines the vertices visited during refinement, is recomputed. By definition, a vertex is on the boundary of a partition if it has at least one edge to another partition. However, to further increase efficiency, METIS filters this set using more than just whether a vertex is on the boundary. As the next section will show, a move is accepted only if it leads to positive or zero gain. Therefore, METIS chooses to ignore boundary vertices with negative gain.

In order to express gain for edge-cut, we need three definitions: internal degree, external degree, and the external degree towards a specific partition. Every vertex has a degree, which represents the number of edges incident to it. This degree can be divided into internal and external degree, which define the number of edges a vertex has toward different partitions. The internal degree is the number of edges from a vertex v to other vertices in the same partition as v. We denote the internal degree of a vertex v by $ID[v]$. The external degree is the number of edges from v to vertices in partitions other than its own. We denote the external degree of v by $ED[v]$. To simplify calculations during refinement, we also specify the external degree toward a specific partition a as $ED_a[v]$.

A vertex v is then added to the boundary set if:

$$ED[v] - ID[v] \geq 0. \tag{3.1}$$

This expression tells us whether a vertex has the potential to be moved or not. For the edge-cut objective function, gain is computed as the specific external degree minus the internal degree (see Equation 3.2). The specific external degree will always be smaller than or equal to the total external degree, which means that if the expression above is negative, the gain will also be negative, and no move will be performed. Therefore, to improve efficiency, METIS only adds vertices to the boundary set if they satisfy Equation 3.1.

To optimize refinement even further, the boundary set is ordered by the potential gain of each vertex. Since the actual gain needs to be calculated for every possible destination a vertex has, METIS estimates the potential gain. For a vertex v, the potential gain is expressed as:

$$\frac{ED[v]}{\sqrt{|N_v|}} - ID[v]$$

where $|N_v|$ is the number of unique partitions (apart from its own) that v is connected to. This gives a normalized estimate of the probable gain. Using this ordering increases the chance of performing high-gain moves early. This set is then used by the refinement algorithm to refine the partitioning quality.

3.4.2 Greedy Refinement

In [28], Karypis and Kumar propose an improved version of the k-way KL refinement algorithm, referred to as Greedy Refinement (GR). This algorithm achieves high-quality partitions while maintaining a computational complexity that is independent of the number of partitions k. Traditional KL-based refinement algorithms implement a lookahead strategy, which permits the movement of clusters of vertices across partition boundaries [28]. However, within the context of a multilevel partitioning framework, the benefit of this mechanism diminishes. Since each vertex in a coarser graph G_i represents a cluster of vertices in a finer graph G_{i-1}, moving a single vertex in G_i already corresponds to relocating a group of vertices. Consequently, the lookahead mechanism becomes redundant. GR eliminates the lookahead step, thereby simplifying the refinement process and infrastructure presented in [21]. This simplification contributes to both the reduced computational complexity and the practical efficiency of the GR algorithm. GR is independent of the choice of objective function. In this section, we describe the main components of the algorithm and use edge-cut as a concrete example to illustrate how gain drives the choice of a move. The next two chapters will show how NVOL performs these calculations.

The GR algorithm proceeds through multiple iterations, in which each iteration visits all boundary vertices. For each vertex, the algorithm determines whether relocating it to a neighboring partition would improve the objective function's value. In some cases, GR may also choose moves that do not change this value but instead lead to improved balance in the graph. Any move must satisfy the balancing conditions [28]. The updated partitioning serves as the input for the next iteration.

For a graph $G_i = (V_i, E_i)$ and its partitioning vector P_i, each vertex $v \in V_i$ has a set of neighboring partitions N_v, defined as the union of the partitions (apart from v's own) which its adjacent vertices belong to. We denote the set of vertices adjacent to v by V_v^{adj}. If all neighbors of v are in the same partition as v, then v is considered an *interior vertex*, and $N_v = \emptyset$. Conversely, if v has neighbors in different partitions, then $N_v \neq \emptyset$, and the cardinality of N_v can be as large as $|V_v^{\text{adj}}|$ (when each neighbor belongs to a distinct partition). During refinement, a vertex v can move to any of the partitions in N_v.

For all boundary vertices, we consider how a move to a neighboring partition (a partition in N_v) affects the objective function's value. For edge-cut, the gain of moving a boundary vertex v to a neighboring partition $a \in N_v$ is defined as:

$$\text{gain} = ED_a[v] - ID[v]. \tag{3.2}$$

GR iterates over all partitions in N_v, looking for any move that yields a positive gain. The move yielding the highest gain is then chosen as the destination. To maintain balance, each move must either satisfy the graph's balance constraints or improve its balance. The weight boundaries are defined as follows:

$$W^{min} = \frac{W}{k(1+\epsilon)}$$

$$W^{max} = \frac{W}{k} \cdot (1+\epsilon)$$

meaning that any partition a must satisfy $W^{min} < w(a) < W^{max}$, even after a move [28]. Note that our notation for W^{min} differs from the one in [28]; we based ours on the partitioner's source code. The only difference is that our expression uses the balance

constraint ϵ, whereas [28] uses a constant. Additionally, as a tiebreaker for moves that yield the same gain, GR accepts moves that improve the load balance in the graph. A move improves balance if its destination is a lighter partition, promoting a more equal distribution of vertex weights across partitions [28]. For a vertex v, this condition is expressed as:

$$w(\text{destination}) < w(\text{source}) - w(v).$$

After each accepted move, the boundary set, neighbor partition sets, and the degrees of the affected vertices are updated. GR continues through the boundary set until all vertices have been visited. Note that whenever a move is made, the boundary set is affected, and vertices may be added to or removed from it. It is also worth mentioning that moved vertices cannot be moved again during the same refinement stage [28]. Once a full pass is completed, the boundary set is computed again and the process repeats. By default, METIS repeats this process up to 10 times, terminating early if a pass yields no improvement in the objective function's value or moves no vertices. Based on their empirical results, Karypis and Kumar observe that GR typically converges within four to eight iterations [28]. This relatively fast convergence contributes to the overall efficiency of the multilevel partitioning approach.

The two chapters that follow build on the multilevel framework presented here. Chapter 4 derives the gain expressions for METIS's total communication volume objective function and for NVOL, while Chapter 5 shows how these expressions plug into the GR algorithm to drive vertex moves.

Chapter 4

Communication Volume Calculation

This chapter extends the discussion from Section 3.4.2 of Chapter 3, where we showed how the change in the objective function's value (the gain) is computed for the edge-cut objective function. Here, we derive the analogous gain computations for two objective functions based on communication volume: the total communication volume objective function (VOL) and our partition-aware communication volume objective function NVOL. The next chapter then shows how these gain calculations are used inside the greedy refinement algorithm.

The chapter is organized into four sections. The first introduces the basic definitions needed to understand communication volume operations. The second describes the gain calculation for VOL, which is more representative of actual communication cost than edge-cut, although it does not necessarily yield a well-balanced result. We treat VOL first because it is the standard approach used by partitioners such as METIS.

The third section presents our novel gain calculation, used by NVOL, which provides finer information about how the volume of each individual partition is affected by a move. Here each vertex has a table representing the partition-specific gain of all possible moves. The section ends with an algorithm that ties the calculation together. The fourth and final section works through a few concrete examples to give the reader a clearer picture of how everything fits together.

Symbol	Description
G	The graph
V	Set of all vertices in the graph
E	Set of all edges in the graph
V_b	Set of all boundary vertices in the graph
V_i^{adj}	Set of all vertices incident to vertex i
N_i	Set of neighboring partitions of vertex i
N_{iv}	Set of partitions neighboring both vertex i and vertex v
N_{i-v}	Set of partitions only neighboring vertex i and not vertex v
$w(i)$	Vertex weight of vertex i
$P[i]$	Partition of vertex i
$PV[a]$	Outgoing communication volume of partition a
$C_i[n]$	Number of edges a vertex i has to a partition n
G_i	Gain vector of a vertex i
SG_i	Gain table of a vertex i

Table 4.1: Notation used in Chapter 4.

4.1 Basic definitions

To facilitate understanding of how gain calculation works, we begin by introducing a formal definition of communication volume. Let $G = (V, E)$ be a graph, and let P be the partition vector, where $P[v]$ is the partition a vertex v is assigned to. A vertex u is defined as a neighbor of v if there exists an edge $\{u, v\} \in E$. Any vertex adjacent to at least one vertex belonging to a different partition is said to lie on the boundary. We denote the set of all boundary vertices by $V_b \subseteq V$. Formally, $v \in V_b$ if there exists at least one neighbor $u \in V$ such that $\{v, u\} \in E$ and $P[v] \neq P[u]$.

For each boundary vertex $v \in V_b$, let N_v denote the set of distinct partitions (excluding $P[v]$) to which its neighboring vertices belong. The number of partitions contained in N_v, defined as $|N_v|$, is the outgoing communication volume of vertex v. Note that the number of edges incident to v is not directly relevant for this measure, only the number of distinct neighboring partitions. In Figure 4.1, vertex A has an outgoing communication volume of 2, as it is connected to vertices in two distinct partitions. In contrast, Figure 4.2 illustrates a case where vertex A has two edges, both connecting to vertices within the same partition. Consequently, the outgoing communication volume of vertex A is 1, since only a single neighboring partition is involved.

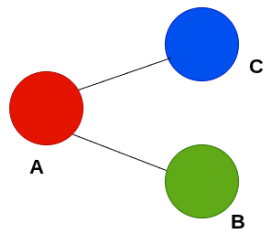


Figure 4.1: Vertex A has an outgoing communication volume of 2. Each color represents a partition.

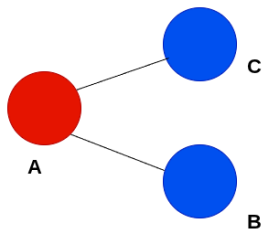


Figure 4.2: Vertex A has an outgoing communication volume of 1. Each color represents a partition.

The total communication volume of the partitioned graph is obtained by summing the outgoing volumes over all boundary vertices:

$$\text{total volume} = \sum_{v \in V_b} |N_v| \, w(v) \tag{4.1}$$

Here, $w(v)$ denotes the weight associated with vertex v, which scales its contribution to the outgoing communication volume. Even if the original graph has unit vertex weights, vertices acquire non-unit weights during coarsening as a result of vertex aggregation, and these weights persist during uncoarsening (see Section 3.4). In Figure 4.3, all vertices are assumed to have unit weight. The boundary set is $V_b = \{A, B, C, D, E\}$. Note that vertex F is not part of this set, as it has no edges connecting to vertices in other partitions.

Applying Equation 4.1, we compute the total communication volume by summing the contributions of all vertices in the boundary set. Vertices A, C, and D each contribute a volume of 2, while vertices B and E each contribute a volume of 1. This results in a total communication volume of 8.

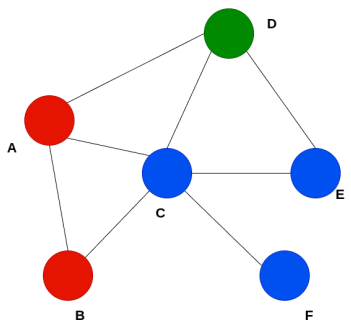


Figure 4.3: The total communication volume is 8.

4.2 VOL gain calculation

This section describes how the gain resulting from a vertex move is computed for VOL. All definitions are deduced from METIS's implementation, but the approach is standard and applies to any implementation of a total communication volume objective function. The gain for VOL represents the change in total communication volume resulting from relocating a vertex to a new partition: a positive gain indicates a reduction, a negative gain an increase.

Since communication only arises when an edge connects two vertices belonging to different partitions, a vertex move can only affect the communication volume of its neighboring partitions. For each boundary vertex $i \in V_b$, a partitioner needs to examine all its adjacent vertices $v \in V_i^{\text{adj}}$, where $V_i^{\text{adj}} \subseteq V$ denotes the set of vertices adjacent to i. When considering moving a boundary vertex i, the gain calculation depends on the relationship that i and each of its neighboring vertices have with other partitions. The effect on volume depends on which partitions they share. To support our calculations later, we define, for every vertex i and every neighboring vertex v

$$N_{i-v} = N_i \setminus (N_v \cup P[v]) \quad \text{and} \quad N_{iv} = (N_i \cap N_v) \cup P[v] \tag{4.2}$$

representing partitions exclusive to i and partitions shared by i and v, respectively. Notice how both sets are subsets of $N_i = N_{i-v} \cup N_{iv}$. Recall that by definition N_v excludes v's own partition $P[v]$. To correctly identify shared partitions, $P[v]$ must therefore be added to N_{iv} and removed from N_{i-v}.

As a concrete example, consider Figure 4.3 and suppose the candidate vertex is A, with neighbor B under consideration. Here, $N_A = \{\text{blue}, \text{green}\}$, while $N_B = \{\text{blue}\}$. Consequently, $N_{A-B} = \{\text{green}\}$, since green is the only partition connected to A but not to B, whereas $N_{AB} = \{\text{blue}\}$ as both vertices have at least one edge to the blue partition. Keeping A as the candidate vertex, consider now neighbor C with $N_C = \{\text{red}, \text{green}\}$. A and C have access to the same partitions, meaning that $N_{A-C} = \emptyset$ and $N_{AC} = \{\text{blue}, \text{green}\}$. This example highlights the need for adding $P[C]$ to the sets. The sets would have been $N_{A-C} = \{\text{blue}\}$, $N_{AC} = \{\text{green}\}$ if we had not accounted for $P[C]$. However, from Figure 4.3 we can see that the blue partition is shared and not unique to A.

Since communication volume is a more complex metric to calculate, and cannot be evaluated on-the-fly like edge-cut, we need to expand our definitions. We define a gain vector G_i with a number of elements equal to the number of partitions, initialized to 0, where $G_i[a]$ represents the total gain obtained by moving vertex i to partition a. To

determine which contributions to apply to G_i, we will also need to count edges between a vertex and a specific partition. Let $C_i[a]$ denote the number of edges connecting i to partition a. In Figure 4.3, for example, $C_A[\text{blue}] = 1$, as A has only one edge connected to the blue partition, while $C_D[\text{blue}] = 2$, as D has two edges connected to the blue partition.

For a candidate vertex i being considered for a move, the gain resulting from moving i to a partition is evaluated by iterating over all neighboring vertices $v \in V_i^{\text{adj}}$. The contribution to G_i depends on the relationship between i, v, and their partitions. There are four cases:

Case 1. If $P[v] = P[i]$, then moving i to any partition $k \in N_{i-v}$ creates a shared inter-partition connection for v, yielding

$$G_i[k] -= w(i) \tag{4.3}$$

Case 2. If $P[v] \neq P[i]$ and $C_v[P[i]] = 1$, then moving i to any partition $k \in N_{iv}$ eliminates an inter-partition connection for v, yielding

$$G_i[k] += w(i) \tag{4.4}$$

Case 3. If $P[v] \neq P[i]$ and $C_v[P[i]] > 1$, then moving i to any partition $k \in N_{i-v}$ introduces a new inter-partition connection for v, yielding

$$G_i[k] -= w(i) \tag{4.5}$$

Case 4. If $C_i[P[i]] = 0$, i.e., i has no internal degree, then moving i to any partition $k \in N_i$ yields an additional gain contribution of:

$$G_i[k] += w(i) \tag{4.6}$$

For every neighboring vertex v, we check each cases for every partition in N_{iv}, N_{i-v}, or N_i, depending on the case; together these cases cover all possible changes in communication volume when moving a candidate vertex i.

In Section 3.4.1 the boundary set is filtered using edge-cut gain. When using VOL, we apply the same filtering with the calculations described in this section. Unlike with edge-cut, there is no general equation that determines whether moving a given vertex will yield a positive gain. Instead, we must visit all neighbors of a vertex to correctly evaluate the gains of possible moves. This requires some changes to the preparation phase before refinement.

METIS solves this by adding the gain calculation as a final step after preparation. Once the graph has been projected and all underlying structures have been recomputed, a full scan of the graph is performed to evaluate the possible gains for every vertex. Algorithm 1 presents the full procedure for evaluating the gain vectors of the boundary vertices. In order to filter out negative moves, METIS looks at the highest value in each gain vector and adds only the vertices with non-negative gain to the boundary set. Section 5.1 shows how the resulting gain vector G_i is used during refinement.

Algorithm 1 VOL gain calculation

```
nparts ← k                                          ▷ Graph with k partitions
for all i ∈ V do
    G_i ← ∅                                         ▷ Reset gain vector
    if |N_i| > 0 then                               ▷ Only visit vertices with neighboring partitions
        for all v ∈ V_i^adj do                      ▷ Visit all adjacent vertices of i
            for n = 0 to nparts − 1 do
                if n ∈ N_i AND n ∈ N_v then
                    N_iv.add(n)                     ▷ Shared partitions between i and v (N_iv)
                else if n ∈ N_i AND n ∉ N_v then
                    N_{i−v}.add(n)                  ▷ Partitions only i is connected to (N_{i−v})
                end if
            end for
            if P[i] = P[v] then
                for all n ∈ N_{i−v} do
                    G_i[n] −= w(i)                  ▷ Case 1
                end for
            else if P[i] ≠ P[v] AND C_v[P[i]] = 1 then
                for all n ∈ N_iv do
                    G_i[n] += w(i)                  ▷ Case 2
                end for
            else if P[i] ≠ P[v] AND C_v[P[i]] > 1 then
                for all n ∈ N_{i−v} do
                    G_i[n] −= w(i)                  ▷ Case 3
                end for
            end if
        end for
        if C_i[P[i]] = 0 then
            for all n ∈ N_i do
                G_i[n] += w(i)                      ▷ Case 4
            end for
        end if
        if MAX(G_i) ≥ 0 then
            BoundarySet.add(i)                      ▷ Store boundary vertices with positive max gain
        end if
    end if
end for
```

4.3 NVOL gain calculation

We now present our novel gain calculation used in NVOL. Whereas VOL aims to minimize the total communication volume of a partitioned graph, NVOL requires finer information to determine what counts as a positive gain for each partition. This is vital for NVOL's refinement, which depends on how each partition's outgoing volume changes. This section shows how to calculate the partition-specific communication volume gain and how to implement it in the multilevel k-way partitioning algorithm.

Before diving into the gain calculation, we need to define how the outgoing communication volume is computed for a specific partition. For a partition a, the

partition-specific outgoing communication volume PV is given by

$$PV[a] = \sum_{v \in V_b | P[v]=a} |N_v| \, w(v) \tag{4.7}$$

Consider Figure 4.3 again. By using Equation 4.7, we can calculate the outgoing communication volume of each partition. As before, we assume unit weights. Both vertices in the red partition satisfy $v \in V_b$, as both have edges to other partitions. Vertex A has two neighbor partitions, while B has only one, resulting in $PV[\text{red}] = 3$. The blue partition has only two boundary vertices: E and C. Just like the red partition, $PV[\text{blue}] = 3$: C has two neighbors and E has one. Finally, the green partition's only vertex has two neighbors, resulting in $PV[\text{green}] = 2$.

Whereas VOL uses a 1D gain vector G_i that summarizes the total effect of moving i to each destination, NVOL needs to know which partitions absorb that change. We therefore replace the gain vector with a 2D structure that decomposes the gain by affected partition. We introduce the gain table SG_i, where each row corresponds to an affected partition and each column to a destination partition for a vertex i. The entry $SG_i[a][b]$ represents the gain in outgoing communication volume for partition a when i is moved to partition b. This structure, in contrast to the gain vector G, is essential for performing partition-specific volume operations. In a graph with m partitions, for a move of vertex i to partition b, the total gain satisfies:

$$\sum_{t=0}^{m-1} SG_i[t][b] = G_i[b] \tag{4.8}$$

i.e., the sum of each column in SG_i equals the corresponding entry in G_i.

With this structure in place, we now describe how its entries are computed. For a candidate vertex i being considered for a move, the partition-specific gain resulting from moving i to a partition is evaluated by iterating over all neighboring vertices $v \in V_i^{\text{adj}}$. The contribution to SG_i depends on the relationship between i, v, and their partitions. Since this calculation is based on the same communication volume formulation as in Section 4.2, all volume changes are covered by the same four cases, but the operations performed in each case differ:

Case 1. If $P[v] = P[i]$, then moving i to any partition $k \in N_{i-v}$ creates a shared inter-partition connection. This affects the source partition:

$$SG_i[P[i]][k] \mathrel{-}= w(i) \tag{4.9}$$

Case 2. If $P[v] \neq P[i]$ and $C_v[P[i]] = 1$, then moving i to any partition $k \in N_{iv}$ eliminates an inter-partition connection. This contributes to partition $P[v]$:

$$SG_i[P[v]][k] \mathrel{+}= w(i) \tag{4.10}$$

Case 3. If $P[v] \neq P[i]$ and $C_v[P[i]] > 1$, then moving i to any partition $k \in N_{i-v}$ introduces a new inter-partition connection. This contributes negatively to partition $P[v]$:

$$SG_i[P[v]][k] \mathrel{-}= w(i) \tag{4.11}$$

Case 4. If $C_i[P[i]] = 0$, then moving i to any partition $k \in N_i$ yields an additional gain contribution for the destination partition:

$$SG_i[k][k] \mathrel{+}= w(i) \tag{4.12}$$

The cases above capture the per-neighbor effect of a move. They do not, however, account for the redistribution of i's own contribution to outgoing volume. Moving i removes outgoing volume from the source partition's volume and adds it to the destination's partition. This contribution depends solely on vertex i, not on any specific neighbor, so it is applied once per potential move:

$$SG_i[P[i]][k] \mathrel{+}= |N_i| \cdot w(i), \tag{4.13}$$

$$SG_i[k][k] \mathrel{-}= |N_i| \cdot w(i). \tag{4.14}$$

This is referred to as a global term operation, as it reflects the overall redistribution of communication volume between the source and destination partitions of vertex i, rather than interactions with individual neighbor vertices. Equation 4.13 represents the volume leaving the source partition, while Equation 4.14 represents the volume the destination receives. Consequently, it is computed once per vertex move, in contrast to the neighbor-specific gain contributions described earlier, which are evaluated separately for each adjacent vertex.

Beyond the addition of the global term, both strategies follow the same case structure; the principal difference lies in how they store the gain information. Rather than recording only the magnitude of the change, NVOL also tracks which partition is affected. Section 5.2 shows that the gain table is essential for enforcing the logic used in NVOL, which permits moves that improve a single partition regardless of the total communication volume.

Ideally, this calculation should be used to filter the boundary set during the preparation phase. However, due to some implementation constraints discussed in Section 6.1.3, this was not the case for NVOL. In short, the reason is that storing a gain table for each vertex would require too much memory for it to be a viable option. Instead, the gain calculation is performed during refinement, building the gain table for one boundary vertex at a time. Vertices in the boundary set are filtered using VOL's gain calculation; this is a conservative filter that may exclude some vertices NVOL would otherwise keep. Algorithm 2 shows how the gain table is calculated for a boundary vertex i under consideration.

Algorithm 2 NVOL gain calculation

```
Input: boundary vertex i under consideration
SG_i[a][b] ← 0 ∀a, b                                   ▷ Reset gain table
nparts ← k                                             ▷ Graph with k partitions
for all v ∈ V_i^adj do                                 ▷ Visit all adjacent vertices of i
    for n = 0 to nparts − 1 do
        if n ∈ N_i AND n ∈ N_v then
            N_iv.add(n)                                ▷ Shared partitions between i and v (N_iv)
        else if n ∈ N_i AND n ∉ N_v then
            N_i−v.add(n)                               ▷ Partitions only i is connected to (N_i−v)
        end if
    end for
    if P[i] = P[v] then
        for all n ∈ N_i−v do
            SG_i[P[i]][n] −= w(i)                      ▷ Case 1
        end for
    else if P[i] ≠ P[v] AND C_v[P[i]] = 1 then
        for all n ∈ N_iv do
            SG_i[P[v]][n] += w(i)                      ▷ Case 2
        end for
    else if P[i] ≠ P[v] AND C_v[P[i]] > 1 then
        for all n ∈ N_i−v do
            SG_i[P[v]][n] −= w(i)                      ▷ Case 3
        end for
    end if
    N_iv ← ∅
    N_i−v ← ∅
end for
for all n ∈ N_i do
    SG_i[P[i]][n] += |N_i| · w(i)                      ▷ Global term operations
    SG_i[n][n] −= |N_i| · w(i)
    if C_i[P[i]] = 0 then
        SG_i[n][n] += w(i)                             ▷ Case 4
    end if
end for
```

4.4 Examples

To better understand the definitions of communication volume and gain, we present a sequence of illustrative examples. Each example highlights a particular configuration and demonstrates how gain contributions arise under different vertex moves.

4.4.1 Two partition example

Consider the graph in Figure 4.4, consisting of five vertices with unit weights. The graph is partitioned into two subsets: the red partition containing vertices $\{A, B, C\}$, and the blue partition containing vertices $\{D, E\}$. Using Equation 4.1, the total communication volume is 4.

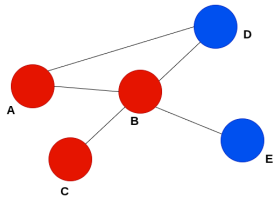


Figure 4.4: Partitioned graph with a volume of 4, each partition has an outgoing volume of 2.

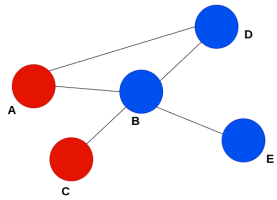


Figure 4.5: Partitinoed graph with a volume of 4, each partition has an outgoing volume of 2 after the move.

Vertex A has one edge to the blue partition and contributes 1 to the communication volume. Vertex B has two edges incident to the blue partition, but contributes only 1 under the given definition. Similarly, vertices D and E each contribute 1. In contrast, vertex C contributes nothing, as all its incident edges are internal to its partition.

Now consider moving vertex B from the red partition to the blue partition, as shown in Figure 4.5. We calculate the gain using the global strategy (Section 4.2). Let B be the candidate vertex with neighbors $\{A, C, D, E\}$. Since there are two partitions, the gain vector G_B has size two. In practice, however, G_B effectively has size one, since $G_B[\text{red}]$ is always zero: a vertex moving from its source partition to the source partition is equivalent to no move at all.

Vertex *A*. Vertices A and B are in the same partition and both already have connections to the blue partition. Hence, the set N_{B-A} is empty and no gain contribution arises.

Vertex *C*. Vertex C lies in the red partition and has no edges to the blue partition. After moving B, C gains an inter-partition edge, increasing the communication volume. As shown in Case 1, this yields a negative gain:

$$G_B[\text{blue}] -= 1.$$

Vertex *D*. Vertex D is in the blue partition meaning that a move to the blue partition would decrease its volume only if one edge were incident on the red partition. Since D has two, Case 3 does not apply, moving B does not change its contribution, hence no case applies.

Vertex *E*. Vertex E satisfies Case 2: it has exactly one connection to the red partition. After the move, this connection disappears, reducing the communication volume and yielding a positive gain:

$$G_B[\text{blue}] += 1.$$

Case 4 does not apply, as B has multiple connections within its original partition. Summing all contributions yields total gain 0, so the communication volume remains unchanged.

NVOL gain formulation. Using the formulation from Section 4.3, we obtain the same result. We use the table SG_B with size 2×1 (the source column will always be zeroed out just like the source in the gain vector), to store each volume change. As shown above only vertices C and E contribute:

$$SG_B[\text{red}][\text{blue}] -= 1, \quad SG_B[\text{blue}][\text{blue}] += 1.$$

Additionally, accounting for the direct effect of moving B via Equations 4.13 and 4.14:

$$SG_B[\text{red}][\text{blue}] \mathrel{+}= 1, \quad SG_B[\text{blue}][\text{blue}] \mathrel{-}= 1.$$

The net effect is zero, confirming that neither total nor per-partition volume changes.

4.4.2 Three partition example

We now extend the example to three partitions. Consider the graph in Figure 4.6, where all vertices have unit weight. The partitions are: red $\{A, B\}$, blue $\{C, D\}$, and green $\{E\}$. The total communication volume is 8, with partition volumes 3, 3, and 2, respectively.

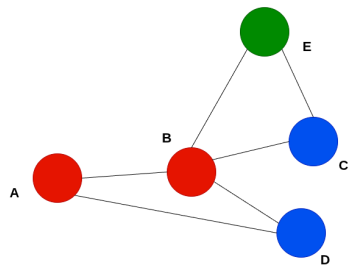


Figure 4.6: Partitioned graph with a total communication volume of 8.

Let B be the candidate vertex with neighbors $\{A, C, D, E\}$. Unlike the previous example, B may move to either the blue or green partition. Thus, G_B has size 3 and SG_B 3×2.

Vertex *A*. Vertex A is in the same partition as B. Since A is connected to the blue partition but not the green partition, Case 1 applies only for a move to green, introducing a new inter-partition edge:

$$G_B[\text{green}] \mathrel{-}= 1.$$

Vertex *C*. Vertex C is outside the red partition and has exactly one edge to it. By Case 2, removing this edge reduces the communication volume. Since C is connected to both the blue and green partitions:

$$G_B[\text{blue}] \mathrel{+}= 1, \quad G_B[\text{green}] \mathrel{+}= 1.$$

Vertex *D*. Vertex D has two edges to the red partition. By Case 3, a gain change occurs only when the destination partition is not already adjacent to D. This holds for the green partition:

$$G_B[\text{green}] \mathrel{-}= 1.$$

Vertex *E*. Vertex E is symmetrical to C, yielding:

$$G_B[\text{blue}] \mathrel{+}= 1, \quad G_B[\text{green}] \mathrel{+}= 1.$$

Summing contributions gives:

$$G_B = [0, 2, 0],$$

indicating that moving B to the blue partition reduces the total communication volume by 2, while moving it to the green partition leaves it unchanged.

NVOL gain formulation. The vector G_B alone does not capture how the volume is redistributed among individual partitions. Using Section 4.3, we compute SG_B as follows.

Vertex *A*. Vertex A follows Case 1for a move to the green partition, yielding:

$$SG_B[\text{red}][\text{green}] -= 1$$

Vertex *C*. Vertex C follows Case 2 for both moves, yielding:

$$SG_B[\text{blue}][\text{blue}] += 1$$
$$SG_B[\text{blue}][\text{green}] += 1$$

Vertex *D*. Vertex D falls under Case 3 for the green partition, yielding:

$$SG_B[\text{blue}][\text{green}] -= 1,$$

Vertex *E*. Vertex E is symmetrical to C, yielding:

$$SG_B[\text{green}][\text{blue}] += 1$$
$$SG_B[\text{green}][\text{green}] += 1$$

Additionally, accounting for the direct effect of moving B via Equations 4.13 and 4.14:

$$SG_B[\text{red}][\text{blue}] += 2, \quad SG_B[\text{red}][\text{green}] += 2,$$

$$SG_B[\text{blue}][\text{blue}] -= 2, \quad SG_B[\text{green}][\text{green}] -= 2.$$

The resulting table SG_B is:

$$\begin{array}{c} \\ \text{red} \\ \text{blue} \\ \text{green} \end{array} \begin{array}{c} \begin{array}{cc} \text{blue} & \text{green} \end{array} \\ \begin{pmatrix} 2 & 1 \\ -1 & 0 \\ 1 & -1 \end{pmatrix} \end{array}$$

Each column corresponds to a destination partition, and each row describes how a partition's volume changes. One can verify that the column sums recover G_B: the sums are 0 (the red partition is implied as no move occurs if the vertex stays in the same partition it already part of), $2 + (-1) + 1 = 2$ and $1 + 0 + (-1) = 0$, consistent with $G_B = [0, 2, 0]$. This formulation provides finer insight: although moving B to the green partition leaves the total volume unchanged, it redistributes volume across partitions. In particular, the blue partition experiences an increase in volume at the partition level.

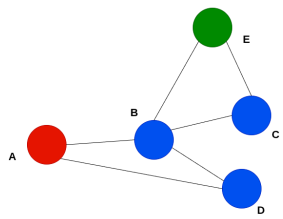


Figure 4.7: Partitioned graph has a volume of 6 after B is moved to the blue partition

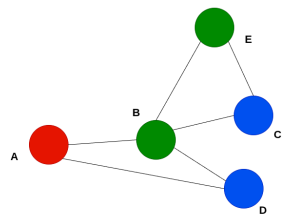


Figure 4.8: Partitioned graph has a volume of 8 after B is moved to the green partition

4.4.3 Case 4 example

None of the preceding examples illustrate a scenario in which Case 4 is relevant. To clarify how such a case may arise and how it affects the communication volume, consider Figure 4.9, where all vertices carry unit weight. The partition assignment is as follows: red $\{A, B, C, G\}$, blue $\{D, E\}$, and green $\{F\}$. The total communication volume is 8, with individual partition volumes of 3, 3, and 2, respectively. As in the previous example, let G_C have size 3 and SG_C size 3×3.

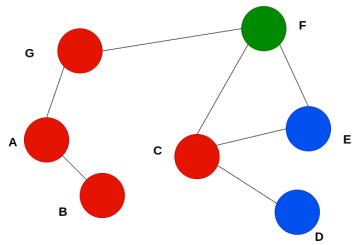


Figure 4.9: Partitioned graph with a total communication volume of 8.

The candidate vertex is C, which has no internal edges within the red partition. Despite the graph containing additional vertices, only the direct neighbors $\{D, E, F\}$ of C need to be visited in order to compute the gain. This illustrates how, even in large graphs, gain computation remains local to the immediate neighborhood of the moving vertex. For conciseness, both computation strategies are consolidated in this example, as they have been thoroughly demonstrated in earlier sections.

Vertex *D*. Vertex D satisfies Case 2 for a move to the blue partition, yielding:

$$\begin{aligned} G_C[\text{blue}] &\mathrel{+}= 1 \\ SG_C[\text{blue}][\text{blue}] &\mathrel{+}= 1 \end{aligned}$$

Vertex *E*. Vertex E satisfies Case 2 for all possible moves of C, yielding:

$$\begin{aligned} G_C[\text{blue}] &\mathrel{+}= 1 \\ G_C[\text{green}] &\mathrel{+}= 1 \\ SG_C[\text{blue}][\text{blue}] &\mathrel{+}= 1 \\ SG_C[\text{blue}][\text{green}] &\mathrel{+}= 1 \end{aligned}$$

Vertex *F*. Vertex F has two edges connecting it to the red partition; however, the set N_{C-F} is empty, meaning Case 3 does not apply and vertex F contributes no change to the gain

Case 4. Since vertex C has no internal edges, any move will reduce the volume of its source partition. Under normal circumstances, when the candidate vertex has at least one internal edge, moving it leaves the receiving partition's incoming volume unchanged through the candidate. In this case, however, it is necessary to increment the gain for

every possible destination partition, yielding:

$$\begin{aligned} G_C[\text{blue}] &+= 1 \\ G_C[\text{green}] &+= 1 \\ SG_C[\text{blue}][\text{blue}] &+= 1 \\ SG_C[\text{green}][\text{green}] &+= 1 \end{aligned}$$

The resulting vector $G_C = [0\,3\,2]$. Accounting for the direct effect of relocating C via Equations 4.13 and 4.14:

$$SG_C[\text{red}][\text{blue}] += 2, \quad SG_C[\text{red}][\text{green}] += 2,$$

$$SG_C[\text{blue}][\text{blue}] -= 2, \quad SG_C[\text{green}][\text{green}] -= 2.$$

The resulting table SG_C is:

$$\begin{array}{c} \\ \text{red} \\ \text{blue} \\ \text{green} \end{array} \begin{array}{c} \begin{array}{cc} \text{blue} & \text{green} \end{array} \\ \begin{pmatrix} 2 & 2 \\ 1 & 1 \\ 0 & -1 \end{pmatrix} \end{array}$$

Figures 4.10 and 4.11 show the resulting graphs after moving C to the blue and green partitions respectively. The reader can verify that the partition volumes in each case are consistent with the gains predicted by SG_C and G_C. Without accounting for Case 4, the gain calculation would be incomplete, potentially leading to incorrect volume estimates when a vertex has no internal edges.

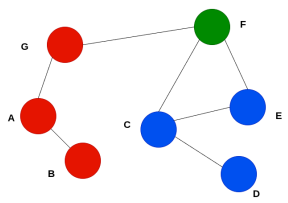


Figure 4.10: Partitioned graph has a volume of 5 after the move.

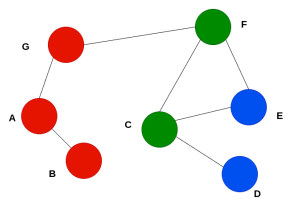


Figure 4.11: Partitioned graph has a volume of 6 after the move.

Chapter 5

Greedy Refinement Using Communication Volume

This chapter discusses how Greedy Refinement (GR) works using the total communication volume objective function (VOL) and our partition-aware communication volume objective function NVOL. While the GR algorithm itself is well-established and used by partitioners such as METIS, its integration with NVOL is a contribution of this work. Building on the gain calculations from Chapter 4, we show how GR uses these gains to decide which vertex moves to perform.

The first section presents the logic used for GR with VOL, the standard communication volume objective function used by partitioners such as METIS. The second section describes how GR works with NVOL. Each section concludes with an algorithm summarizing how GR evaluates vertex moves under the corresponding objective.

5.1 GR with VOL

In this section, we describe how GR works when using VOL as the objective function. The goal of VOL is to minimize the total communication volume of the graph. After the preparation phase is completed, refinement starts: using GR, the partitioner visits all the vertices in the boundary set.

For each boundary vertex i, all the neighboring partitions N_i are visited, checking the corresponding value in the G_i vector. A vertex move is only considered if removing i from its current partition does not violate the minimum partition weight. Among all considered moves, GR selects the one with the highest gain in G_i that is allowed by the balance constraints, i.e., the move that reduces the total communication volume the most. To prioritize high-gain moves, METIS relaxes the balance constraint in proportion to the gain. Using a constant called the fudge factor f, a move to partition p is allowed to exceed the maximum partition weight by $f \cdot G_i[p]$. This allows moves that improve the communication volume significantly, at the cost of a few extra vertices in a partition.

When two moves have the same gain, METIS breaks ties using two further criteria: external degree (see Section 3.4.1) and partition balance. It first chooses the move with the highest specific external degree. Since all vertices within a coarse vertex are assigned to the same partition upon projection (see Section 3.4.1), a move toward a partition with high external degree may reduce volume at finer levels. If instead two candidates have the same gain and the same external degree, the move to the more underweight partition

is chosen. This improves the weight balance among the partitions. Algorithm 3 shows the logic GR with VOL applies to each boundary vertex i.

Algorithm 3 GR with VOL

```
Input: boundary vertex i under consideration, with neighbor partitions N_i[0...n−1]
                                        ▷ N_i[k] denotes the k-th neighbor partition of i
if w(P[i]) − w(i) ≥ W^min then
    for k = n−1 downto 0 do                          ▷ Find first feasible candidate
        if G_i[N_i[k]] ≥ 0 AND
           w(N_i[k]) + w(i) ≤ W^max + f · G_i[N_i[k]] then
            break
        end if
    end for
    if k < 0 then continue                  ▷ Go to next vertex, no feasible candidate
    end if
    for j = k−1 downto 0 do                          ▷ Search for a better candidate
        p_j ← N_i[j], p_k ← N_i[k]
        if G_i[p_j] > G_i[p_k] AND
           w(p_j) + w(i) ≤ W^max + f · G_i[p_j] then
            k ← j                                     ▷ Better gain, relaxed balance
        else if G_i[p_j] = G_i[p_k] AND
                ED_{p_j}[i] > ED_{p_k}[i] AND
                w(p_j) + w(i) ≤ W^max then
            k ← j                                     ▷ Same gain, better connectivity
        else if G_i[p_j] = G_i[p_k] AND
                ED_{p_j}[i] = ED_{p_k}[i] AND
                w(p_j) < w(p_k) then
            k ← j                          ▷ All else equal, prefer underloaded partition
        end if
    end for
    Move i to N_i[k]
end if
```

5.2 GR with NVOL

While GR itself follows the same overall patter as the VOL variant described above, its adaptation to NVOL is a contribution of this work. The goal of NVOL is to minimize the maximum outgoing communication volume ($MV^{\max}$) held by any single partition. In Section 4.3, we explained how the gain table SG_i is computed for a boundary vertex i. To briefly recall, each entry $SG_i[p][j]$ records the change in outgoing communication volume that partition p would experience if vertex i were moved to the neighboring partition j. The rows of the table correspond to affected partitions, while the columns correspond to possible destinations.

The GR algorithm follows the same pattern as before. For each boundary vertex i, all the neighboring partitions N_i are visited, checking the corresponding values of the SG_i table. Here too, moves are only considered if removing i from its current partition does not violate the minimum partition weight. As with VOL, NVOL also enforces partition max weight constraints: any move must keep the destination partition's weight within the allowed bounds. For a potential move to a destination partition j, NVOL computes

the highest volume that any affected partition would hold after the move. If any partition exceeds the current global $MV^{\max}$, the move is immediately discarded.

NVOL will look first for a candidate move that does not increase the global $MV^{\max}$. As is further discussed in Section 5.2.1, this might lead to an increase in local $MV^{\max}$. After the first candidate has been found, NVOL looks for a better option ranked according to three criteria, applied in the following order of priority: local $MV^{\max}$, external degree, and balance. Under the first criterion, NVOL seeks a move that minimizes the local top volume. Among the possible destinations, NVOL chooses the destination j that results in the lowest top volume across the remaining affected partitions, provided that this is lower than the previous candidate. When two moves have the same local top volume, NVOL favors destinations toward which vertex i has a high external degree as a tiebreaker. This tiebreaker follows the same logic as in VOL: it favors moves where the vertex is strongly connected to the destination partition. The final criterion is used when the previous two are tied. NVOL prefers moving vertices to lighter partitions, improving the weight distribution among the partitions.

Just like VOL, NVOL uses the fudge factor to influence the weight constraint; however, it is used slightly differently. Section 5.2.1 explains how the fudge factor is exploited by NVOL to promote and prohibit moves at the same time.

Algorithm 4 shows how GR chooses a vertex move using NVOL. The algorithm consists of a main loop iterating over all possible destinations of the boundary vertex. In an inner loop, it computes the highest volume after the move. To do this, it scans all entries $SG_i[k][j]$ for $k \in N_i \cup \{P[i]\}$ and computes $PV[k] - SG_i[k][j]$, where $PV[k]$ is the volume partition k holds before the move. The gain is subtracted as a positive gain equal a decrease in volume and vice versa. We include $P[i]$ in the loop to also record how the source partition is affected by the move. The partition achieving the highest post-move volume is recorded as the local top partition, with volume `tmpVol` and partition id `tmpPid`. The remaining if-else branches encode the three ranking criteria described above. At the end, if a candidate has been found, the move is performed.

5.2.1 Encouraging more moves

A deliberate design choice in NVOL was to promote a higher number of vertex moves. Especially when choosing the first candidate, NVOL does not require an immediate improvement of the local top volume. Instead, it chooses any move that leads to a local top below the global maximum. This means that in practice NVOL also chooses moves that increase the local top volume as long as it remains below the global maximum.

The goal of this design is to increase the number of moves NVOL performs during a refinement stage. During testing, we experimented with a stricter criterion that required every accepted move to improve the local top volume. Under this stricter rule, significantly fewer vertices were moved during refinement. On average, the stricter rule performed half as many moves as the more relaxed approach. By encouraging more moves, we observed two effects. First, NVOL was able to explore more solutions: since more moves were performed, more configurations were evaluated. Second, we increased the chance of avoiding getting stuck in a local maximum. If no move could improve the partition's volume, GR would remain stuck, unable to find an improving move. By allowing a "worsening" move, we increase the chance of finding better solutions. Since the condition still guarantees that the global maximum is never exceeded, the objective function's value is protected throughout the process.

To prevent this relaxed condition from accepting moves that are too damaging, NVOL incorporates the fudge factor as a safety mechanism. The fudge factor is used to adjust

the maximum allowed partition weight; however, in NVOL we multiply it by the gain for the top partition. Specifically, for a vertex i, the partition m with the highest local volume, and a destination partition j, we multiply the fudge factor by $SG_i[m][j]$. This ensures that moves with a negative impact on the local top partition are more likely to be filtered out by the weight constraint. By making the maximum allowed weight smaller, it becomes harder to accept a move. In the opposite case, it acts as a relaxing factor, allowing moves that improve the local top. In this way, f acts as a soft penalty that discourages harmful moves and promotes positive ones.

Chapter 6 describes how NVOL was implemented giving a description of how Algorithm 4 was implemented into the METIS source code.

Algorithm 4 GR with NVOL

```
Input: boundary vertex i under consideration, with neighbor partitions N_i[0...n−1],
global maximum partition volume MV^max
if w(P[i]) − w(i) ≥ W^min then
    candidateVol ← −1
    destination ← −1
    Compute gain table SG_i
    for all j ∈ N_i do
        tmpVol ← 0
        tmpPid ← −1
        for all k ∈ N_i ∪ P[i] do
            if tmpVol< PV[k] − SG_i[k][j] then
                tmpVol ← PV[k] − SG_i[k][j]
                tmpPid ← k
            end if
        end for
        if candidateVol = -1 AND
           MV^max ≥ tmpVol AND
           w(j) + w(i) ≤ W^max + f · SG_i[tmpPid][j] then
            destination ← j                          ▷ Move does not increase MV^max
            candidateVol ← tmpVol
        else if candidateVol ≠ −1 AND
                candidateVol > tmpVol AND
                w(j) + w(i) ≤ W^max + f · SG_i[tmpPid][j] then
            destination ← j                ▷ Lower local max partition volume after move
            candidateVol ← tmpVol
        else if candidateVol ≠ −1 AND
                candidateVol = tmpVol AND
                ED_j[i] > ED_destination[i] AND
                w(j) + w(i) ≤ W^max + f · SG_i[tmpPid][j] then
            destination ← j                          ▷ Same gain, better connectivity
            candidateVol ← tmpVol
        else if candidateVol ≠ −1 AND
                 candidateVol = tmpVol AND
                ED_j[i] = ED_destination[i] AND
                w(j) < w(destination) then
            destination ← j                ▷ All else equal, prefer underloaded partition
            candidateVol ← tmpVol
        end if
    end for
    if destination ≠ −1 then                 ▷ Make sure a candidate has been found
        Move i to destination
    end if
end if
```

Chapter 6

METIS with NVOL and Supporting Tooling

Chapters 4 and 5 developed NVOL, our partition-specific communication-volume objective function, and showed how its integrated in the refinement phase. This chapter turns from theory to practice: it describes how NVOL was integrated into METIS, and the Python tooling developed alongside it to analyze and visualize the resulting partitions. The implementation and tools described here are used to produce the results presented in Chapter 7.

The first section starts with a general description of METIS and how to run it. Section 6.1.3 then details how the NVOL algorithm was integrated into METIS's C source. The second section presents the tools developed to help analyze the results. Together they form the complete experimental pipeline: the modified METIS produces the partitions, and the Python tools turn those partitions into the analyses we report in Chapter 7.

6.1 METIS

For this thesis we used METIS as our multilevel k-way partitioner. The METIS framework is open source[1] and developed at the Department of Computer Science & Engineering at the University of Minnesota [26]. Its quality and efficiency have made it widely used. The software, written in C, ships with several programs for partitioning a graph, partitioning a mesh, computing a fill-reducing ordering of a sparse matrix, and converting a mesh into a graph. Given the scope of this thesis, we worked with the graph partitioning programs. Since most of the logic behind the multilevel k-way algorithm has already been covered in Chapters 3–5, we refer the reader to those chapters. Using METIS as the base, we implemented our own objective function, adapting the C source to our needs. This adapted version of METIS with NVOL has been made public[2]. The details are presented in Section 6.1.3.

6.1.1 Inputs

METIS requires three inputs: a graph file, the number of partitions, and the objective function. A few optional flags are also available to fine-tune the algorithm.

[1] https://github.com/KarypisLab/METIS
[2] https://github.com/riccardoie/METIS-with-NVOL

The graph file must follow a specific structure. The first line is a header whose first two values are the number of vertices and the number of unique edges in the graph. The header may also contain two additional parameters specifying whether the graph has vertex/edge weights and whether it has multiple weights per vertex or edge. Further details can be found in the official manual[3]. All graphs used in this work are single-weight, with unit weights on both edges and vertices. A list of them can be found in Table 7.1 and Table 7.2. Note, however, that NVOL also supports graphs with single (non-unit) weight per vertex.

The remaining lines of the graph file describe the graph's connectivity: each line corresponds to a vertex, and its entries are the ids of the vertices it is connected to.

6.1.2 Output

Once the algorithm terminates, METIS produces a partition file. This text file contains n lines, where n is the number of vertices in the graph, and each line holds a single integer in the range $[0, k-1]$ indicating the partition the corresponding vertex was assigned to. The partition files are named graphName.part.k, where k is the number of partitions.

6.1.3 Implementing NVOL

Implementing NVOL required extending METIS's existing infrastructure. We first defined a new volume-based objective function called NVOL, which can now be passed as an argument when running the program. Since it is volume-based, NVOL reuses the coarsening and initial partitioning code of the total volume objective function (VOL) (see Sections 3.2–3.3); our extensions are confined to the refinement stage (see Section 3.4).

Before describing those extensions, we briefly review the data layout they build on. Most of METIS's data is held in two structs, `graph_t` and `ctrl_t`. The former contains the graph itself and all the structures required to represent it, while the latter holds the information METIS needs to run, such as memory allocation requirements and the number of vertices in the coarse graph. We added two new structs to METIS: one to carry per-partition information, and one for the gain table (see Section 4.3). The `pinfo_t` struct holds an integer representing a partition's outgoing volume, and `graph_t` now carries a pointer to an array of these. At the start of refinement we allocate k `pinfo_t` entries, one per partition, and update them throughout refinement to track partition-specific volumes. The second struct, `vol_refinement_table`, holds a pointer to an integer array called `gain_list`; `graph_t` in turn stores a pointer to an array of `vol_refinement_table` structs. Together, these form the two-level gain table described in Chapter 4. We opted for this structure over a plain double pointer for readability: it makes the per-partition grouping of the gains explicit at the type level.

When refinement begins, METIS allocates all the memory needed by its refinement structures. NVOL reuses these structures and adds the two described above. The per-partition information requires k integers, while the refinement table requires $k \times k$ integers. Allocating a full table for every boundary vertex would be prohibitively expensive, so we instead allocate a single shared refinement table that is repopulated for each vertex under consideration. This keeps NVOL's memory overhead only marginally higher than that of METIS with VOL.

Once allocated, these structures must be refreshed each time the graph is projected to a new level. During the preparation phase at each level, METIS zeroes out its

[3] https://github.com/KarypisLab/METIS/blob/master/manual/manual.pdf

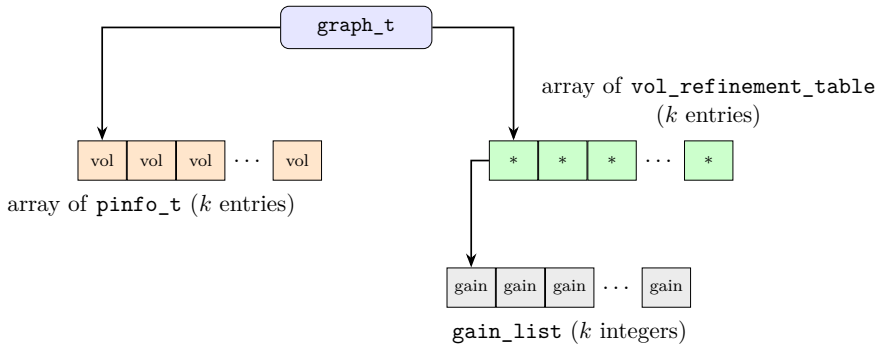


Figure 6.1: Layout of NVOL's data structures. `graph_t` stores pointers to two parallel arrays of length k (one entry per partition): an array of `pinfo_t` structs holding each partition's outgoing volume, and an array of `vol_refinement_table` structs, each of which points to its own `gain_list` of k integers. Only the first `gain_list` is drawn; the others follow the same pattern.

refinement structures and recomputes them. The graph is projected up to a finer level, and all the structs containing information about neighboring vertices, partitions, and edges must be updated. We added an NVOL branch in all relevant preparation dispatch functions. Most of the existing code was reusable, since NVOL shares the same base as VOL. When evaluating the external degree and the set of neighboring partitions of each vertex, we now also store the needed information in `pinfo_t`. The cost of these additions is negligible, since the information was already being computed and was simply not retained. For each visited vertex, we increment its partition's outgoing volume by the vertex's size once per unique neighboring partition.

With the data structures and setup work in place, we now turn to the refinement algorithm itself. The heart of NVOL lives in `kwayfm.c`, which contains all the refinement algorithms. The file's dispatching function, `Greedy_KWayOptimize`, selects a refinement algorithm based on the objective function; we added a new branch that dispatches to our refinement function, `Greedy_KWayNVolOptimize`. NVOL's implementation rests on two functions: `Greedy_KWayNVolOptimize` and `compute_potential_move_volume_change`. The former iterates over the boundary set to decide which moves to perform; it is adapted from METIS's existing `Greedy_KWayVolOptimize`, retaining the overall priority-queue-driven structure but replacing the move-selection logic to target the NVOL objective function. The latter is a new helper that supplies the per-partition volume gain calculations (see Section 4.3) required by the modified selection logic. Each is described in detail below.

Greedy_KWayNVolOptimize

`Greedy_KWayNVolOptimize` follows the same outer structure as the other greedy refinement algorithms: boundary vertices are inserted into a priority queue ordered by the objective function's value, extracted one at a time, and moved accordingly. Using partition-specific gain to order the queue would have required either storing a full table for every vertex or recomputing it twice, once when ordering and once during refinement. To avoid this duplication, we chose to order the queue by the total communication volume gain, which was already available and equals the sum of the partition-specific gains. NVOL diverges from METIS in how it chooses the destination partition.

The algorithm then iterates through the priority queue, extracting the top vertex on each pass. NVOL strictly follows the description in Section 5.2, and its implementation

closely mirrors Algorithm 4. Each time a vertex is extracted, NVOL populates the gain table using `compute_potential_move_volume_change` (described below) and then iterates over all possible destinations to find the most viable option. Each iteration follows the same pattern: identify the highest local volume that would result from the move, then evaluate the move itself. Since only the neighbor partitions of the extracted vertex experience a change in volume, an inner loop inspects only those. Within this loop, the relevant gain values in the gain table are subtracted from the corresponding current partition volumes stored in `pinfo_t`.

After the inner loop, NVOL evaluates the candidate move through a series of conditional checks. If no move has yet been accepted, we simply require that the highest local volume after the move be smaller than the highest recorded volume in the graph. This value is stored in the `graph_t` struct and updated within `compute_potential_move_volume_change`. This guarantees that any accepted move will not increase the graph's maximum per-partition outgoing volume. Once an initial candidate has been accepted, we look for the move that yields the smallest maximum local volume. Ties are broken in two stages: we first prefer destinations toward which the vertex has the greater external degree, and if those are also tied we prefer moves toward the more underweight partition. METIS's own structures already supplied all the information needed about degree and weights.

Once every destination has been evaluated and a move selected, we update the `pinfo_t` entries accordingly; if no move was accepted, the algorithm simply continues to the next boundary vertex. The remaining machinery (boundary updates, subdomain connectivity tracking, and the rebalancing branch) is left essentially unchanged from the upstream METIS code. This process repeats until the priority queue is empty. The greedy refinement as a whole is repeated a fixed number of times (10 by default); however, if no moves are made or no change in volume occurs, the algorithm exits early.

compute_potential_move_volume_change

`compute_potential_move_volume_change` is the helper that makes move selection possible. The function takes three arguments: the `ctrl_t` structure, the `graph_t` structure, and the id of the vertex under consideration, denoted i.

It implements Algorithm 4, described in Section 5.2. The function begins by zeroing out the refinement table to ensure no stale values from previous vertices remain. Since this pass already touches each of the k rows of the table, we also use it to record which partition currently has the highest volume; this value serves as the ceiling when choosing moves. The two-level structure of the refinement table behaves like a matrix: the outer index selects a row, and the `gain_list` index selects a column. Thus `vol_refinement_table[p].gain_list[j]` stores the volume change that partition p would experience if vertex i were moved to partition j. To fill this table, the function walks the neighborhood of i and updates the relevant entries according to the first three cases described in Section 4.3. Once all neighbors have been accounted for, the function handles the fourth case and the moving vertex's own volume contribution in an additional loop, adjusting the source and destination partition volumes accordingly.

Although the helper is conceptually simple, this per-vertex work is the dominant source of NVOL's runtime overhead, which the next subsection examines.

Time complexity

The additional work required by NVOL comes at a measurable cost to METIS's runtime. The two parameters that drive this slowdown are the average vertex degree and the number of partitions. The added iterations walk over both the neighboring vertices of each visited vertex and the partitions adjacent to it, so denser graphs and finer partitionings each independently inflate the work performed per refinement step. When both quantities are large simultaneously, the effects compound and the runtime penalty becomes substantial.

The source of this overhead becomes clear when we isolate the portion of the code where `Greedy_KWayNVolOptimize` and `Greedy_KWayVolOptimize` diverge. Central to the NVOL variant is the call to `compute_potential_move_volume_change`. This routine has a time complexity of $O\left(|N_i|(|N_i| + |V_i^{\text{adj}}|)\right)$, where $|N_i|$ denotes the number of neighboring partitions of a vertex i and $|V_i^{\text{adj}}|$ the number of neighboring vertices of i. Because this work is repeated for every vertex considered during refinement, even a modest constant-factor difference at the per-vertex level translates into a significant aggregate slowdown over a full pass. The same part of code in VOL, on the other hand, has a complexity of $O(|N_i|)$, simply iterating once over the neighboring partitions of the vertex.

For the smaller graphs there is no difference, but for most graphs NVOL takes on average 7-11 times more than VOL, with the biggest graphs at high partition counts being up to 60 times slower. During refinement, every visited vertex triggers a set of additional iterations that are not present in the VOL implementation of METIS. These extra passes accumulate quickly as the graph grows. The gap widens as the input size increases.

Having described how NVOL is implemented and at what cost it runs, we now turn to the Python tooling built around it for analyzing and visualizing the partitions it produces.

6.2 Graph Tools

In this section we present a few Python tools developed during this thesis to display and analyze the results produced by the partitioning algorithm, as well as to format graphs. All scripts are available in a public repository[4]. Each subsection presents one tool.

6.2.1 translate_to_metis

All graphs used in this thesis were in Matrix Market exchange format (.mtx) and therefore needed to be translated into a format METIS could use. The .mtx format is a plain-text format for representing sparse and dense matrices; when the matrix encodes a graph's adjacency, each non-zero entry corresponds to an edge. After the comment lines (prefixed with `%`) and a header, each subsequent line lists the two endpoints of an edge: for example, `1 24` denotes an edge between vertex 1 and vertex 24. Our script parses the file, discards self-loops, and stores each remaining edge as a sorted tuple in a set, which deduplicates the entries. It then builds an adjacency list by iterating over the edge set and, for each edge (u, v), appending each endpoint to the other's neighbor list. Since Matrix Market files are 1-indexed while Python lists are 0-indexed, we offset the list indices by one while keeping the vertex ids themselves 1-indexed, as required by

[4] https://github.com/riccardoie/Graph-tools/tree/main

the METIS format. Once all edges have been processed, the adjacency list is written as a METIS graph file.

6.2.2 calc_partitions

To verify and analyze the per-partition outgoing communication volumes produced by our experiments, we wrote a small program, `calc_partitions`, that computes them directly from the partition files. The program contains a function, `outgoing_volstats`, which takes two arguments: the original graph filename and the path to the folder containing the partition files. Given these inputs, the function computes the volumes for every `.part.`k file associated with that graph. The results are written to a text file divided into sections, one per partition file, each introduced by a header of the form `graphName.part.`k. Within each section, the partitions are listed in order of partition id, from 0 to $k-1$.

For every partition file, the function initializes an array of length k and then iterates over the graph's vertices, collecting for each vertex the set of distinct partitions among its neighbors that differ from its own; `outgoing_volstats` adds the size of this set to the entry for the vertex's own partition. Using a set ensures each external partition is counted at most once per vertex, matching the definition of communication volume from Section 4.3. Once the graph has been fully traversed, the resulting array is stored in a dictionary keyed by `graphName.part.`k and written to the output file.

6.2.3 form_statistics

To aggregate the per-partition volumes computed by `calc_partitions` without recomputing them, we wrote the script `form_statistics`, which processes the text files produced by that program. For each input file, `form_statistics` reads the contents line by line and parses two types of entries: lines containing `.part.`, treated as headers marking the start of a new partition file's section, and lines starting with `Pid`, each containing the outgoing volume of the corresponding partition.

The values are grouped by section in a dictionary, and for each group we compute the total, maximum, average, and standard deviation of the per-partition outgoing volume. The results are written to a CSV file, one per input file.

6.2.4 graph_results_vol

To evaluate the distribution of per-partition outgoing volumes, we wrote a simple script that plots them for a given partition file. The script takes three inputs: the graph file, the number of partitions, and an identifier specifying which partition file folder to read. It reads the file produced by `calc_partitions` and collects all values from the section matching the requested number of partitions. For comparison, the script also reads the corresponding VOL volumes from a hardcoded path; the two sets of volumes are then plotted side by side as a bar chart.

6.2.5 stats_vol

`stats_vol` reads the files produced by `form_statistics` and plots their results. It takes a graph filename and an identifier specifying which partition file folder to read as input, and compares the resulting statistics against the corresponding VOL reference values. It produces four plots: the total outgoing communication volume, the maximum partition

volume, the percentage difference in maximum partition volume between NVOL and VOL, and the standard deviation of the partition volumes.

6.2.6 model_estimates

`model_estimates` estimates the resulting per-partition communication time induced by a partitioning. We use these values to compare how NVOL's partitions perform against VOL's; reducing the communication time should let the partitioned application finish faster. The formulas implemented in this program are taken from [46]. They estimate the resulting per-process communication time while accounting for the network bandwidth contention caused by competing Message Passing Interface (MPI) processes. We specifically use Equation 6.2 from [46], which models processes with multiple neighbors and non-uniform message sizes. In this model, each process is assigned a partition and is symmetrically connected through a network whose accessible bandwidth is reduced by contention among the processes.

To understand the program, a few definitions are needed. All calculations in the model are based on the incoming communication volume of a partition. Whereas outgoing communication volume measures the data a partition must send out, incoming communication volume measures the data it must receive from outside. Using the notation from Chapter 4, the incoming communication volume of partition n is:

$$\text{incoming volume}[n] \;=\; \sum_{\substack{v \in V_b \\ n \in N_v}} w(v) \tag{6.1}$$

where V_b is the set of boundary vertices and N_v is the set of neighboring partitions of v, excluding v's own partition $P[v]$. In other words, every boundary vertex v contributes its weight $w(v)$ to the incoming volume of partition n if n is one of v's neighboring partitions. Each process is assigned a partition, so the incoming volume V of a process is the incoming volume of its assigned partition.

To compute the estimates as described in [46], we sort the incoming volumes in ascending order, $V_0 \leq V_1 \leq \cdots \leq V_{N-1}$. For each process i we also record the number of its neighboring processes M_i^{in} and the message size $s_{i,j}$ sent from process j to i. The formulas additionally require a start-up latency τ and a network bandwidth array BW. With these in hand, we compute:

$$T_i = M_i^{\text{in}} \cdot \tau + \max\left(t_i^{\text{recv}},\; t_{i,0}^{\text{send}},\; t_{i,1}^{\text{send}},\; \ldots,\; t_{i,M_i^{\text{out}}-1}^{\text{send}}\right). \tag{6.2}$$

The model estimates the resulting communication time for process i based on two components: the latency of sending messages, and the longest time it takes to finish communicating with other processes. The latter is computed by estimating both the time to receive all incoming messages and the time to deliver all outgoing messages, then taking the maximum. The time process i takes to receive its messages is defined by Equation 6.3:

$$\begin{aligned} t_0^{\text{recv}} &= \frac{N \cdot V_0}{BW_{MP}(N)}, \\ t_i^{\text{recv}} &= t_{i-1}^{\text{recv}} + \frac{(N-i)\,(V_i - V_{i-1})}{BW_{MP}(N-i)}, \quad i = 1, 2, \ldots, N-1. \end{aligned} \tag{6.3}$$

To compute the communication time it takes to deliver a message, $t_{i,j}^{\text{send}}$, the calculation is reversed: the time for i to send to j equals the time for j to receive from i.

Equation 6.4 shows how this is computed; the incoming messages $s_{i,j}$ are ordered by value, $s_{i,0} \leq s_{i,1} \leq \cdots \leq s_{i,M_i^{\text{in}}-1}$:

$$\begin{aligned} t_{i,0}^{\text{recv}} &= \frac{M_i^{\text{in}} \cdot s_{i,0}}{V_i} \cdot t_i^{\text{recv}}, \\ t_{i,j}^{\text{recv}} &= t_{i,j-1}^{\text{recv}} + \frac{(M_i^{\text{in}} - j)\,(s_{i,j} - s_{i,j-1})}{V_i} \cdot t_i^{\text{recv}}, \quad j = 1, 2, \ldots, M_i^{\text{in}} - 1. \end{aligned} \tag{6.4}$$

For a more detailed description of these formulations, we refer the reader to [46].

With the formulations in place, we can turn to the implementation. Our program takes four arguments: the graph, an identifier specifying which partition file folder to read, the number of partitions, and the startup latency. The network bandwidth values are hardcoded in the program; however, they could easily be exposed as an argument later. Both the startup latency and the network bandwidth values were kindly provided by the authors of [46]. These bandwidth values describe a realistic example of the accessible network bandwidth as a function of the number of competing processes.

The program begins by computing the incoming communication volume for each process. From these volumes, together with the network bandwidth and latency arguments, we estimate how long each process takes to complete its communication step. The formulas operate on the incoming volumes sorted from lowest to highest and evaluate the receive and send times for each process: Equation 6.3 estimates the total communication time a process spends receiving messages, while Equation 6.4 gives the time to send a message to a specific destination.

Equation 6.2 then combines these results: it takes the maximum of the receive time and the individual send completion times, and adds the startup latency multiplied by the number of neighboring processes. The idea behind taking the maximum time is that a process can only progress as fast as its slowest communication operation. The latency captures the fact that every message exchange incurs a fixed startup cost regardless of the volume transferred. All three formulas are applied to every process, and the results are plotted as a bar chart.

For comparison, the program calculates the estimate on two partitionings: the one produced using NVOL and the one produced using VOL. Both sets of results are shown in the same chart for easy comparison. In Section 7.4 we look at some of the charts produced by `model_estimates`.

Chapter 7

Results

This chapter presents the results of running METIS with the NVOL objective function. Throughout this chapter we will compare these results to partitionings using METIS's own total communication volume objective function(VOL). To generate the graphs and plots in this chapter, the tools described in Chapter 6 were used. These results show the impact of applying the partition-aware objective function NVOL and compare whether this leads to any improvements in the resulting partitioning.

The chapter follows this structure. The first section briefly describes the selection of graphs we have used for our tests. They are divided into two sets: the Heart dataset and the SuiteSparse Matrix Collection. The following section examines the maximum per-partition volume ($MV^{\max}$) resulting from partitioning the graphs using NVOL and VOL. Here we also study how well the partitionings perform on other metrics, such as standard deviation and average volume. The third section shows how METIS can be tuned to affect refinement, and how NVOL reacts to this tuning, once again by looking at the partitioned graphs. In the final section we present the resulting communication time estimates for the partitioned graphs. Here too, we analyze the results to understand why NVOL performs as it does.

7.1 Graphs used for testing

To evaluate NVOL we used symmetric matrices drawn from two datasets, treating each matrix as the adjacency representation of an undirected graph. A symmetric matrix is a matrix where element $a_{ij} = a_{ji}$, representing an undirected edge between i and j. Following our definition of GPP in Section 2.1, we use therefore such matrices, it is however, possible to form graphs from other kinds of matrices as well. The two datasets were chosen to cover a wide range of graph sizes and to include instances arising from different application domains. We tried to prioritize matrices with fewer non-zero elements, or with a higher average degree (the degree represents the number of edges a vertex has).

The first dataset is taken from *Repository for modelling acute myocardial ischemia* [35], in which the authors simulate acute myocardial ischemia in a human heart using the Chaste finite-element framework. We simplify the name to Heart dataset. The matrices produced with the data in [35] were then translated into a format accepted by METIS. Table 7.1 contains the seven graphs, ranging from roughly 10^3 to almost 10^7 vertices. For each graph it reports the number of vertices, the number of edges, the maximum and minimum vertex degree, and the standard deviation of the vertex degree.

Graphs	Vertices	Edges	Max degree	Min degree	Avg. degree	Std. dev.
heart01	1,394	6,946	21	5	10	2.1
heart02	45,384	275,661	24	5	12	3.4
heart03	296,774	1,948,530	25	5	13	3.1
heart04	555,433	3,715,278	26	5	13	3.0
heart05	1,257,588	8,574,893	25	5	14	2.8
heart06	4,041,680	28,138,550	25	5	14	2.6
heart07	9,349,543	65,800,809	26	4	14	2.6

Table 7.1: Heart dataset [35]

The second data set was retrieved from *The SuiteSparse Matrix Collection* [11]. This collection contains a large variety of matrices and keeps growing. We chose a subset of matrices that were symmetric with different numbers of vertices and with varying average degree. These matrices were then translated from Matrix Market to a format METIS supports. Table 7.2 summarizes the 13 matrices we chose ordered by vertex number.

Graphs	Vertices	Edges	Max deg	Min deg	Avg. deg	Std. dev.
pa2010	421,545	1,029,231	64	1	4	2.6
coPapersDBLP	540,486	15,245,729	3,299	1	56	66.2
packing -500x100x100 -b050	2,145,852	17,488,243	18	0	16	2.8
channel -500x100x100 -b050	4,802,000	42,681,372	18	6	18	1.0
hugetric-00000	5,824,554	8,733,523	3	2	3	0.03
italy_osm	6,686,493	7,013,978	9	1	2	0.4
hugetric-00020	7,122,792	10,680,777	3	2	3	0.03
rgg_n_2_23_s0	8,388,608	63,501,393	40	0	15	3.9
delaunay_n23	8,388,608	25,165,784	28	3	6	1.3
hugetrace-00020	16,002,413	23,998,813	3	2	3	0.02
delaunay_n24	16,777,216	50,331,601	26	3	6	1.3
kmer_V2a	55,042,369	58,608,800	39	1	2	0.7
kmer_U1a	67,716,231	69,389,281	35	1	2	0.4

Table 7.2: Selection of matrices from SuiteSparse Matrix Collection [11]

Note that rgg_n_2_23_s0 and packing-500x100x100-b050 have a minimum degree of 0. That means that at least one of the vertices is isolated with no neighbors. We have purposely added two such graphs to test how NVOL performs on this type of graphs as well.

7.2 Maximum outgoing volume per partition

Using the graphs listed above, we ran METIS with two objective functions: NVOL (see Section 5.2) and VOL (see Section 5.1). For each run we then used `calc_partitions` to analyze the resulting partitions and compare their outgoing volumes. We partitioned the graphs into 2 to 2048 partitions; however, due to their size, not all graphs were partitioned up to 2048.

The following sections will go through the results from each dataset, specifically comparing $MV^{\max}$ between VOL and NVOL. $MV^{\max}$ represents the single highest

outgoing volume of any partition in a partitioned graph. Using $MV^{\max}_{\text{VOL}}$ as the baseline, we report each value as the percentage difference of $MV^{\max}_{\text{NVOL}}$ relative to $MV^{\max}_{\text{VOL}}$, $(MV^{\max}_{\text{VOL}} - MV^{\max}_{\text{NVOL}})/MV^{\max}_{\text{VOL}} \times 100$. Cells are color-coded for readability: blue indicates that NVOL achieves a lower maximum (better), while orange indicates a higher maximum (worse). Shading intensity reflects the magnitude of the difference, with darker shades marking larger improvements or regressions. Some cells are marked with N/A, which simply means that the graph was too small for that number of partitions (k), cells marked N/R indicate runs that did not finish discussed further in Section 7.2.2.

7.2.1 Heart dataset

Graphs	2	4	8	16	32	64	128	256	512	1024	2048
heart01	-1.30	**1.04**	**14.40**	**6.49**	**0.00**	N/A	N/A	N/A	N/A	N/A	N/A
heart02	**0.79**	**5.93**	**6.82**	**3.38**	**10.44**	**10.38**	**17.57**	N/A	N/A	N/A	N/A
heart03	**1.78**	**3.85**	**3.21**	**2.91**	**8.42**	**7.08**	**9.47**	**8.96**	**5.34**	N/A	N/A
heart04	-0.18	**4.82**	**3.39**	**2.68**	**4.49**	**4.55**	**2.17**	**5.71**	**6.08**	N/A	N/A
heart05	**0.42**	**1.78**	**3.61**	**5.66**	**5.23**	**6.27**	**9.69**	**1.68**	-1.16	**6.91**	N/A
heart06	**0.03**	**2.01**	**2.20**	**2.43**	**5.83**	**4.70**	**5.47**	**9.65**	**4.26**	**8.83**	N/A
heart07	-0.10	**1.58**	**2.72**	**3.56**	**5.85**	**5.26**	**4.92**	**4.59**	**7.90**	**3.53**	**2.75**

Table 7.3: Percentage difference in $MV^{\max}$ between NVOL and VOL for the Heart dataset across different k. Each value is the result of $(MV^{\max}_{\text{VOL}} - MV^{\max}_{\text{NVOL}})/MV^{\max}_{\text{VOL}} \times 100$. Cells marked with N/A were not run.

Table 7.3 shows the results for the Heart dataset. The first thing to notice is that NVOL improves $MV^{\max}$ in almost all runs. On average, NVOL achieves a 4.70% lower $MV^{\max}$ than VOL across all runs, with improvements lying mostly between 2% and 7% and peaking at 17.57%. Table 7.3 records in total 4 losses, averaging 0.69% across them with a peak at 1.30%. As was already apparent from Table 7.1, these graphs are very similar in structure: they have comparable standard deviations and a very close average number of edges per vertex.

This similarity is also reflected in the partitioning results, as most values within each column are relatively close to one another, with only a few exceptions. Looking at the average value per graph confirms this too. Table 7.4 shows the average percentage difference of $MV^{\max}$ for the Heart dataset. All seven averages fall within a narrow 4.15% range, from heart04 at 3.75% to heart02 at 7.90%, reinforcing that the graphs behave similarly under partitioning.

Graphs	**Avg. difference**	**Best result**	**Worst result**
heart01	4.13%	14.40%	−1.30%
heart02	7.90%	17.57%	0.79%
heart03	5.67%	9.47%	1.78%
heart04	3.75%	6.08%	−0.18%
heart05	4.01%	9.69%	−1.16%
heart06	4.54%	9.65%	0.03%
heart07	3.87%	7.90%	−0.10%

Table 7.4: Average, best, and worst percentage difference in $MV^{\max}$ per graph for VOL and NVOL (Heart dataset).

Notably, NVOL generally performs better for higher k values in each graph. On

average, NVOL performs two to three times as many moves as VOL during a single refinement round, and the number of partitions clearly amplifies this effect. This allows NVOL to explore more options and increases the chance of finding a better partitioning. Additionally, since more partitions lead to more moves, they also reduce the likelihood of getting stuck in a local minimum.

The runs where NVOL loses are: heart01 with $k = 2$, heart04 with $k = 2$, heart05 with $k = 512$, and heart07 with $k = 2$. Almost all of the failures occur when $k = 2$. A likely cause is that, at each refinement step, NVOL sets a ceiling on the outgoing volume of a partition and then allows any move in which the affected partitions do not exceed this maximum. When the number of partitions is large, it is easier to find a move that decreases this value. However, with only two partitions, it is more likely that no available move can improve it. NVOL therefore tends to get stuck unable to lower $MV^{\max}$, losing to VOL for this metric.

Interestingly, Figure 7.1 reveals that in every run with $k = 2$, the standard deviation of the partition volumes for NVOL is 0. This means that both partitions produced by minimizing NVOL have exactly the same outgoing volume. By only allowing moves that maintain or decrease $MV^{\max}$, possibly at the cost of partitions with lower outgoing volumes, NVOL drives the partitions closer together. The values in Figure 7.1 have been normalized using the average outgoing volume of the partitions.

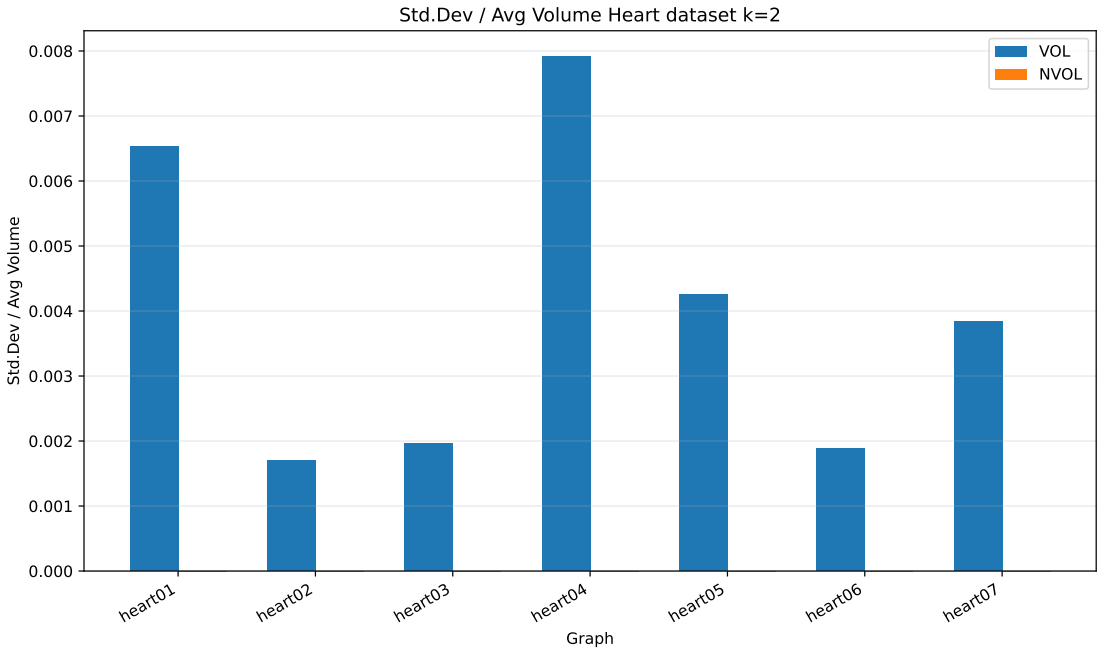


Figure 7.1: Standard deviation normalized by the average volume for runs with $k = 2$ on the Heart dataset. VOL (blue) shows nonzero variation across graphs; NVOL (orange) is 0 on every graph, meaning its two partitions have identical outgoing volume.

NVOL achieves a slightly lower standard deviation than VOL across almost all k, indicating a more even distribution of outgoing volume across partitions. All the partitioned graphs in the Heart dataset seem to follow the same pattern where the standard deviation peaks at $k = 8$ and then gradually decreases as the number of partitions grows, regardless of objective function. NVOL maintains an edge throughout, generally having the best difference for $k = 2$ and $k = 4$ runs. Table 7.5 presents the standard deviations for VOL and NVOL on heart04, which sits in the middle of the

dataset and is representative of this behavior; the other graphs show similar numbers.

Partitions	VOL std. dev.	NVOL std. dev.	VOL tot. vol.	NVOL tot. vol.
2	34.5	0.0	8,717	8,802
4	529.48	410.57	17,241	17,143
8	856.11	832.52	26,791	26,805
16	670.39	652.02	40,485	40,383
32	491.23	488.32	60,323	59,879
64	340.87	338.55	88,753	88,676
128	238.4	231.87	128,651	127,155
256	162.38	157.77	184,231	183,157
512	110.09	105.71	257,257	255,394

Table 7.5: Comparison of the standard deviation of the outgoing volumes and total volume on heart04 graph for VOL and NVOL

Both objective functions produce similar distributions of values, but NVOL consistently yields lower standard deviation. This is because NVOL favors moves that bring partition volumes closer together, whereas VOL's purely greedy approach optimizes only for volume reduction. As a result, NVOL's partition volumes are more evenly balanced, especially compared to VOL.

Table 7.5 also reveals that optimizing NVOL yields a lower total communication volume in many runs. This is not just the case for heart04, but across the entire Heart dataset. On heart04, NVOL loses twice ($k = 2$ and $k = 8$) on total communication volume. When it wins, the improvement is small, but notable: NVOL beats VOL on the very metric VOL optimizes for. Given the regular structure of the heart graphs, it appears that the better-balanced partitions produced by optimizing NVOL tend to indirectly reduce total communication volume as well.

This increased balance does come with some trade-offs. Figure 7.2 shows a plot of the outgoing volumes of heart04 when partitioned with $k = 4$. The blue bars represent VOL's partitions outgoing volumes and the orange ones NVOL. By looking closely at each bar one might notice that NVOL's bars are all higher than VOL's apart from the tallest one. This makes sense as NVOL moves the vertices in a way that pushes down the maximum potentially at the cost of other lower volume partitions. As a result the pattern shown in Figure 7.2 can be found in most results from NVOL.

Aside from $k = 2$ cases discussed earlier, heart05 with $k = 512$ is the only graph–partition combination in the Heart dataset where minimizing NVOL yields a higher $MV^{\max}$ than VOL. The exact cause is hard to pin down from the data alone; NVOL probably made some moves from which no further improvement could be found. However, with some simple tuning (Section 7.3), NVOL recovers a win on this case, suggesting it had indeed settled into a local minimum it could not escape. Even in this failure, the loss is narrow: NVOL exceeds $MV^{\max}_{\text{VOL}}$ by only 1.16%. It does, however, yield 1% lower total communication volume and a lower standard deviation than VOL.

Throughout the Heart dataset, NVOL has the best $MV^{\max}$ in 57 out of 61 runs (4.70% on average, peaking at 17.57%) and yields more evenly balanced partition volumes in almost every run. It also yields slightly lower total communication volume in many cases, despite not optimizing for it directly. The main weakness is at $k = 2$, where the constrained move space limits NVOL's ability to lower $MV^{\max}$. Minimizing NVOL does, however, produce perfectly even partitions in these cases.

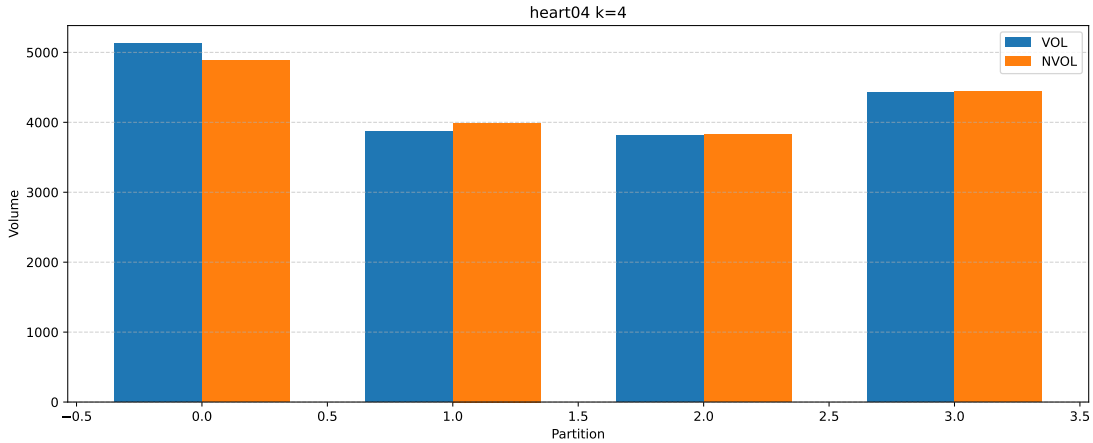


Figure 7.2: Comparison of the outgoing communication volumes of the partitions of heart04 $k = 4$ for VOL (blue) and NVOL (orange).

7.2.2 SuiteSparse Matrix Collection

Graphs	2	4	8	16	32	64	128	256	512	1024	2048
pa2010	-5.02	**6.69**	**10.11**	**3.08**	**12.54**	**11.03**	**15.92**	**9.46**	**16.03**	N/A	N/A
coPapersDBLP	-32.09	-17.81	-8.82	**1.78**	**0.41**	**24.10**	**40.37**	**48.44**	**56.70**	N/A	N/A
packing -500x100x100 -b050	**3.61**	**0.02**	**4.42**	**5.11**	**8.49**	**5.76**	**9.61**	**13.64**	**9.37**	**11.97**	N/A
channel -500x100x100 -b050	**10.67**	**8.79**	**10.32**	**9.83**	**9.92**	**13.69**	**11.57**	**9.82**	**8.54**	**9.72**	**3.16**
hugetric-00000	**9.02**	**12.71**	**12.24**	**14.62**	**12.94**	**7.63**	**16.01**	**18.53**	**1.76**	**16.23**	**19.72**
italy_osm	**0.00**	-1.02	**0.90**	-1.09	**8.87**	**4.00**	**3.85**	-0.94	-21.21	-6.82	**2.86**
hugetric-00020	**7.50**	**7.05**	**7.47**	**7.27**	**8.39**	**8.05**	**14.48**	**10.97**	**13.83**	**10.95**	**20.52**
rgg_n_2_23_s0	-6.46	**4.47**	**6.86**	**6.33**	**4.84**	**6.47**	-0.64	**8.16**	**18.60**	**7.30**	**0.0**
delaunay_n23	-1.48	**3.19**	**3.35**	**4.53**	**3.12**	**8.54**	**6.98**	**6.47**	**8.33**	**6.05**	**5.88**
hugetrace-00020	**8.27**	**8.34**	**7.07**	**7.41**	**8.51**	**8.46**	**11.12**	**10.38**	**13.29**	**12.95**	**15.63**
delaunay_n24	-2.60	**4.71**	**5.97**	**2.40**	**6.23**	**5.93**	**4.85**	**7.98**	**5.01**	**2.93**	**5.01**
kmer_V2a	-0.28	**10.34**	**20.27**	**3.98**	**1.53**	**11.14**	-4.49	**7.94**	**18.98**	**18.00**	N/R
kmer_U1a	**0.72**	**6.71**	**14.18**	**26.55**	**29.76**	**35.88**	**28.85**	**46.34**	**34.06**	**37.64**	N/R

Table 7.6: Percentage difference in $MV^{\max}$ between NVOL and VOL for the SuiteSparse Matrix Collection selection across different k. Each value is the result of $(MV^{\max}_{\text{VOL}} - MV^{\max}_{\text{NVOL}})/MV^{\max}_{\text{VOL}} \times 100$. Cells marked with N/R and N/A were not run.

Now consider the results for the SuiteSparse Matrix collection in Table 7.6. The graphs vary considerably in size and structure, and so do the results. Regardless, NVOL is able to achieve a lower $MV^{\max}$ than VOL 89% of the time. Compared to the Heart dataset, NVOL wins more convincingly with an average improvement of 9.19% across all runs. Minimizing NVOL leads to an improvement above 20% on several runs with a peak at 56.70% for coPapersDBLP with $k = 512$. However, NVOL loses more often and by larger margins than on the Heart dataset. In the 15 recorded losses NVOL averages 7.40% loss with a peak at 32.09% for coPapersDBLP with $k = 2$. Most losses are concentrated in the italy_osm graph and for several runs with $k = 2$.

Table 7.7 reports the average per graph, where once again the graph difference is represented by the spread. Apart from italy_osm, NVOL wins on average by at least 4.34% and on the biggest graph, kmer_U1a, it wins on average by 26.07%. The table,

however, does not seem to suggest any obvious relationship between graph properties (size, average degree, or degree variance) and the magnitude of the improvement, as the two next best winners are very different from each other.

Graphs	**Avg. difference**	**Vertices**	**Avg. deg**	**Std. dev.**
pa2010	8.87%	421,545	4	2.6
coPapersDBLP	12.56%	540,486	56	66.2
packing-500x100x100-b050	7.20%	2,145,852	16	2.8
channel-500x100x100-b050	9.64%	4,802,000	18	1.0
hugetric-00000	12.86%	5,824,554	3	0.03
italy_osm	−0.96%	6,686,493	3	0.4
hugetric-00020	10.59%	7,122,792	3	0.03
rgg_n_2_23_s0	5.59%	8,388,608	15	3.9
delaunay_n23	5.00%	8,388,608	6	1.3
hugetrace-00020	10.13%	16,002,413	3	0.02
delaunay_n24	4.34%	16,777,216	6	1.3
kmer_V2a	8.74%	55,042,369	2	0.7
kmer_U1a	26.07%	67,716,231	2	0.2

Table 7.7: Average percentage difference in $MV^{\max}$, vertex number, average degree, and standard deviation of vertex degree per graph (SuiteSparse Matrix Collection).

Graphs	**VOL std. dev.**	**NVOL std. dev.**	**VOL tot. vol.**	**NVOL tot. vol.**
pa2010	6.5	0.0	425	460
coPapersDBLP	8,854.0	5.5	166,524	243,349
rgg_n_2_23_s0	36.0	0.0	8,684	9,322
delaunay_n23	2.0	0.0	5,672	5,760
delaunay_n24	5.0	0.0	8,302	8,528
kmer_V2a	2,876.5	0.0	265,323	271,834

Table 7.8: Comparison of the standard deviation of the outgoing volumes and total volume of VOL and NVOL for $k = 2$ runs where NVOL is worse in Table 7.6.

In total, NVOL loses 15 of 136 runs. Setting italy_osm aside (discussed below), this drops to 10 losses, 6 of which occur at $k = 2$. As with the Heart dataset, NVOL achieves a more even volume distribution at the cost of a higher per-partition maximum on these specific $k = 2$ runs. Because the standard deviations vary across orders of magnitude for VOL, we report them in Table 7.8 rather than plot them as in Figure 7.1. The table clearly shows how VOL creates disparity between the partitions with extremes like coPapersDBLP where it has a standard deviation 8,848.5 above NVOL's.

As opposed to the Heart dataset, minimizing NVOL does not improve the total communication volume for the runs in Table 7.8. Most of the results are close to each other, except coPapersDBLP where NVOL's partitioning total volume is 1.5 times greater than VOL. This does however seem reasonable given that coPapersDBLP has a very high degree variance, meaning that the amount of edges each vertex has is very different from vertex to vertex. This can explain why even when perfectly balanced, the high degree variance greatly affects the communication volume, making NVOL lose badly.

In general, partitioning with NVOL achieves a lower standard deviation than VOL across all graphs. The picture is more mixed for total communication volume: both kmer graphs and coPapersDBLP show visibly higher total volumes under NVOL. On channel-500x100x100-b050, packing-500x100x100-b050, hugetric, and hugetrace, on the other hand, NVOL yields a lower total communication volume. This is interesting, since these are also the graphs where NVOL achieves the most consistent improvements on $MV^{\max}$. For the remaining graphs, the total volumes are close to VOL's, either slightly higher or slightly lower. Table 7.9 shows this behavior on a selection of graphs for $k = 32$.

Graphs	**VOL tot. vol.**	**NVOL tot. vol.**	**VOL std. dev.**	**NVOL std. dev.**
pa2010	6,739	6,718	44.84	39.73
coPapersDBLP	973,564	1,486,409	8,017.1	4,028.23
channel -500x100x100 -b050	389,549	359,277	1,601.68	1,381.41
hugetric-00020	32,272	29,826	152.29	141.36
delaunay_n24	76,489	76,241	499.86	488.09
kmer_U1a	1,212,680	1,235,309	4,973.48	1,385.25

Table 7.9: Comparison of the total communication volume and the standard deviation of the outgoing volumes on a selection of graphs from Table 7.2 using VOL and NVOL for $k = 32$.

Looking at Table 7.6, we see that structurally similar graphs yield similar results, just as in the Heart dataset. The different structural properties of these graphs are reflected in how well NVOL manages to lower $MV^{\max}$. Of all the graph types tested, NVOL struggles mostly with italy_osm, the only road network in the dataset. For the others it works consistently well, lowering $MV^{\max}$ and distributing the volume more evenly than VOL across partitions.

italy_osm graph

NVOL struggles greatly with the italy_osm graph, achieving its second worst result across all our runs of -21.21% at $k = 512$ and failing to improve on VOL's standard deviation in most runs. The italy_osm graph represents the Italian road network extracted from OpenStreetMap, with vertices placed at intersections and edges corresponding to the road segments between them. Road networks differ structurally from the regular graphs of the Heart dataset in several important ways. They tend to have high sparsity, with both average and maximum vertex degrees in the single digits. More importantly, they are strongly heterogeneous: dense urban clusters are connected by long, thin stretches of rural roads, and natural features such as rivers, mountains, and coastlines act as small separators between regions of very different sizes.

This heterogeneity makes it noticeably harder for NVOL to push down $MV^{\max}$ while keeping partitions balanced. A partition containing a dense urban area cannot easily be matched in volume to one covering a sparse rural region, so the moves NVOL relies on to equalize partition volumes have limited room to operate. The low average degree compounds the issue, since each move affects only a handful of edges and therefore shifts the outgoing volume by very little, leaving NVOL with little leverage to escape configurations where the maximum cannot be lowered.

Despite the negative overall picture, NVOL is not uniformly worse on this graph:

wins and losses occur in roughly equal numbers, with a best result of 8.87% at $k = 32$. The losses are simply larger in magnitude than the wins, which drags the average into negative territory. Figure 7.3 makes this asymmetry visible, a handful of moderate wins offset by a dramatic -21.21% loss at $k = 512$. This suggests NVOL can occasionally find a favorable configuration even on irregular structures, but the sparse and heterogeneous nature of road networks does not seem to benefit from its partition-aware optimization; if anything, it works against it.

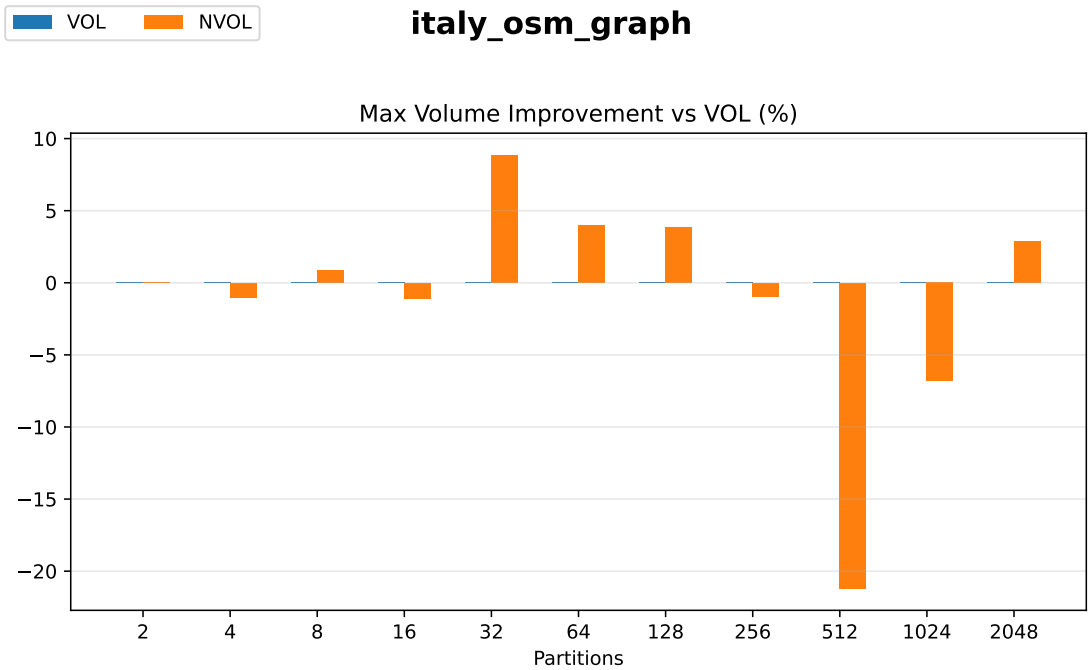


Figure 7.3: Percentage difference in $MV^{\max}$ between NVOL and VOL for italy_osm graph across different k. Each value is the result of $(MV^{\max}_{\text{VOL}} - MV^{\max}_{\text{NVOL}})/MV^{\max}_{\text{VOL}} \times 100$.

coPapersDBLP graph

The graph with the best and worst result in Table 7.6 is coPapersDBLP. coPapersDBLP is a citation network derived from the DBLP computer science bibliography. Citation networks have a different structure from the heart matrices and road networks discussed earlier. They are scale-free, with a heavy-tailed degree distribution: a small number of hub vertices carry a disproportionate share of the connections, while a long tail of vertices have only a handful. This is reflected in coPapersDBLP itself, where the maximum degree of 3,299 sits nearly two orders of magnitude above the average degree of 56. Citation networks are also small-world, with short average path lengths despite the network's size, and exhibit strong community structure corresponding to research areas or fields. This means that the average shortest distance between any two vertices grows very slowly with the graph size.

The results seem to follow a trend where they improve as the number of partitions increases. Despite the 3 negative results, the positive ones outweigh them, giving the graph an average difference of 12.56%, the third highest in Table 7.7. Minimizing NVOL yields much more balanced results here as well. Table 7.8 already showed that at $k = 2$ NVOL achieves a standard deviation of 5.5 against VOL's 8,854.

These large differences persist for $4 \leq k \leq 32$. Figure 7.4 shows the normalized standard deviations of the partition volumes across all coPapersDBLP runs. Comparing this with the percentage differences in Table 7.6 suggests a plausible explanation. The heavy-tailed degree distribution of citation networks makes it difficult to achieve a balanced, low-volume result when partitions are few. A handful of hub vertices end up dominating their host partition's outgoing volume, and without many options for moves NVOL is unable to redistribute this volume efficiently. In fact, as the number of partitions increases, it is easier to find optimal moves for such heavy hub vertices, in turn making the heavy-tailed structure easier to exploit. This does, however, cost a higher variance in the partition volumes which is still lower than VOL's.

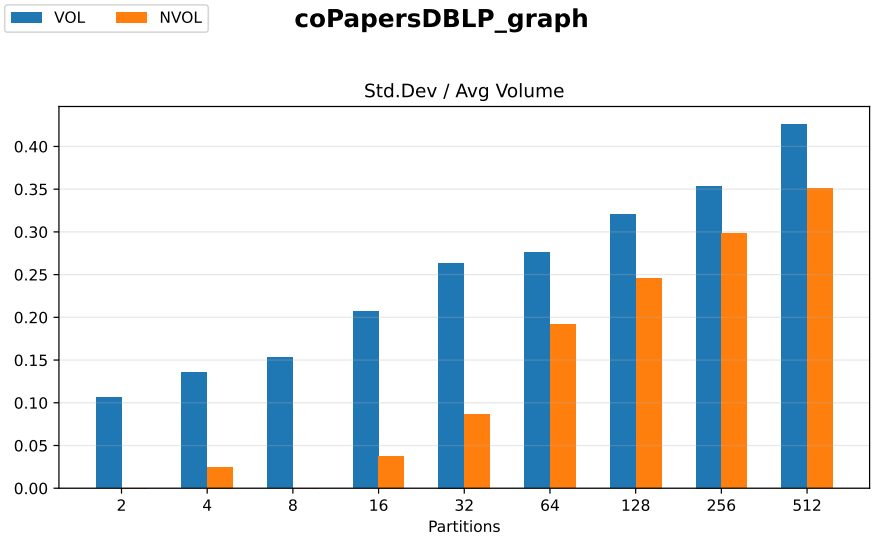


Figure 7.4: Comparison of the average normalized standard deviation on coPapersDBLP across different k for VOL (blue) and NVOL (orange).

Figure 7.5 shows an example that supports this hypothesis. At $k = 8$, NVOL achieves an almost perfectly balanced distribution with a standard deviation of 0.33 against VOL's 10,473. As with the heart04 case discussed earlier, this balance comes at the cost of pushing each partition's volume closer to each other. For this graph NVOL does not manage to improve $MV^{\max}$, losing by 8.82% to VOL, and as is visible in the figure, all the partitions are pushed closer to each other.

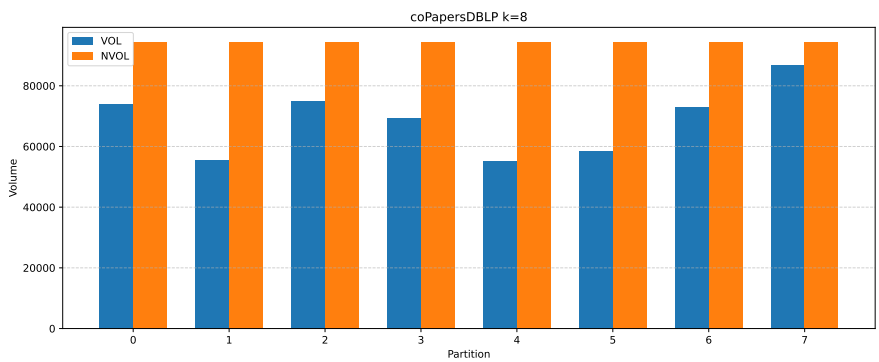


Figure 7.5: Comparison of the outgoing communication volumes of the partitions of coPapersDBLP $k = 8$ for VOL (blue) and NVOL (orange).

At high partition counts, NVOL's advantage on coPapersDBLP becomes substantial: $MV^{\max}$ improves by 40.37%, 48.44%, and 56.70% over VOL at $k = 128$, $k = 256$, and $k = 512$ respectively. Figure 7.6 illustrates the $k = 128$ case. NVOL's partition volumes cluster tightly around the mean while still achieving a significantly lower $MV^{\max}$ than VOL. The struggle at low k and these gains at high k are two sides of the same coin: the heavy-tailed degree distribution that prevents balancing with few partitions becomes a feature NVOL can exploit once k is large enough to spread the hubs across many partitions. Citation networks therefore appear to be a graph class where NVOL's balancing strategy has real potential, provided k is sufficiently high.

kmer_U1a and kmer_V2a graphs

Both kmer_U1a and kmer_V2a are protein k-mer graphs from the GenBank group of the SuiteSparse Matrix Collection, originally distributed as part of the 2018 GraphChallenge. Each vertex represents a short segment of amino acids (a k-mer) extracted from protein sequences, and edges connect k-mers that overlap in the underlying sequence. These are the two largest graphs in our selection, with kmer_U1a containing 67.7 million vertices and kmer_V2a 55 million. Both have high sparsity, with an average degree of approximately 2, and are highly fragmented into many small connected components. These graphs are the only ones marked N/R: NVOL was unable to finish partitioning them at $k = 2048$ due to their size. We discuss this limitation of NVOL compared to VOL, and ideas for addressing it, in Section 8.2.1.

Despite their structural similarity, partitioning the two graphs yields noticeably different partitioning results. On the larger instance, kmer_U1a, NVOL achieves consistent improvements over VOL on $MV^{\max}$, averaging 26.07% across all values of k. On kmer_V2a the gain is more modest: NVOL loses to VOL at $k = 2$ and $k = 128$, but still averages a positive 8.74%. On the total volume metric, however, NVOL fails to improve over VOL for both graphs, and the gap on kmer_V2a widens as k grows.

The two graphs also differ in how evenly NVOL distributes volume across partitions. On kmer_V2a, optimizing NVOL yields a higher run-to-run standard deviation than on kmer_U1a, where the standard deviation remains zero for all $k < 16$ and only begins to grow from $k = 16$ onward. Figures 7.7 and 7.8 show the total volume and standard deviation across all runs on kmer_V2a and kmer_U1a respectively.

A possible explanation for these differences lies in the disparity of fragmentation. Although the two graphs differ in size by only 18%, kmer_U1a contains 44,132 connected components against 15,343 in kmer_V2a, almost three times as many. The much larger number of small components in kmer_U1a may give the partitioner more flexibility when distributing volume across partitions, while in kmer_V2a the components are on average larger and harder to distribute as k grows, which is consistent with the higher standard deviation observed for both VOL and NVOL compared to kmer_U1a.

The combination of low standard deviation and improvement on $MV^{\max}$ is most evident in the high k runs of kmer_U1a. Figure 7.9 shows the per-partition outgoing-volume distribution of VOL and NVOL at $k = 128$. As in Figure 7.6, optimizing NVOL produces partitions with much more uniform volumes than VOL and visibly reduces $MV^{\max}$. The pattern becomes even more pronounced at higher values of k, but those plots are harder to read, so we show only $k = 128$ here.

Overall, the kmer graphs paint a nuanced picture of NVOL's behavior on high sparsity, highly fragmented inputs. On $MV^{\max}$, NVOL clearly outperforms VOL on average for both graphs. On the total volume, however, neither graph shows an improvement, indicating that NVOL's redistribution of volume loads comes at the cost

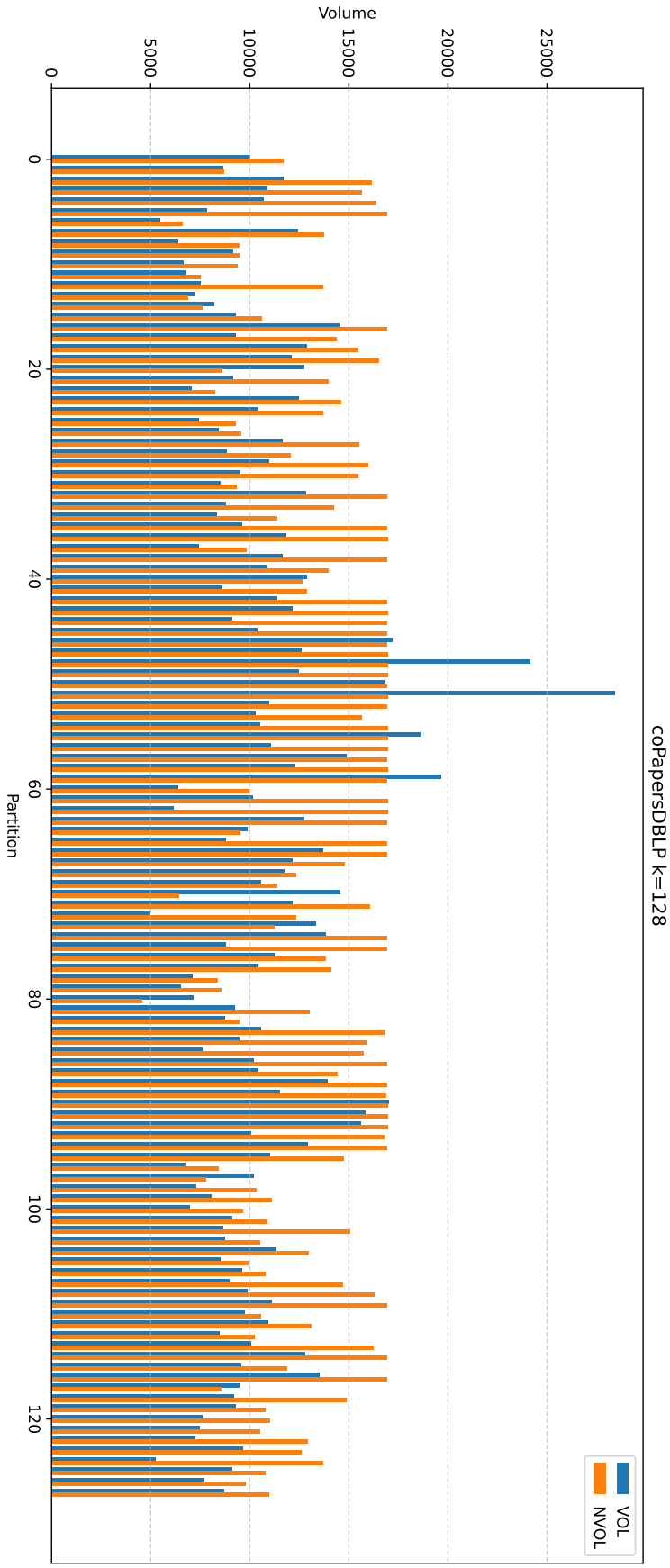


Figure 7.6: Comparison of the outgoing communication volumes of the partitions of coPapersDBLP $k = 128$ for VOL (blue) and NVOL (orange).

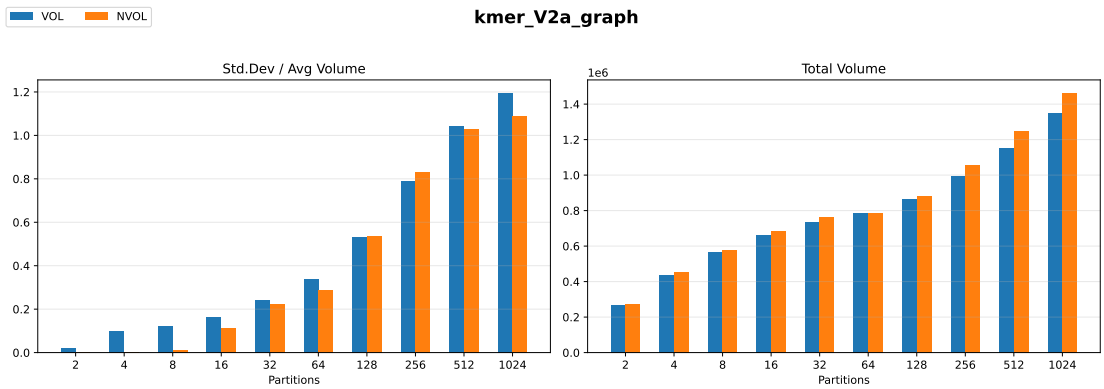


Figure 7.7: Comparison of the average normalized standard deviation and total communication volume on kmer_V2a across different k for VOL (blue) and NVOL (orange).

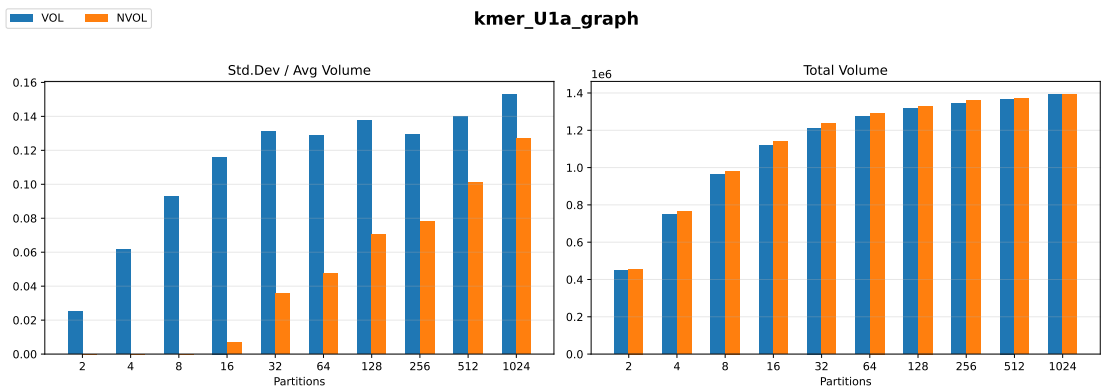


Figure 7.8: Comparison of the average normalized standard deviation and total communication volume on kmer_U1a across different k for VOL (blue) and NVOL (orange).

of a higher average volume per partition. The contrast between the two instances further suggests that fragmentation level matters: kmer_U1a, with nearly three times as many connected components, admits more stable and more consistent improvements, whereas the larger, fewer components of kmer_V2a constrain how the volume can be balanced as k grows. This uneven volume distribution is visible for both NVOL and VOL.

Numerical simulation graphs

The most represented group of graphs in our selection are numerical simulation graphs. These graphs are derived from physical domains, a chunk of fluid, a 3D box of space, a mechanical structure, by discretizing the domain into a mesh of small cells. Each vertex represents a cell, and edges connect cells that share a face or boundary in the mesh. This type of graph tends to be very regular: most vertices have a similar number of neighbors, and there are no hub vertices. Five of our graphs fall into this category: channel-500x100x100-b050, packing-500x100x100-b050, hugetric-00000, hugetric-00020, and hugetrace-00020.

NVOL performs consistently well on these graphs, improving $MV^{\max}$ in every single run (55 runs). As mentioned earlier, NVOL also beats VOL on total communication volume in several of these cases. Structurally these graphs are close cousins of the Heart dataset: both classes are originally regular meshes with low degree variance, which partly

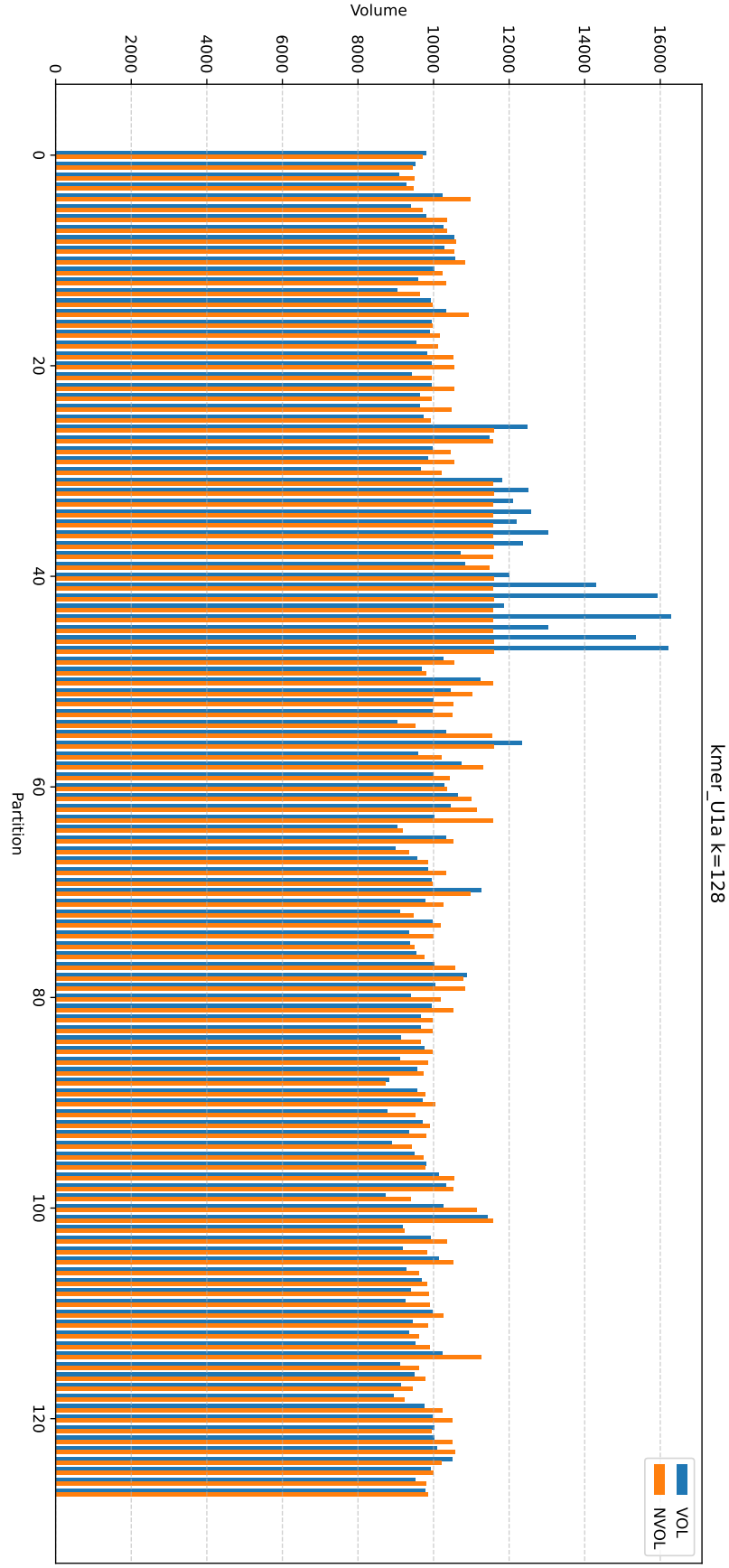


Figure 7.9: Comparison of the outgoing communication volumes of the partitions of kmer_U1a $k = 128$ for VOL (blue) and NVOL (orange).

explains why NVOL performs so well. The symmetry in these graphs lets NVOL easily find moves that improve the volume, as opposed to some of the previous examples.

Table 7.10 shows the percentage difference in total volume and standard deviation for hugetric-00000, the numerical simulation graph with the highest average improvement on $MV^{\max}$. We chose this graph specifically because it is representative of the class. A positive value indicates that NVOL achieves a lower value than VOL, following the same convention as Table 7.6. NVOL achieves a lower total volume in all runs, with an average improvement of 10.59%. NVOL also achieves a consistently better standard deviation, suggesting that the volumes are more evenly distributed across the partitions.

Partitions	**Percentage difference tot. vol.**	**Percentage difference std. dev.**
2	9.02%	0.00%
4	12.78%	12.85%
8	12.36%	9.46%
16	13.00%	16.67%
32	11.94%	14.47%
64	9.89%	4.41%
128	10.12%	14.73%
256	9.39%	14.72%
512	8.85%	14.59%
1024	10.77%	20.28%
2048	8.39%	18.09%

Table 7.10: Comparison of the percentage difference of total volume and standard deviation on hugetric-00000 for VOL and NVOL. A positive value means that optimizing NVOL produces a lower value than optimizing VOL, and vice versa.

On hugetric-00000, minimizing NVOL produces excellent partitions, achieving a lower total communication volume, a lower $MV^{\max}$, and a more even distribution than VOL. This pattern is also visible across the remaining numerical simulation graphs, making NVOL a valuable partitioning objective for this graph class.

Geometric graphs

The remaining graphs all fall under the geometric graph class. These graphs are derived from points in space rather than from physical adjacency or relationships. pa2010, for example, represents the Pennsylvania Census 2010 redistricting graph, with vertices as Census blocks and edges connecting blocks that share a border. The two delaunay graphs are synthetic Delaunay triangulations of random points in a 2D unit square, while rgg_n_2_23_s0 is a random geometric graph. All four have low degree variance with no hub vertices.

Because these graphs differ slightly from each other, they do not follow as clean a pattern as the numerical simulation graphs. They show broadly similar averages in Table 7.7. Figure 7.10 shows a plot of the values in Table 7.6. The three synthetic graphs average around 4-6%, while pa2010 sits notably above at nearly 9%.

pa2010 is structurally similar to italy_osm, yet NVOL performs well on it. The likely reason is that pa2010 covers a single US state, whose census blocks are far more uniform in size and density than the urban-rural-mountain extremes spanning all of Italy. Its average degree of 4 with low variance reflects this uniformity, giving NVOL the kind of regular structure it can balance effectively.

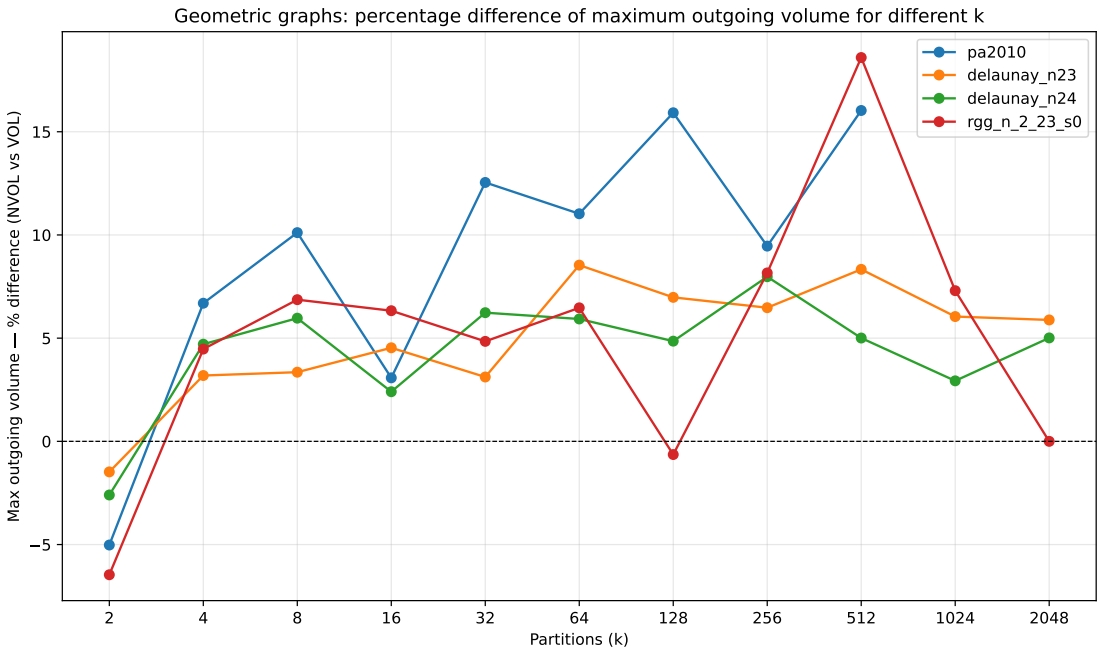


Figure 7.10: Percentage difference in $MV^{\max}$ between NVOL and VOL for our selection of geometric graphs across different k ($(MV^{\max}_{\text{VOL}} - MV^{\max}_{\text{NVOL}})/MV^{\max}_{\text{VOL}} \times 100$).

Both the best and worst results in this group come from rgg_n_2_23_s0, which records a loss of 6.46% at $k = 2$ and a win of 18.60% at $k = 512$. It is also the only graph with a loss outside $k = 2$, losing 0.64% at $k = 128$. As with heart05 at $k = 512$, there is no clear indication of why NVOL loses on this run. The partitions still have a lower standard deviation than VOL's, and their total volumes are very close. This appears to be once again a case of a succession of bad moves leading NVOL into a configuration it cannot escape.

On this class of graphs, NVOL and VOL seem to end up with similar numbers for total communication volume with either objective function achieving the slightly lower value. The standard deviation of the partition volumes is also very close in value, with NVOL regularly achieving a slightly lower standard deviation.

Geometric graphs are therefore a class where NVOL is reliably useful: it consistently lowers $MV^{\max}$, achieves total volumes very close to those of VOL, and yields a slightly lower standard deviation in most runs.

7.3 Tuning NVOL

METIS provides several options for tuning refinement. The one we found to have the largest impact on the results is the imbalance factor, which is set to 3% by default. Increasing this value permits greater imbalance in the partition weights, which in turn opens up moves that the default setting would have rejected. In this section we test NVOL at a 4% imbalance, one percentage point above the default. Although the change is small, it noticeably affects the resulting partitions. To assess how this affects NVOL as an objective function, we compare NVOL at 4% against two baselines: VOL at 3% (the default used in the previous section) and NVOL at 3%. This should make clear how a small change in imbalance influences the results.

7.3.1 Heart dataset with 4% imbalance

Table 7.11 shows the percentage difference between VOL and NVOL. Compared to Table 7.3, it is immediately clear that NVOL loses a few more times. In Table 7.3, NVOL loses 4 of 61 runs, as opposed to 11 of 61 here. It does, however, win a few more times by 10% or more. In a similar manner, the losses are also more significant, with five of them at 4% or above.

Graphs	2	4	8	16	32	64	128	256	512	1024	2048
heart01	-1.30	**2.08**	**16.49**	**5.19**	**18.18**	N/A	N/A	N/A	N/A	N/A	N/A
heart02	**0.57**	**2.92**	**3.41**	**12.55**	**4.62**	**6.76**	**15.45**	N/A	N/A	N/A	N/A
heart03	-1.46	-4.52	**2.37**	**4.08**	**6.68**	**6.27**	**18.01**	**8.08**	**11.07**	N/A	N/A
heart04	-4.99	**9.60**	**17.20**	**4.19**	**1.84**	**5.65**	**4.20**	**7.35**	**6.89**	N/A	N/A
heart05	**5.34**	-1.05	**3.11**	**6.64**	**8.75**	**7.24**	**8.93**	**0.00**	**5.75**	**2.60**	N/A
heart06	**2.75**	-2.86	**4.51**	-7.14	**0.35**	**9.89**	**9.77**	**2.82**	-0.93	**9.16**	N/A
heart07	-0.86	**4.79**	**2.75**	**2.83**	**4.12**	-4.42	**7.61**	**7.44**	**7.45**	-6.97	**5.40**

Table 7.11: Percentage difference in $MV^{\max}$ between NVOL 4% and VOL for the Heart dataset across different k. Each value is the result of $((MV^{\max}_{\text{VOL}} - MV^{\max}_{\text{NVOL}})/MV^{\max}_{\text{VOL}} \times 100)$. Cells marked with N/A were not run.

In Table 7.12 we compare the average results of each graph. For several graphs, the larger wins pull the average above NVOL with 3% despite the additional losses. NVOL with 4% works best on average on smaller graphs, as the average difference decreases as the graph size grows. This can possibly be explained by the fact that in smaller graphs, some moves are more restricted by the weight constraints. A move that decreases $MV^{\max}$ may not be accepted because the partition weight is too high. It is also visible in heart01, heart03, and heart04 that NVOL with 4% yields the best result compared to both VOL with 3% and NVOL with 3%.

This imbalance comes at a trade-off: when NVOL does lose, it loses by a larger margin. The worst for NVOL with 3% loss was only -1.30% in Table 7.3, while nine of the losses in Table 7.11 are larger in magnitude. This is very visible in heart06 and heart07, each registering three losses and dragging their averages below that of NVOL with 3%. Total volume and standard deviation are relatively close to each other in most runs for NVOL with 3% and NVOL with 4% imbalance.

Graphs	**NVOL 3% avg. diff.**	**NVOL 4% avg. diff.**	**NVOL 4% Best result**	**NVOL 4% Worst result**
heart01	4.13%	8.13%	18.18%	−1.30%
heart02	7.90%	6.61%	15.45%	0.57%
heart03	5.67%	5.62%	18.01%	−4.52%
heart04	3.75%	5.77%	17.20%	−4.99%
heart05	4.01%	4.73%	8.93%	−1.05%
heart06	4.54%	2.83%	9.89%	−7.14%
heart07	3.87%	2.74%	7.61%	−6.97%

Table 7.12: Average, best, and worst percentage difference in $MV^{\max}$ per graph for VOL and NVOL 4% (Heart dataset).

Interestingly, NVOL with 4% recovers an earlier loss on heart05 with $k = 512$. With the 4% imbalance, NVOL is able to perform 11,888 more moves compared to the default run. The looser restriction allows NVOL to escape the configuration it got stuck in

during the previous run, reaching a better result. This is the only case in the Heart dataset where the imbalance recovers an earlier loss.

On the Heart dataset, raising the imbalance to 4% helps small graphs and hurts large ones, though even on small graphs the individual losses are larger and more frequent than at the default imbalance. In the next section we will examine whether this pattern holds for the diverse SuiteSparse graphs.

7.3.2 SuiteSparse Matrix Collection with 4% imbalance

Table 7.13 shows the results on the selection of the SuiteSparse graphs when running NVOL with 4% imbalance. As with the Heart dataset, the spread of values widens in both directions. Runs with an improvement of 20% or more rise from 13 to 18, while the number of losses also grows from 15 to 18, with an average loss of −14.54%. The 4% imbalance yields two new extremes as well: a peak of 57.51% for coPapersDBLP at $k = 512$ and a low of −59.78% for italy_osm at $k = 16$.

Graphs	2	4	8	16	32	64	128	256	512	1024	2048
pa2010	-18.26	**11.85**	**13.48**	**3.69**	**3.14**	**14.07**	**12.44**	**11.49**	**19.08**	N/A	N/A
coPapersDBLP	-31.82	-17.75	-9.85	**1.09**	**10.57**	**22.58**	**40.18**	**49.04**	**57.51**	N/A	N/A
packing -500x100x100 -b050	-9.51	-0.76	-7.29	**11.83**	**12.33**	**5.62**	**5.46**	**14.52**	**3.51**	**11.76**	N/A
channel -500x100x100 -b050	**11.49**	**9.46**	**9.48**	**9.22**	**17.57**	**12.91**	**10.12**	**10.60**	-2.73	**9.92**	**5.18**
hugetric-00000	**16.01**	**14.29**	**6.68**	**15.72**	**13.17**	**4.04**	**8.69**	**16.14**	**12.94**	**12.08**	**21.60**
italy_osm	**4.41**	-6.12	-32.43	-59.78	**26.61**	**2.00**	**13.08**	**1.89**	**12.12**	-25.00	-22.86
hugetric-00020	**4.81**	**11.31**	**17.91**	**1.62**	**1.02**	**0.94**	**15.03**	**11.90**	**21.36**	**13.43**	**13.10**
rgg_n_2_23_s0	-4.55	**3.40**	**2.00**	**9.06**	**5.45**	**7.47**	**8.25**	**3.54**	**16.22**	**3.17**	**5.01**
delaunay_n23	-1.94	**5.67**	**3.07**	**6.61**	**4.88**	**4.24**	**8.32**	**11.99**	**10.83**	**7.67**	**6.47**
hugetrace-00020	**3.83**	**8.04**	**3.70**	**3.57**	**12.26**	**5.74**	**0.97**	**13.59**	**12.94**	**10.23**	**24.78**
delaunay_n24	**0.77**	**5.34**	**10.34**	**6.33**	**9.57**	**1.41**	**3.82**	**7.80**	**2.98**	-6.84	**1.37**
kmer_V2a	-2.14	**9.35**	**20.52**	**13.63**	**0.66**	**6.48**	-2.14	**6.54**	**23.22**	**21.48**	N/R
kmer_U1a	**0.53**	**6.59**	**14.77**	**26.60**	**29.53**	**35.46**	**29.07**	**49.69**	**33.61**	**34.63**	N/R

Table 7.13: Percentage difference in $MV^{\max}$ between NVOL 4% and VOL for the SuiteSparse Matrix Collection selection across different k. Each value is the result of $((MV^{\max}_{\text{VOL}} - MV^{\max}_{\text{NVOL}})/MV^{\max}_{\text{VOL}} \times 100)$. Cells marked with N/R and N/A were not run.

Comparing the per-graph averages in Table 7.14, most rows shift very little, typically within ±1%. The clearest exception is italy_osm, whose average drops from −0.96% to −7.83%. Best-case results improve on nearly every graph, usually by 1-3%, with italy_osm and hugetrace-00020 showing the largest single-run gains. The worst-case results, by contrast, deteriorate almost everywhere; only three graphs see an improvement at their worst tail: hugetric-00000, rgg_n_2_23_s0, and kmer_V2a. The greatest deterioration occurs on italy_osm, whose worst result collapses from −21.21% (see Table 7.6) to −59.78% (see Table 7.13). As in the Heart dataset, total volume and standard deviation differ only slightly between the two configurations.

Here too, NVOL with 4% manages to recover some earlier losses. The most striking case is italy_osm at $k = 512$, which moves from −21.21% to 12.12%. In total, four earlier losses are recovered: italy_osm at $k = 256$ and $k = 512$, rgg_n_2_23_s0 at $k = 128$, and delaunay_n24 at $k = 2$. On the other hand, seven previously winning results regress into losses, slightly fewer than the eight regressions in the Heart dataset.

Unlike in the Heart dataset, graph size does not appear to drive the difference between the two configurations here. The 4% imbalance mainly stretches the tails, pushing the

Graphs	NVOL 3% avg. diff.	NVOL 4% avg. diff.	NVOL 4% Best result	NVOL 4% Worst result
pa2010	8.87%	7.89%	19.08%	−18.26%
coPapersDBLP	12.56%	13.51%	57.51%	−31.82%
packing-500x100x100-b050	7.20%	4.75%	14.52%	−9.51%
channel-500x100x100-b050	9.64%	9.38%	17.57%	−2.73%
hugetric-00000	12.86%	12.85%	21.60%	4.04%
italy_osm	−0.96%	−7.83%	26.61%	−59.78%
hugetric-00020	10.59%	10.22%	21.36%	0.94%
rgg_n_2_23_s0	5.59%	5.37%	16.22%	−4.55%
delaunay_n23	5.00%	6.16%	11.99%	−1.94%
hugetrace-00020	10.13%	9.06%	24.78%	0.97%
delaunay_n24	4.34%	3.90%	10.34%	−6.84%
kmer_V2a	8.74%	9.76%	23.22%	−2.14%
kmer_U1a	26.07%	26.05%	49.69%	0.53%

Table 7.14: Average, best, and worst percentage difference in $MV^{\max}$ per graph for VOL and NVOL 4% (SuiteSparse Matrix Collection).

best results higher and the worst results lower, while leaving the per-graph averages largely unchanged. NVOL also continues to struggle on the road-network structure of italy_osm, where the loosened constraint allows even larger negative results.

The 4% setting is therefore best treated as a per-graph tuning option rather than a default: it can yield better results in some cases, but with no clear predictor of when it helps and when it hurts.

7.4 Communication times estimates

Thus far we have evaluated how NVOL performs based on outgoing communication volume metrics. However, the end goal of graph partitioning is to improve the communication time when running the partitions. In order to test if the lower volumes actually improved this, we used the model estimate described in Section 6.2 which produces estimates of the communication time among the partitions. We point out that this is simply a model, and not actual communication time, meaning that these values are an approximation that closely models the ones we would observe on a real machine.

We test our graphs for a setup with 32 processors with a startup latency and a set of network bandwidths we got sent by the authors of [46]. These values are realistic numbers recorded on actual hardware, and are therefore not a universal set. This means that the results would look different using different values from a different machine. Unfortunately, we were not able to retrieve the network bandwidth for a bigger machine in order to test some of our best results at higher k.

Since a parallel application is generally as slow as its slowest process, we start this evaluation by looking at this value first. On each graph we compare the slowest communication time estimate for each process on VOL and NVOL and calculate their percentage difference. If we define the communication time estimates as R, each value in the following Table 7.15 equals $\Delta R_{\text{NVOL}} = (R_{\text{VOL}} - R_{\text{NVOL}})/R_{\text{VOL}}$, where positive values indicate NVOL is faster and vice versa. We check this value for both NVOL with 3% imbalance, and NVOL with 4% imbalance to VOL with 3% imbalance, all at $k = 32$.

Graphs	$\Delta R_{\text{NVOL3\%}}$	$\Delta R_{\text{NVOL4\%}}$
heart01	0.0%	9.99%
heart02	0.6%	0.2%
heart03	1.1%	-8.6%
heart04	−0.1%	6.3%
heart05	0.4%	5.6%
heart06	6.6%	6.6%
heart07	3.2%	4.3%
pa2010	0.2%	−0.3%
coPapersDBLP	−29.4%	−17.97%
packing-500x100x100-b050	6.5%	4.6%
channel-500x100x100-b050	5.6%	5.7%
hugetric-00000	2.8%	3.6%
italy_osm	0.1%	−27.8%
hugetric-00020	2.0%	−8.8%
rgg_n_2_23_s0	−1.0%	−1.3%
delaunay_n23	−0.9%	−9.3%
hugetrace-00020	2.6%	−5.7%
delaunay_n24	−6.1%	0.1%
kmer_V2a	−3.7%	−2.9%
kmer_U1a	2.2%	2.1%

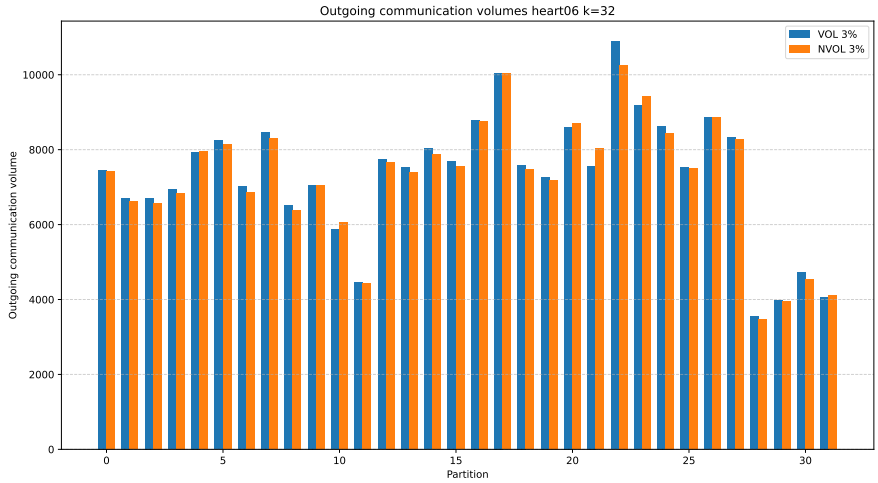
Table 7.15: Percentage difference for the slowest communication time estimate for NVOL 3% and NVOL 4% compared to VOL 3%(e.g. $\Delta R_{\text{NVOL}} = (R_{\text{VOL}} - R_{\text{NVOL}})/R_{\text{VOL}}$).

Table 7.15 shows how across both configurations, NVOL 3% wins 60% of the time. More specifically, using the default imbalance it has 13 wins, one tie and six losses, while the 4% imbalance runs have 11 wins and nine losses. NVOL with 4% maintains the same pattern (see Section 7.2.2) of having a greater variance in its results with a greater span between best and worst results. Both approaches struggle greatly for coPapersDBLP yielding slower runtimes. The Heart dataset and the numerical simulations record good results. For NVOL with 3% they are all positive, NVOL with 4% records a few losses here too.

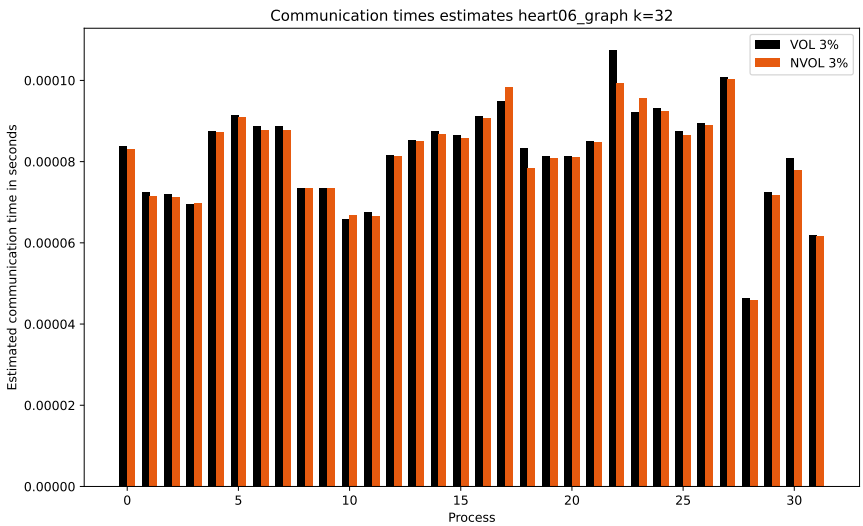
Among the larger graphs, the best result comes from heart06 (4 million vertices), where NVOL improves the slowest communication time by 6.6% (see Table 7.15). We focus on heart06 rather than heart01 because heart01 has only 1,340 vertices, and on graphs that small the startup latency of sending a message affects communication time more than volume does.

Figure 7.11a shows the outgoing communication volume per partition for NVOL at 3% and VOL at 3%. NVOL (orange) consistently has a lower volume across all partitions. The same pattern carries over to the communication time estimates in Figure 7.11b. This shows that for this graph optimizing NVOL not only improves for the partition with the highest volume, but generally across the whole graph.

The next subsections will discuss the two cases where NVOL produces significantly worse runtimes. As it will become clear, the runtime is not only dependent on the single maximum value, but also on other factors like total communication volume and which partitions have the highest number of neighbors.



(a) Comparison of the outgoing communication volumes of the partitions of heart06 $k = 32$ for VOL 3%(blue) and NVOL 3%(orange).



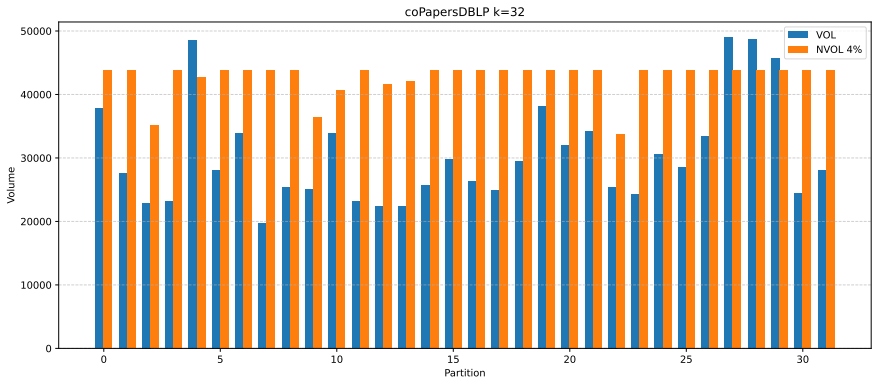
(b) Estimated communication time per partition on heart06 $k = 32$ for VOL 3%(black) and NVOL 3%(orange).

7.4.1 coPapersDBLP graph estimate results

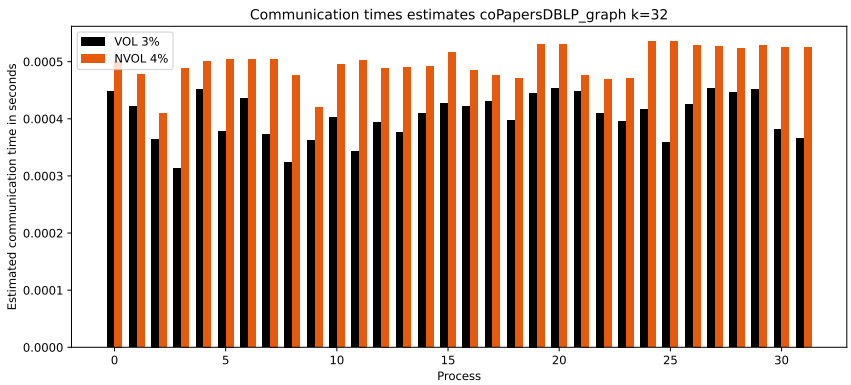
On coPapersDBLP, NVOL loses to VOL on estimated communication time despite achieving a lower $MV^{\max}$. Table 7.15 shows NVOL with 3% and NVOL with 4% trailing VOL with 3% by 29.4% and 17.97% on estimated communication time, while improving $MV^{\max}$ by 0.41% and 10.57% respectively (see Table 7.6 and Table 7.13).

Although NVOL achieves a lower $MV^{\max}$, its overall distribution is shifted upward: both the mean and the lighter partitions are larger than VOL's. This is clearly visible in Figure 7.12a, where NVOL with 4%(orange) has a visibly higher total volume. This higher average affects badly the communication time estimates. Figure 7.12b shows how NVOL with 4% has a slower estimated communication time than VOL with 3% in every process. We chose NVOL with 4% for this example because it achieves the larger

$MV^{\max}$ improvement of the two configurations. The result suggests that an improvement in $MV^{\max}$ alone may not be enough to reduce communication time if it comes at the cost of a much higher total communication volume.



(a) Comparison of the outgoing communication volumes of the partitions of coPapersDBLP $k = 32$ for VOL 3%(blue) and NVOL 4% (orange).

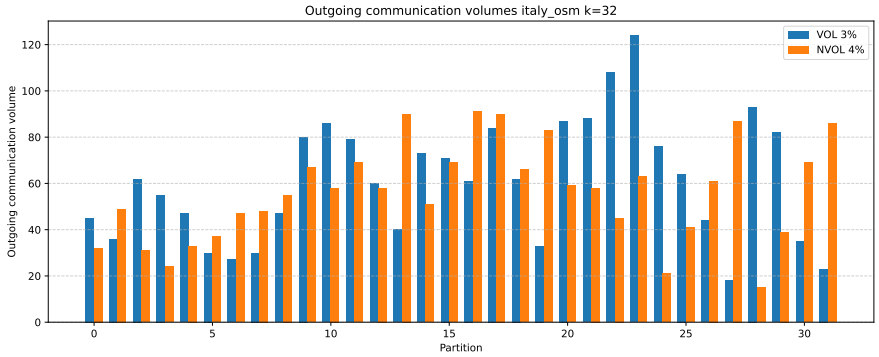


(b) Estimated communication time per partition on coPapersDBLP $k = 32$ for VOL 3%(black) and NVOL 4%(orange).

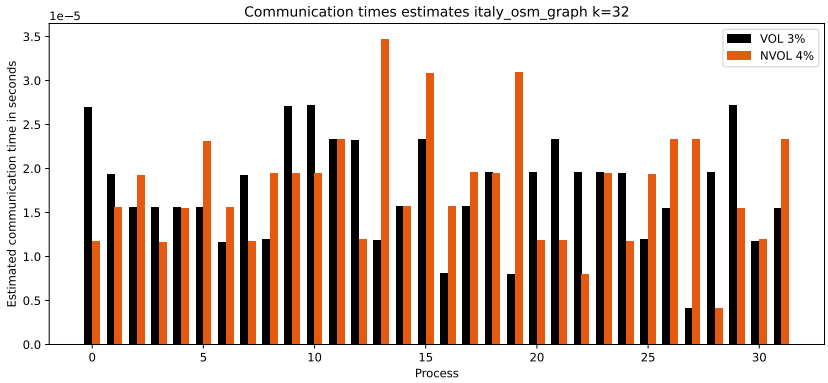
7.4.2 italy_osm graph estimate results

At first glance the results for this graph seem wrong. NVOL with 3% improves $MV^{\max}$ by 8.87% and NVOL with 4% imbalance by 26.61%, yet their slowest estimated communication times compared to VOL 3% score 0.1% and −27.8% respectively. The 4% variant essentially loses on communication time by roughly the same margin it gains on outgoing volume. To understand why, we need to look beyond $MV^{\max}$ and total communication volume. Here too, we use NVOL 4% as its values are the ones that seems the most contradictory.

Analyzing Figure 7.13a we can clearly see how NVOL 4% (orange) has a lower $MV^{\max}$. It has also a lower total communication volume of 1,927 compared to VOL's 1,950. Below, Figure 7.13b shows the estimated communication times, where NVOL 4% loses on many occasions.



(a) Comparison of the outgoing communication volumes of the partitions of italy_osm $k = 32$ for VOL 3%(blue) and NVOL 3%(orange).



(b) Estimated communication time per partition on italy_osm $k = 32$ for VOL 3%(black) and NVOL 3%(orange).

Since the volume values give no direct answer let's look in more detail at the model used to calculate these estimates. The estimates are calculated using Equation 6.2, which gives the communication time for partition i as the time needed to send and receive its messages, plus the startup latency for each outgoing message. Both partitionings have very low volumes overall, likely due to the high sparsity of road network graphs. This is reflected directly in the estimate: the largest t^{recv} (see Section 6.2.6) value is $0.69\mu s$ for VOL 3% and $0.56\mu s$ for NVOL 4%. So far, the volume advantage of NVOL 4% behaves as expected.

The latency term, however, tells a different story. The latency value measured by the authors of [46] is $3.8\,\mu s$, which by Equation 6.2 is multiplied by the number of neighboring partitions. This constant is already an order of magnitude larger than t^{recv}, so once it is scaled by the neighbor count it dominates the estimate. NVOL 4% has one partition with nine neighbors and two with eight, all above VOL's maximum of seven, and these three partitions stand out in Figure 7.13b.

In Figure 7.14 we show the resulting communication time estimates when ignoring the latency. Here it is clear that NVOL 4%'s wins on volume translate to lower communication time estimates.

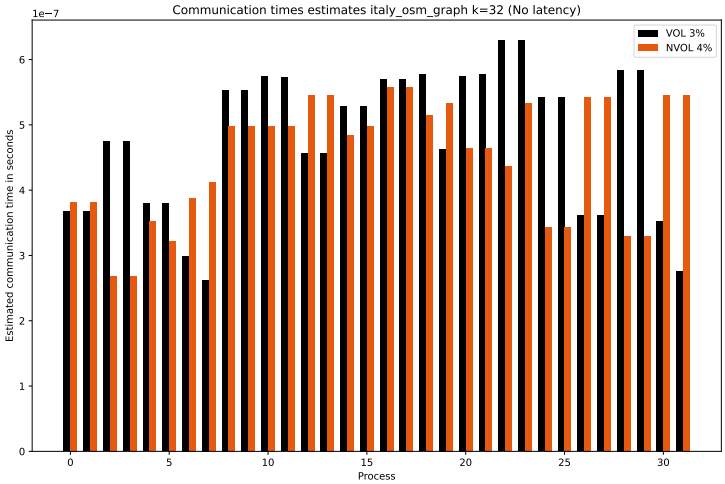


Figure 7.14: Estimated communication time per partition on italy_osm $k = 32$ for VOL 3%(black) and NVOL 3%(orange) when ignoring the latency.

This example illustrates that on graphs with very low communication volumes, latency can dominate the communication times, and a partitioning that improves volume at the cost of more neighbors per partition may end up slower overall. The specific numbers here are tied to our setup, but the broader pattern, that latency matters more in low-volume graphs, is general.

Chapter 8

Conclusions and Future Work

This chapter concludes the thesis, evaluating how well the goals we set in the introduction were reached. We look back at the results and evaluate the work done so far. In the final section we look at future work to be done on this project.

8.1 Conclusions

In Chapter 1 of this thesis we posed two research questions:

- **RQ1**: Can a partition-aware objective function produce more balanced communication loads than a global one?
- **RQ2**: Can the performance of a parallel application be improved in practice by using a partition-aware objective function?

8.1.1 Contributions

- **NVOL**, a partition-aware objective function that minimizes the maximum per-partition outgoing communication volume rather than the global total. Chapters 4 and 5 describe its design.
- An implementation of NVOL within the METIS framework, integrated into the refinement stage with the supporting gain-table data structures. Chapter 6 describes the adaptations needed to implement NVOL into METIS.
- An empirical study of NVOL against METIS's VOL objective across 20 graphs partitioned into 2 to 2048 parts, on both quality metrics and simulated runtime.

The remainder of this chapter evaluates these contributions against the two research questions.

8.1.2 RQ1 - Does NVOL improve the volume balance?

Our results show that NVOL generates partitions with volumes comparable to those produced by METIS when optimizing for total communication volume. NVOL focuses on minimizing the maximum outgoing volume ($MV^{\max}$) of any single partition, allowing moves that raise the volume elsewhere as long as the maximum is not increased. This consistently yields partitions with a lower maximum and higher minimum outgoing volume compared to VOL (METIS's total communication volume objective function).

The more even distribution is reflected in the lower variance in outgoing volumes, especially in runs with 2 partitions, where NVOL produces perfectly balanced partitions on almost all graphs.

On more regular graphs, NVOL consistently improves on $MV^{\max}$. The numerical simulation graphs and the cardiac graphs are particularly notable, with NVOL achieving a lower $MV^{\max}$ than VOL in almost every run. On these graphs NVOL also reduces the spread of outgoing volumes across partitions, producing more uniform partition volumes. Notably, on the cardiac graphs and a few others, NVOL even achieves a lower total communication volume than VOL despite not optimizing for that metric.

These results are not universal: NVOL clearly struggles on certain graph structures. Road network graphs in particular proved challenging in our tests, this was the only graph type where NVOL failed to improve on $MV^{\max}$ in as many as 45% of runs. These graphs are sparse, with a few high-degree vertices and many low-degree ones, and this irregular structure prevents NVOL from producing consistent improvements.

The largest graph we managed to test NVOL on was kmer_U1a, a protein k-mer graph with 67.7 million vertices. This graph is also sparse, with many small, highly connected components. NVOL nevertheless produces a better $MV^{\max}$ than VOL and consistently lower standard deviations in the volume values, reaching a reduction in $MV^{\max}$ of up to 46.34%. This suggests that NVOL performs well when given many options, and raises an interesting question: how well would NVOL perform on even larger graphs? On the more negative side, this graph was already too large for NVOL to partition into 2048 parts, making efficiency improvements a priority before scaling further.

This point about NVOL performing better when it has many options is also supported by the results for coPapersDBLP, a graph with a wide variance in edges per vertex. This structure is common in citation networks, which have a few high-degree hub vertices and many low-degree ones. NVOL struggles considerably at low k, with the worst result at -32.09%. The trend reverses as k grows, however, peaking at an improvement of 56.70%.

We also tested NVOL with imbalance tuning, which produced more variable results. Allowing greater imbalance introduced more instability, with several additional runs failing to improve on $MV^{\max}$, though NVOL maintained a positive win rate overall. The main effect of relaxed imbalance appears to be a larger magnitude in both wins and losses. The high variance of these results suggests that imbalance tuning should be treated as a per-run option rather than a default setting.

Taken together, these results answer the first question affirmatively: NVOL produces partitions with more uniform volumes and lower variance compared to METIS's volume objective function. This does push the average partition volume higher, but the increase comes primarily from redistributing the spikes that previously concentrated on a few partitions. NVOL improves the maximum outgoing volume in the partitions, achieving results up to 56.70% compared to METIS's total volume objective.

8.1.3 RQ2 - Do the partitions produced by NVOL improve performance?

The answer to this question is yes, but with an asterisk. NVOL achieves better runtimes than VOL in 65% of our tests, and maintains a 60% win rate when imbalance tuning is enabled. On its wins, NVOL improves runtime by an average of 2.61%, peaking at 6.6% (without imbalance tuning).

Some results, such as those for italy_osm, also show that latency can dominate volume in certain settings. The time spent transmitting the data itself is negligible compared to the latency of initiating each message. As a result, on italy_osm the tuned

NVOL produced a runtime 27.8% slower than VOL, even though it achieved a 26.61% lower $MV^{\max}$. The slowdown came purely from an increase in the number of partition neighbors: with more partitions to message, the per-message latency dominated the overall runtime.

The coPapersDBLP graph highlights another issue: by producing a higher average volume, NVOL slows down the more heavily loaded processes. Other processes must then wait for them to finish communicating, so the increased per-partition volume affects the overall runtime negatively. This suggests that NVOL will struggle on irregular graphs such as road and citation networks, particularly when they are partitioned into a small number of parts.

These results come with important caveats. We were only able to test on an idealized setup with 32 processors, which significantly limited the number of configurations we could explore. Most of NVOL's greatest improvements appear on partitionings into a large number of parts. Furthermore, these values are only estimates, providing no definitive confirmation that NVOL's runtimes are actually faster on real hardware.

In summary, NVOL does appear to improve efficiency in many cases, but these improvements remain simulated. Future work would require additional testing on real machines to validate these findings. Even so, the estimates we do have are promising and suggest that NVOL's partitions could lead to meaningful performance improvements, especially on graphs partitioned into many parts.

8.2 Future Work

NVOL shows good potential, delivering consistently strong results across a wide variety of graphs. The conclusions above point out two natural directions for future work: reducing the partitioning time overhead introduced by NVOL, and extending the objective function with hardware-aware information to better target real parallel machines.

8.2.1 Improved efficiency

One of METIS's main strengths is its partitioning speed, and the additional structures and operations introduced by NVOL inevitably affect this, particularly on larger graphs. For small graphs the partitioning time is essentially unchanged, but on larger ones NVOL was observed to be significantly slower than VOL.

The overhead has a straightforward explanation: NVOL adds code to METIS's refinement stage and increases the number of iterations performed there. METIS itself is carefully designed to avoid redundant work, most of the volume operations are performed during graph projection. This way the refinement stage can reuse it rather than reiterating over the graph. NVOL breaks this pattern by recomputing the gain information for each vertex it visits during refinement, effectively repeating work that METIS already performs at projection time.

Three considerations led us not to integrate NVOL's gain calculation directly into the projection stage. First, NVOL has a larger memory requirement than VOL. As described in Chapter 4, VOL requires a vector whose length equals the number of partitions a vertex neighbors, whereas NVOL requires a table. For a vertex with a gain vector of length k, the corresponding gain table has dimensions $(k+1) \times k$, so a gain vector of length 5 already implies a 6×5 table per vertex. Second, VOL uses incremental updates between moves to keep the volume gains of affected vertices current; implementing the

equivalent logic for NVOL was outside the scope of this thesis. Third, and perhaps most decisive, time constraints during development, combined with the considerable effort required to address the first two points, prevented us from pursuing this integration further.

This decision was nonetheless preceded by an attempt at the integration. Our strategy allocated the exact memory needed for each gain table and deallocated it once the corresponding vertex left the boundary; gain updates were handled by simply recomputing all neighbors of the moved vertex. In principle this is much closer to what VOL does and should have improved performance. In practice, however, the algorithm became even slower than the original NVOL implementation, with the frequent allocation and deallocation of memory as the dominant cost. A better solution would be to allocate a single block of memory at projection time, sized from an estimate of how many boundary vertices the finer graph will contain, with additional logic to extend the block when the estimate falls short and to release it once refinement completes. Combined with proper incremental updates of neighboring vertices' gains after each move, this would allow NVOL to partition in a time frame close to VOL, at the cost of higher memory use.

8.2.2 Introduce hardware knowledge

Using the partition-specific implementation of NVOL, it would be interesting to incorporate some of the logic presented in [46] to further improve actual runtime. Currently, we partition using communication volume and then model the runtime of each partition based on a set of constants and the incoming volume. With the right adaptations, NVOL could already evaluate this during refinement. Changing NVOL's volume calculations from outgoing to incoming volume is non-trivial, but it has the potential for even larger improvements.

The general idea would be to extend METIS to accept additional parameters describing the latencies and bandwidth of the machine running the partitions. Then, during refinement, one could use the equations shown in Section 6.2.6 to evaluate candidate moves. This would increase the chance of producing a partitioning that performs well on the target parallel machine. NVOL's per-partition structure is essential to such an approach.

This could be extended further to optimize for heterogeneous machines. So far we have only considered tests on machines where each component was identical, but in reality modern parallel computers are larger and more complex. Their heterogeneous structure therefore brings an added level of complexity that can be exploited. In [46], the authors also develop models for such setups. With the right structures in place, NVOL could support this logic as well.

Bibliography

[1] Stephen T. Barnard and Horst D. Simon. 'Fast multilevel implementation of recursive spectral bisection for partitioning unstructured problems'. In: *Concurrency: Practice and Experience* 6.2 (1994), pp. 101–117. DOI: https://doi-org.ezproxy.uio.no/10.1002/cpe.4330060203.

[2] Erik G. Boman, Karen D. Devine and Sivasankaran Rajamanickam. 'Scalable matrix computations on large scale-free graphs using 2D graph partitioning'. In: *SC '13: Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis*. 2013, pp. 1–12. DOI: 10.1145/2503210.2503293.

[3] Aydın Buluç, Kamesh Madduri, Henning Meyerhenke, Peter Sanders, David A. Bader and Dorothea Wagner. 'Graph partitioning for scalable distributed graph computations'. In: *Graph Partitioning and Graph Clustering*. Vol. 588. Contemporary Mathematics. American Mathematical Society, 2013, pp. 83–101. DOI: 10.1090/conm/588/11709.

[4] Aydın Buluç, Henning Meyerhenke, Ilya Safro, Peter Sanders and Christian Schulz. 'Recent Advances in Graph Partitioning'. In: *Algorithm Engineering: Selected Results and Surveys*. Springer International Publishing, 2016, pp. 117–158. DOI: 10.1007/978-3-319-49487-6_4. URL: https://doi.org/10.1007/978-3-319-49487-6_4.

[5] U.V. Catalyurek and C. Aykanat. 'Hypergraph-partitioning-based decomposition for parallel sparse-matrix vector multiplication'. In: *IEEE Transactions on Parallel and Distributed Systems* 10.7 (1999), pp. 673–693. DOI: 10.1109/71.780863.

[6] Ümi˙t V. Çatalyürek, Cevdet Aykanat and Bora Uçar. 'On Two-Dimensional Sparse Matrix Partitioning: Models, Methods, and a Recipe'. In: *SIAM Journal on Scientific Computing* 32.2 (2010), pp. 656–683. DOI: 10.1137/080737770.

[7] Ümit Çatalyürek et al. 'More Recent Advances in (Hyper)Graph Partitioning'. In: 55.12 (2023). DOI: 10.1145/3571808.

[8] Ümit V. Çatalyürek, Mehmet Deveci, Kamer Kaya, Bora Uçar, Henning Meyerhenke, Peter Sanders, David A. Bader and Dorothea Wagner. 'UMPa: A multi-objective, multi-level partitioner for communication minimization'. In: *Graph Partitioning and Graph Clustering*. Vol. 588. Contemporary Mathematics. American Mathematical Society, 2013, pp. 53–65. DOI: 10.1090/conm/588/11704.

[9] P.K. Chan, M.D.F. Schlag and J.Y. Zien. 'Spectral K-way ratio-cut partitioning and clustering'. In: *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 13.9 (1994), pp. 1088–1096. DOI: 10.1109/43.310898.

[10] Jingsheng Jason. Cong and Joseph R. Shinnerl. *Multilevel Optimization in VLSICAD*. 1st ed. 2003. Combinatorial Optimization. Springer US, 2003. DOI: 10.1007/978-1-4757-3748-6.

[11] Timothy A. Davis and Yifan Hu. 'The university of Florida sparse matrix collection'. In: *ACM Trans. Math. Softw.* 38.1 (Dec. 2011). DOI: 10.1145/2049662.2049663. URL: https://sparse.tamu.edu/.

[12] Ralf Diekmann, Robert Preis, Frank Schlimbach and Chris Walshaw. 'Shape-optimized mesh partitioning and load balancing for parallel adaptive FEM'. In: *Parallel Computing* 26.12 (2000). Graph Partitioning and Parallel Computing, pp. 1555–1581. ISSN: 0167-8191. DOI: https://doi.org/10.1016/S0167-8191(00)00043-0.

[13] Executive Office of the President. *Executive Order 13702—Creating a National Strategic Computing Initiative*. Federal Register, Vol. 80, No. 149, pp. 46177–46180. July 2015. URL: https://www.federalregister.gov/documents/2015/08/03/2015-19183/creating-a-national-strategic-computing-initiative (visited on 13/04/2026).

[14] C.M. Fiduccia and R.M. Mattheyses. 'A Linear-Time Heuristic for Improving Network Partitions'. In: *19th Design Automation Conference*. 1982, pp. 175–181. DOI: 10.1109/DAC.1982.1585498.

[15] Miroslav Fiedler. 'A property of eigenvectors of nonnegative symmetric matrices and its application to graph theory'. In: *Czechoslovak mathematical journal* 24.4 (1975), pp. 619–633. DOI: 10.21136/cmj.1975.101357.

[16] Santo Fortunato. 'Community detection in graphs'. In: *Physics Reports* 486.3 (2010), pp. 75–174. DOI: https://doi.org/10.1016/j.physrep.2009.11.002.

[17] M.R. Garey, D.S. Johnson and L. Stockmeyer. 'Some simplified NP-complete graph problems'. In: *Theoretical Computer Science* 1.3 (1976), pp. 237–267. DOI: https://doi.org/10.1016/0304-3975(76)90059-1.

[18] Bruce Hendrickson. 'Graph partitioning and parallel solvers: Has the emperor no clothes?' In: *Solving Irregularly Structured Problems in Parallel*. Springer Berlin Heidelberg, 1998, pp. 218–225. DOI: 10.1007/BFb0018541.

[19] Bruce Hendrickson and Tamara G Kolda. 'Graph partitioning models for parallel computing'. In: *Parallel Computing* 26.12 (2000). Graph Partitioning and Parallel Computing, pp. 1519–1534. DOI: https://doi.org/10.1016/S0167-8191(00)00048-X.

[20] Bruce Hendrickson and Robert Leland. 'A multilevel algorithm for partitioning graphs'. In: *Proceedings of the 1995 ACM/IEEE Conference on Supercomputing*. Supercomputing '95. ACM, 1995, pp. 28–42. DOI: 10.1145/224170.224228.

[21] Bruce Hendrickson and Robert Leland. 'An Improved Spectral Graph Partitioning Algorithm for Mapping Parallel Computations'. In: *SIAM Journal on Scientific Computing* 16.2 (1995), pp. 452–469. DOI: 10.1137/0916028.

[22] John L. Hennessy and David A. Patterson. 'A new golden age for computer architecture'. In: *Commununications of the ACM* 62.2 (2019), pp. 48–60. DOI: 10.1145/3282307.

[23] Costas Iliopoulos, Jason T. L Wang, Mourad Elloumi and Albert Y Zomaya. 'Wiley Series on Bioinformatics: Computational Techniques and Engineering'. In: *Pattern Recognition in Computational Molecular Biology*. John Wiley & Sons, Incorporated, 2015. DOI: 10.1002/9781119078845.scard.

[24] Andrew B. Kahng, Jens. Lienig, Igor L. Markov and Jin. Hu. *VLSI Physical Design: From Graph Partitioning to Timing Closure*. 1st ed. 2011. Springer Netherlands, 2011. DOI: 10.1007/978-90-481-9591-6.

[25] G. Karypis and V. Kumar. 'Analysis of Multilevel Graph Partitioning'. In: *Supercomputing '95:Proceedings of the 1995 ACM/IEEE Conference on Supercomputing*. 1995, pp. 29–29. DOI: 10.1145/224170.224229.

[26] George Karypis. *METIS: A Software Package for Partitioning Unstructured Graphs, Partitioning Meshes, and Computing Fill-Reducing Orderings of Sparse Matrices*. Version 5.1.0. 2013. URL: https://github.com/KarypisLab/METIS/blob/master/manual/manual.pdf.

[27] George Karypis and Vipin Kumar. 'A Fast and High Quality Multilevel Scheme for Partitioning Irregular Graphs'. In: *SIAM Journal on Scientific Computing* 20.1 (1998), pp. 359–392. DOI: 10.1137/S1064827595287997.

[28] George Karypis and Vipin Kumar. 'Multilevel k-way Partitioning Scheme for Irregular Graphs'. In: *Journal of Parallel and Distributed Computing* 48.1 (1998), pp. 96–129. DOI: https://doi.org/10.1006/jpdc.1997.1404.

[29] George Karypis and Vipin Kumar. 'Parallel Multilevel series k-Way Partitioning Scheme for Irregular Graphs'. In: *SIAM Review* 41.2 (1999), pp. 278–300. DOI: 10.1137/S0036144598334138.

[30] George Karypis and Vipin Kumar. 'Parallel Multilevel series k-Way Partitioning Scheme for Irregular Graphs'. In: *SIAM Review* 41.2 (1999), pp. 278–300. DOI: 10.1137/S0036144598334138.

[31] B. W. Kernighan and S. Lin. 'An efficient heuristic procedure for partitioning graphs'. In: *The Bell System Technical Journal* 49.2 (1970), pp. 291–307. DOI: 10.1002/j.1538-7305.1970.tb01770.x.

[32] Johannes Langguth, Sebastian Schlag and Christian Schulz. 'Load-Balanced Bottleneck Objectives in Process Mapping'. In: (2020). DOI: 10.48550/arXiv.2001.09645.

[33] Hao Li, G.W. Rosenwald, J. Jung and Chen-ching Liu. 'Strategic Power Infrastructure Defense'. In: *Proceedings of the IEEE* 93.5 (2005), pp. 918–933. DOI: 10.1109/JPROC.2005.847260.

[34] Juan Li and Chen-Ching Liu. 'Power system reconfiguration based on multilevel graph partitioning'. In: *2009 IEEE Bucharest Power Tech*. IEEE, 2009, pp. 1–5. DOI: 10.1109/PTC.2009.5281971.

[35] H Martinez-Navarro, B Rodriguez, A Bueno-Orovio and A Minchole. *Repository for modelling acute myocardial ischemia: simulation scripts and torso-heart mesh*. 2019. URL: https://ora.ox.ac.uk/objects/uuid:951b086c-c4ba-41ef-b967-c2106d87ee06.

[36] Irene Moulitsas, George Karypis, Anu G Bourgeois and Si Quing Zheng. 'Architecture Aware Partitioning Algorithms'. In: *Algorithms and Architectures for Parallel Processing*. Vol. 5022. Lecture Notes in Computer Science. Springer Berlin / Heidelberg, 2008, pp. 42–53. DOI: 10.1007/978-3-540-69501-1_6.

[37] M. E. J. Newman. 'Community detection and graph partitioning'. In: *Europhysics Letters* 103.2 (2013). DOI: 10.1209/0295-5075/103/28003.

[38] M. E. J. Newman. *Networks: An Introduction*. Oxford University Press, 2010. DOI: 10.1093/acprof:oso/9780199206650.001.0001.

[39] Joel Nishimura and Johan Ugander. 'Restreaming graph partitioning: simple versatile algorithms for advanced balancing'. In: *Proceedings of the 19th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. KDD '13. ACM, 2013, pp. 1106–1114. DOI: 10.1145/2487575.2487696.

[40] Fareed Qararyah, Mohamed Wahib, Doğa Dikbayır, Mehmet Esat Belviranli and Didem Unat. 'A computational-graph partitioning method for training memory-constrained DNNs'. In: *Parallel Computing* 104-105 (2021), pp. 1–12. DOI: https://doi.org/10.1016/j.parco.2021.102792.

[41] Semih Salihoglu, Jennifer Widom, Magdalena Balazinska, Alex Szalay, Ahmet Sacan, Tamas Budavari and Alexandra Meliou. 'GPS: a graph processing system'. In: *Proceedings of the 25th International Conference on Scientific and Statistical Database Management*. ACM, 2013, pp. 1–12. DOI: 10.1145/2484838.2484843.

[42] Horst D. Simon and Shang-Hua Teng. 'How Good is Recursive Bisection?' In: *SIAM Journal on Scientific Computing* 18.5 (1997), pp. 1436–1445. DOI: 10.1137/S1064827593255135.

[43] Xianfeng Song, Yi Zou and Zheng Shi. 'CaPGNN: Optimizing parallel graph neural network training with joint caching and resource-aware graph partitioning'. In: *Neurocomputing (Amsterdam)* 675 (2026), pp. 1–15. DOI: 10.1016/j.neucom.2026.132978.

[44] Isabelle Stanton and Gabriel Kliot. 'Streaming graph partitioning for large distributed graphs'. In: *Proceedings of the 18th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. KDD '12. ACM, 2012, pp. 1222–1230. DOI: 10.1145/2339530.2339722.

[45] Herb Sutter and James Larus. 'Software and the concurrency revolution'. In: *ACM Queue* 3.7 (2005), pp. 54–62. DOI: 10.1145/1095408.1095421.

[46] Andreas Thune, Sven-Arne Reinemo, Tor Skeie and Xing Cai. 'Detailed Modeling of Heterogeneous and Contention-Constrained Point-to-Point MPI Communication'. In: *IEEE Transactions on Parallel and Distributed Systems* 34.5 (2023), pp. 1580–1593. DOI: 10.1109/TPDS.2023.3253881.

[47] Charalampos Tsourakakis, Christos Gkantsidis, Bozidar Radunovic and Milan Vojnovic. 'FENNEL: streaming graph partitioning for massive scale graphs'. In: *Proceedings of the 7th ACM International Conference on Web Search and Data Mining*. WSDM '14. ACM, 2014, pp. 333–342. DOI: 10.1145/2556195.2556213.

[48] C. Walshaw, M. Cross and M.G. Everett. 'Parallel Dynamic Graph Partitioning for Adaptive Unstructured Meshes'. In: *Journal of Parallel and Distributed Computing* 47.2 (1997), pp. 102–108. DOI: https://doi.org/10.1006/jpdc.1997.1407.

[49] David P. Williamson and David B. Shmoys. 'An Introduction to Approximation Algorithms'. In: *The Design of Approximation Algorithms*. Cambridge University Press, 2011, pp. 3–26.